OpenMV 视觉套件

使用手册



2025 年 5 月

序言

尊敬的客户：

感谢您选用众灵科技的 OpenMV 视觉套件，本视觉套件会对 OpenMV H7 及其相关板载功能及视觉功能进行讲解；本套件中也会讲解到 OpenMV H7 PLUS 相关的示例代码，如果您购买了 OpenMV H7 PLUS，就可以使用 OpenMV H7 PLUS 的相关示例代码了；本套件中还会对我们的扩展板及其对舵机、电机的控制等进行讲解，如果您购买的套餐中包含了我们的扩展板，则可以兼容使用我们店里的所有总线设备来进行各种开发；本套件的拓展篇中还会有我们的语音控制、微信小程序控制、巡线小车等的讲解。

OpenMV 摄像头是一款小巧，低功耗，低成本的电路板，它帮助你很轻松的完成机器视觉（machine vision）应用。你可以通过高级语言 Python 脚本（准确的说是 MicroPython），而不是 C/C++。Python 的高级数据结构使你很容易在机器视觉算法中处理复杂的输出。但是，你仍然可以完全控制 OpenMV，包括 I0 引脚。你可以很容易的使用外部终端触发拍摄或者或者执行算法，也可以把算法的结果用来控制 IO 引脚。

OpenMV 可以用来进行颜色识别，形状识别，矩形识别，圆形识别，机器人巡线，直线识别，人脸识别，边缘检测，连通域检测，光流，人眼追踪，模板匹配，特征点追踪，二维码识别，瞳孔检测，条形码识别，矩形码识别，AprilTag 目标跟踪，绘图写字，帧差异，录制视频等。

在本教程中，我们使用的是 OpenMV H7 R1。在给 OpenMV 编写程序时，使用的是 openmv ide 软件。该软件是官方提供的 IDE 编译器，编程、调试、可视化界面一应俱全，还配套有全面的开发调试手册!

在基础篇中我们会对 OpenMV 及扩展板进行讲解，并对其板载功能进行讲解。在进阶篇中我们会对 OpenMV 的视觉功能进行讲解。在拓展篇中我们会对 OpenMV 进行拓展功能的讲解，包括用语音控制和微信小程序控制 OpenMV 小车及巡线小车。

再次感谢您的支持，祝您学有所成。

声明

本着为用户提供更好服务的原则，杭州众灵科技有限公司（以下简称“众灵”）在本手册中将尽可能地为用户呈现详实、准确的产品信息。但介于本手册的内容具有一定的时效性，众灵不能完全保证该文档在任何时段的时效性与适用性。众灵有权在没有通知的情况下对本手册上的内容进行更新，恕不另行通知。为了得到最新版本的信息，您可以定时访问众灵官方网站或者与众灵工作人员联系。感谢您的包容与支持！

众灵科技

警告

电路板相关部分，避免接触金属物体或人体，否则容易造成短路或者静电击穿。

目录

序言 2

声明 3

警告 3

基 础 篇 7

第 1 课 认识 OpenMV 8

第 2 课 认识扩展板 14

第 3 课 OPENMV IDE 的安装 16

第 4 课 OPENMV IDE 的使用 21

第 5 课 升级固件 25

第 6 课 SD 内存卡的使用 27

第 7 课 脱机运行 29

第 8 课 测距 31

第 9 课 测量物体大小 39

第 10 课 串口收发数据 46

第 11 课 PWM 舵机的控制 48

第 12 课 总线舵机的控制 50

第 13 课 马达的控制 53

进 阶 篇 56

第 1 课 颜色识别 57

第 2 课 颜色跟随 64

第 3 课 人脸识别 67
第 4 课 人脸跟随 69
第 5 课 分辨不同人脸 72
第 6 课 线段识别 77
第 7 课 直线识别 79
第 8 课 圆形识别 81
第 9 课 矩形识别 83
第 10 课 AprilTag 标签识别 85
第 11 课 二维码识别 87
第 12 课 条形码识别 89
第 13 课 数字识别（只适用于 openmv4 h7 plus） 92
第 14 课 口罩识别（只适用于 openmv4 h7 plus） 108
第 15 课 综合例程 111
拓 展 篇 117
第 1 课 垃圾分类（只适用于 openmv4 h7 plus） 118
第 2 课 追小球的小车 121
第 3 课 巡线小车 125
第 4 课 微信小程序控制 128
第 5 课 语音交互 138
项 目 篇 144
第 1 课 识别颜色并与 arduino 串口通信夹取木块 145
附录： 151

售后服务： 151

众灵科技

基础篇

第 1 课 认识 OpenMV

【本课目标】：

本课了解 OpenMV 及其板载资源

【课前准备】：

OpenMV

【本课内容】：

一 、OpenMV 简介:



OpenMV 摄像头是一款小巧，低功耗，低成本的电路板，它帮助你很轻松的完成机器视觉（machine vision）应用。你可以通过高级语言 Python 脚本（准确的说是 MicroPython），而不是 C/C++。Python 的高级数据结构使你很容易在机器视觉算法中处理复杂的输出。但是，你仍然可以完全控制 OpenMV，包括 I0 引脚。你可以很容易的使用外部终端触发拍摄或者或者执行算法，也可以把算法的结果用来控制 IO 引脚。

它的基于开源硬件计算机视觉摄像头模块，解决图像识别编程难，开发门槛高等问题。具有操作简单，编程高效，上手快等优.非常适用初学者适用即使你是刚接触编程的初学者，采用提供的操作教程及功能库，无需老师指导，即可投入到竞赛课题及项目实验中。甚至你是资深的开发工程师，采用提供的的视觉图像函数库，可节省大量开发成本，有效提升产品开发效率。

OpenMV 可以用来进行颜色识别，形状识别，矩形识别，圆形识别，机器人巡线，直线识别，人脸识别，边缘检测，连通域检测，光流，人眼追踪，模板匹配，特征点追踪，二维码识别，瞳孔检测，条形码识别，矩形码识别，AprilTag 目标跟踪，绘图写字，帧差异，录制视频等。

OpenMV 文档参考网站：https://book.openmv.cc/

OpenMV3 M4 ， OpenMV4 H7， OpenMV4 H7 Plus 有什么区别?

主要是外设不同：

Tables	OpenMV2 M4	OpenMV3 M7	OpenMV4 H7	OpenMV4 H7 Plus
Pin	9	10	10	10
ADC/DAC	1	1	1	1
SPI	1	1	1	1
I2C	1	2	2	2
UART	1	2	2	2
Servo	2	3	3	2
CAN bus	no	1	1	1
IC	STM32F427	STM32F765	STM32H743	STM32H743
RAM	256KB	512KB	1MB	32MB + 1MB
Flash	1MB	2MB	2MB	32MB + 2MB
频率	180MHz	216MHZ	480MHZ	480MHZ
标配感光元件	OV7725(30W 像素)	OV7725(30W 像素)	OV7725(30W 像素)	OV5640(500W 像素)

重点：算法上 OpenMV2 不能使用 apriltag，OpenMV3 可以。版本越高，功能越多。

最大可支持分辨率：OpenMV2 320x240；OpenMV3 640x480；OpenMV4 752x480（MT9V034 全局快门模组）；OpenMV4 PLus 2592x1944.

二、参数:

1.摄像头参数（摄像头可调焦）

STM32H750 ARM Cortex M7 处理器，480 MHz ，1 MB RAM，1 MB flash. 所有的 I/O 引脚输出 3.3V 并且 5V 耐受。这个处理器有以下的 IO 接口:

全速 USB (12Mbs) 接口，连接到电脑。当插入 OpenMV 摄像头后，你的电脑会出现一个虚拟 COM 端口和一个“U 盘”

μSD 卡槽拥有 100Mbs 读写，这允许你的 OpenMV 摄像头录制视频，和把机器视觉的素材从 SD 卡提取出来

一个 SPI 总线高达 54Mbs 速度，允许你简单的把图像流数据传给 LCD 扩展板，WiFi 扩展板，或者其他控制器

一个 I2C 总线，CAN 总线，和一个异步串口总线（TX/RX），用来链接其他控制器或者传感器

一个 12-bit ADC 和一个 12-bit DAC

3 个 I/O 引脚用于舵机控制

所有的 IO 口都可以用于中断和 PWM(板子上有 10 个 I/O 引脚)

一个 RGB LED（三色）， 两个高亮的 850nm IR LED（红外）

2.产品规格

处理器	ARM® 32-bit Cortex®-M7 CPU
	w/ Double Precision FPU
	480 MHz (462 DMIPS)
	Core Mark Score: 1082
	(compare w/ Raspberry Pi Zero: 2060)
	RAM Layout 128KB .DATA/.BSS/Heap/Stack
	384KB Frame Buffer/Stack
	(1 MB Total)
	Flash Layout 32KB Bootloader
	96KB Embedded Flash Drive
	1920KB Firmware
	(1MB Total)
支持的图像格式	Grayscale
	RGB565
	JPEG (and BAYER)
最大支持的像素	Grayscale: 640×480 and under
	RGB565: 320×240 and under
	Grayscale JPEG: 640×480 and under

RGB565 JPEG: 640×480 and under

镜头信息	焦距: 2.8mm
	光圈: F2.0
	尺寸: 1/3″
	视角: 115°
	安装: M12*0.5
	红外滤光片（截断红外光）: 650nm (可拆卸)
电气信息	所有引脚均可承受 5V 电压，输出电压为 3.3V。所有引脚都可以提供最高 25mA 的拉电流(source)或灌电流(sink)。在 ADC 或 DAC 模式下，P6 不能承受 5V 电压。引脚总共可提供最高 120mA 的拉电流(source)或灌电流(sink)。VIN 可以在 3.6V 和 5V 之间。不要从 OpenMV Cam 的 3.3V 引脚输出超过 250mA 的电流。
重量	16g
长度	45mm
宽度	36mm
高度	30mm

3.功耗

空闲 – 无 SD 卡 110mA @ 3.3V

空闲 – 有 SD 卡 110mA @ 3.3V

运行 – 无 SD 卡 190mA @ 3.3V

运行 – 有 SD 卡 200mA @ 3.3V

4.温度范围

存储 -40°C to 125°C

运行 -20°C to 70°C

5.供电

以下两种方式二选一，也可以同时供电:

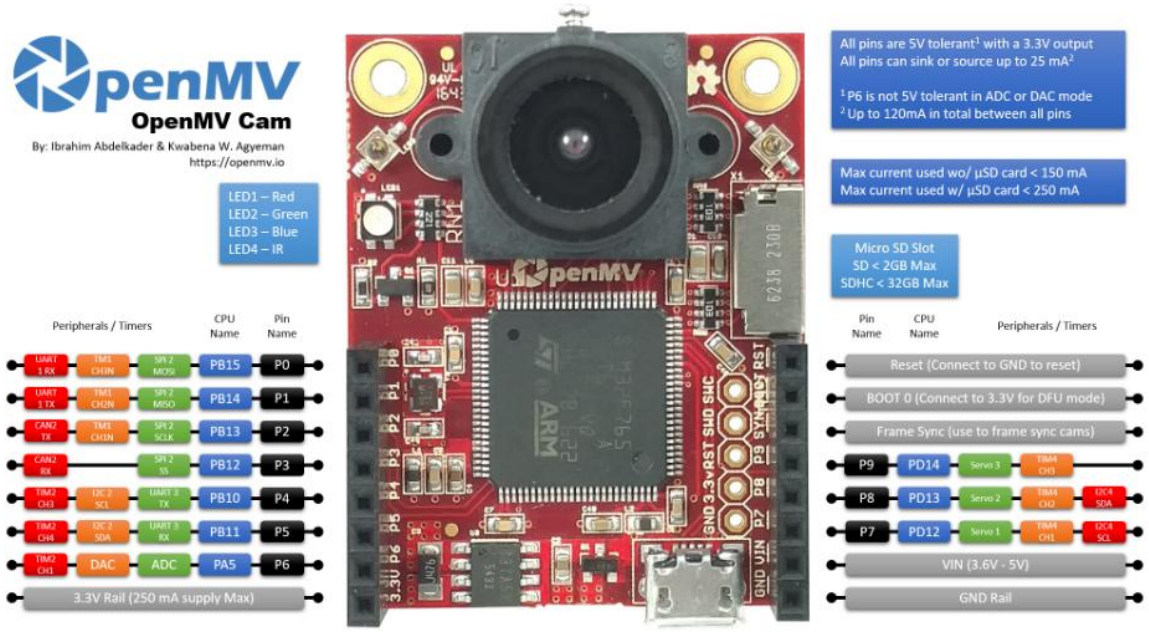
1.直接连 USB 口供电

2.连 VIN-GND，3.7-5V，推荐 5V，电源的额定电流需要大于 250MA。

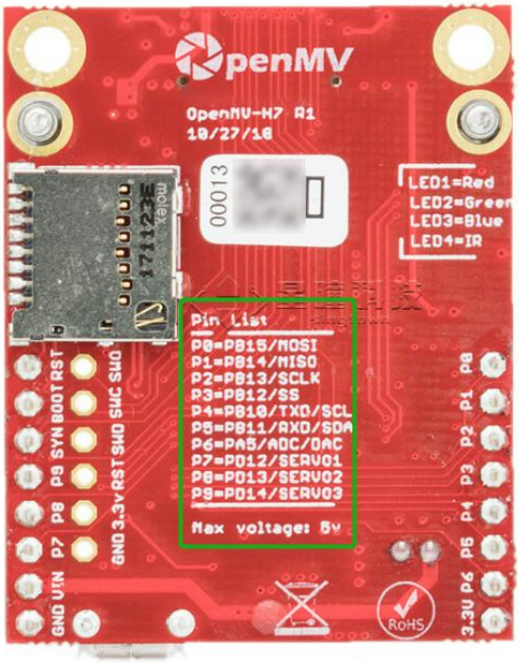
注意!任何情况下，都不应该用 3.3V 引脚给 0penMV 供电

在本套件中，我们一般都是使用 usb 口供电的，在后面的课程中，也可以通过我们的扩展板来给 openmv 供电。

三、引脚定义



此处引脚一共有 P0-P9 共 10 个引脚，其中可以用于控制舵机的是 P7、P8、P9，关于引脚的详情还可以在 openmv 板子背面看到。



【本课小结】：

本课我们认识了 OpenMV4 H7 并了解了它的一些参数，以及它的一些引脚，为我们后面使用它奠定了基础。

众灵科技

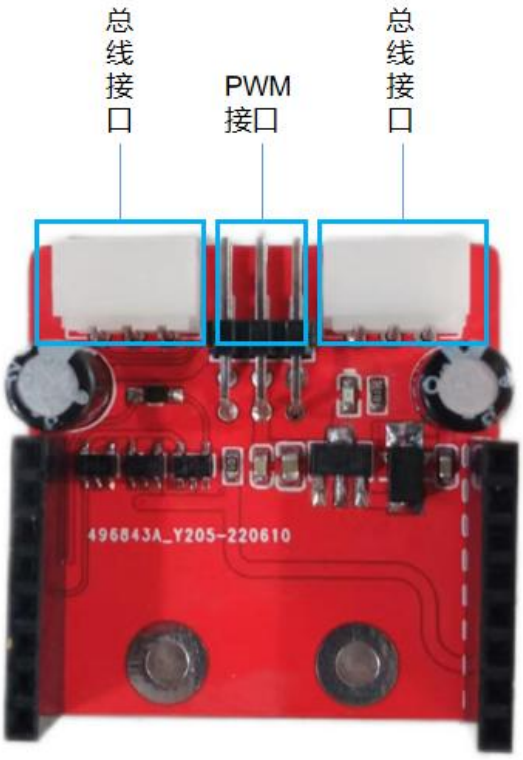
第 2 课 认识扩展板

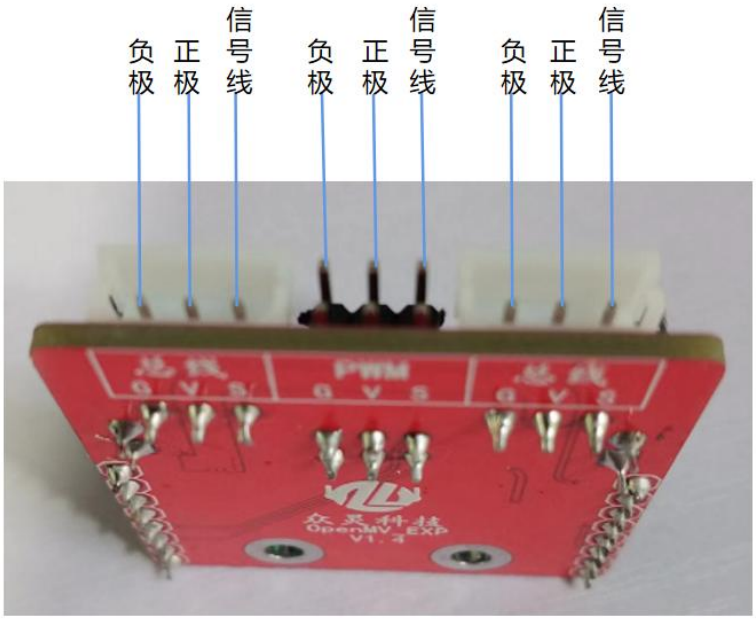
【本课目标】：

本课认识我司制作的 openmv 总线扩展板。

【本课内容】：

扩展板如图所示：







1、两路总线接口（将 TX 和 RX 通过电路变成一路输出），有斜角的一侧为信号线。

2、两路 PWM 接口：靠近板子的 PWM 接口用的是 openmv 的 P7 引脚,远离板子的 PWM 接口用的是 openmv 的 P8 引脚。PWM 接口的 3 个针脚的说明在板子背面。

3、固定孔孔径 3MM，孔间距 14mm

4、适用于连接本店所有的总线设备，通过串口直接发送字符串就可以控制所有的总线设备。

OpenMV 与扩展板的插接方法如下图所示：



【本课小结】：

本课认识了转接板，并知道了 OpenMV 与扩展板的插接方法。

第 3 课 OPENMV IDE 的安装

【本课目标】：

本课学习 OpenMV IDE 的安装

【本课内容】：

首先我们需要下载 openmv ide（在我们资料中的相关工具中也会有提供的软件）

也可直接从官网下载，下载地址（二选一即可）：

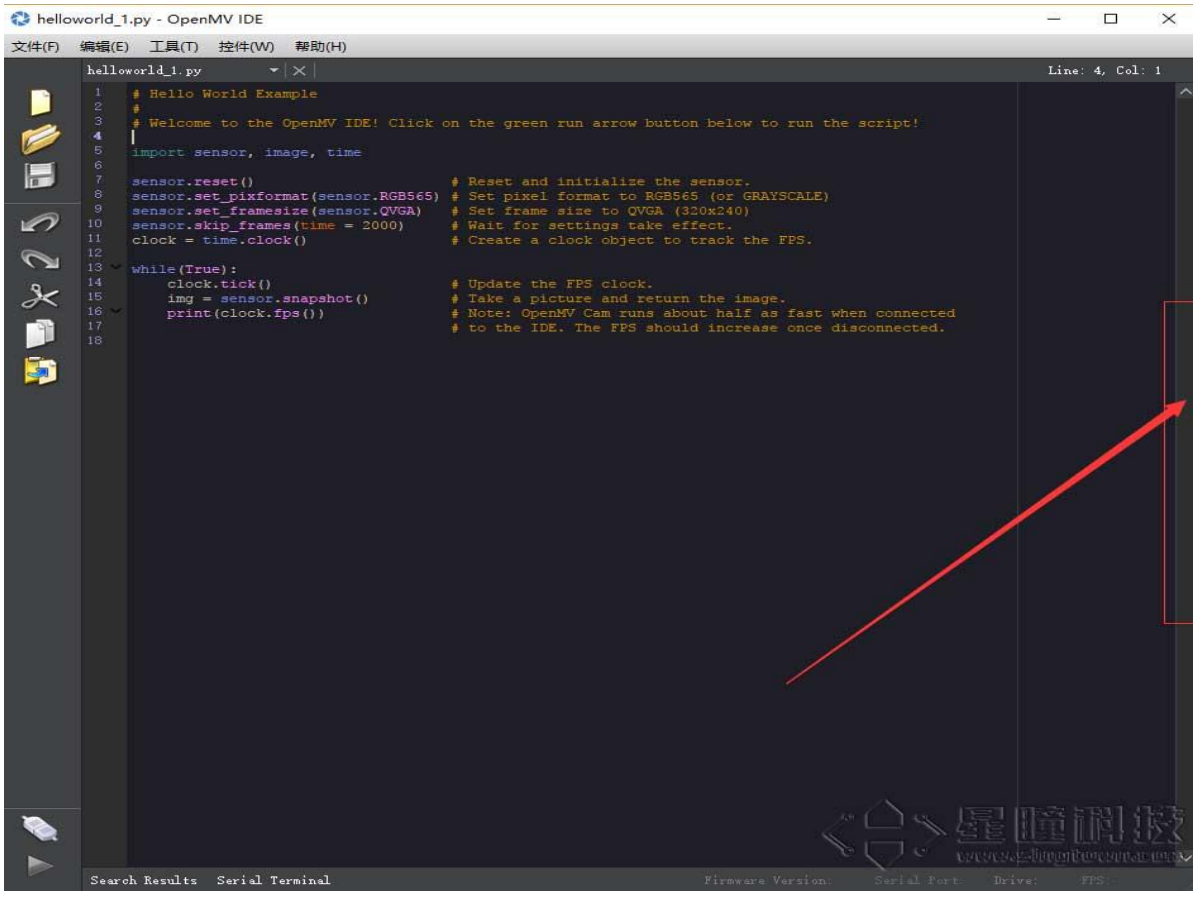
中国官网地址：https://singtown.com/openmv-download/

美国官网下载地址：https://openmv.io/pages/download

Windows 安装 openmv ide：

下载后一直点击下一步，就正常安装完成了。

安装完成，打开之后如果没有图像窗口，从右侧可以拖拽出来。



安装驱动：

将 OpenMV 插到电脑。

正常情况下，会**自动**安装驱动，一切不需要手动安装。在**设备管理器**中会看到：



但是可能在一些电脑，驱动不会自动安装。

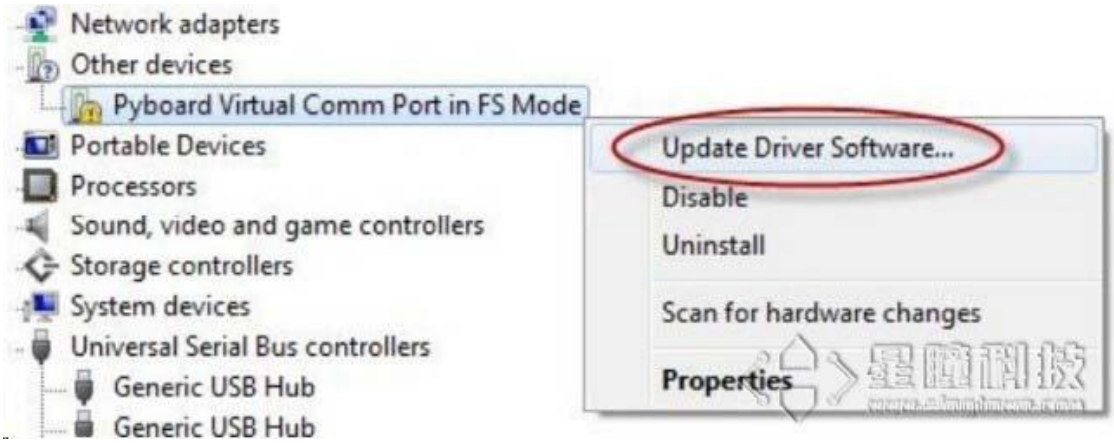
这时在设备管理器中会出现一个叹号，表示没有正常安装驱动。



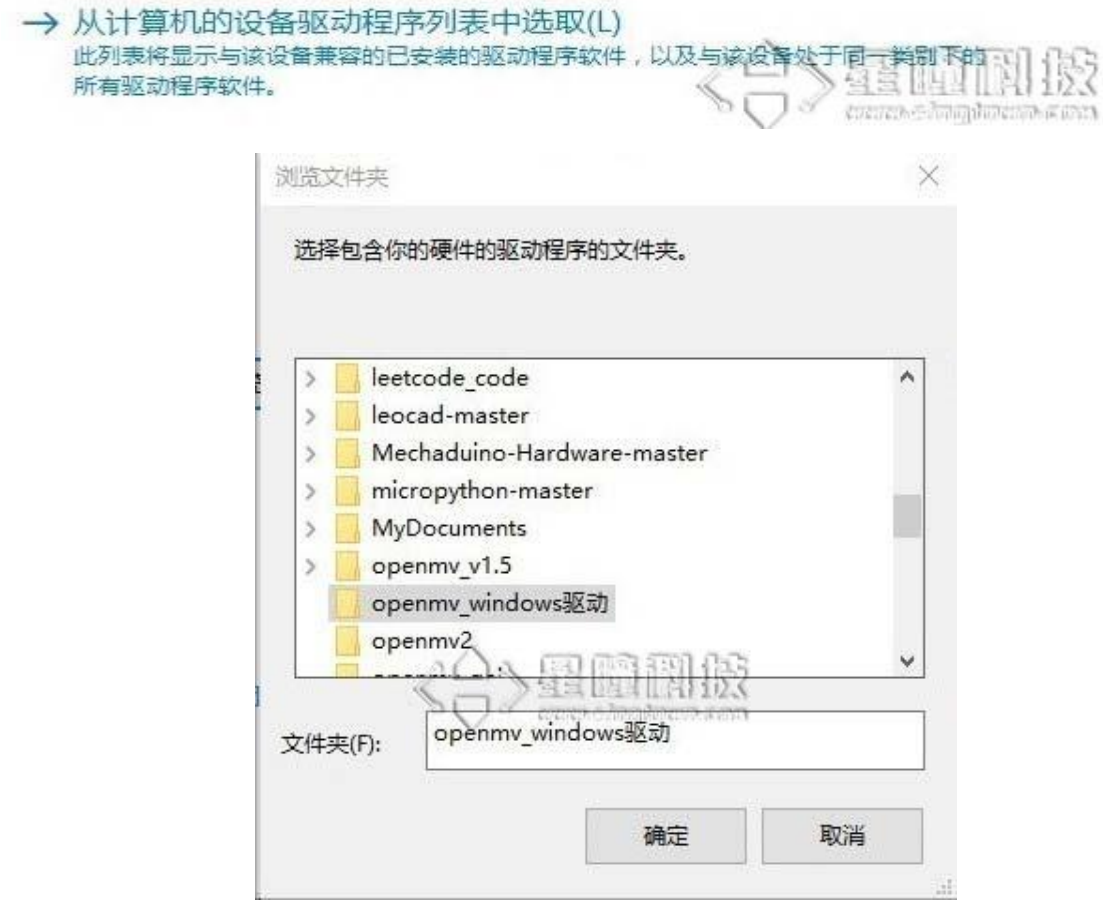
这时就需要自己手动安装。

首先下载驱动： https://dl.singtown.com/openmv/openmv_windows 驱动.zip

解压到桌面，然后右键设备管理器中的这个设备，然后点升级驱动：




你希望如何搜索驱动程序软件?
→ 自动搜索更新的驱动程序软件(S)
Windows 将在你的计算机和 Internet 上查找用于相关设备的最新驱动程序软件，除非在设备安装设备中禁用该功能。
→ 浏览计算机以查找驱动程序软件(R)
手动查找并安装驱动程序软件。
浏览计算机上的驱动程序文件
在以下位置搜索驱动程序软件:
E:
浏览(R)...
包括子文件夹(I)


→ 从计算机的设备驱动程序列表中选取(L)
此列表将显示与该设备兼容的已安装的驱动程序软件，以及与该设备处于同一类别下的所有驱动程序软件。
浏览文件夹
选择包含你的硬件的驱动程序的文件夹。
leetcode_code
leocad-master
Mechaduino-Hardware-master
micropython-master
MyDocuments
openmv_v1.5
openmv_windows驱动
openmv2
文件夹(F):
openmv_windows驱动
确定
取消



Windows 驱动安装故障:

如果出现下面的情况（此处极少发生，属于电脑故障）：



OpenMV 驱动安装失败，90%的情况都是电脑的问题，精简版操作系统和使用了一些优化软件通常是引起此类问题的原因。OpenMV 驱动解决办法跟 arduino 类似。

这是因为精简版的 window 系统删掉了一些不常用的驱动信息引起的。

故障参考资料：

https://forum.singtown.com/topic/230

最后，建议：

- 买电脑的时候，买带正版系统的电脑。
- 打开 Windows 更新（默认就是打开的，不要手贱关掉它）。
- 即使使用盗版的系统，请在 MSDN 上下载纯净完整版。
- 千万不要用 Ghost 安装系统！90%的驱动问题，原因就是 Ghost。

【本课小结】：

本节课我们学习了 Windows 电脑上 openmv ide 的安装及必要驱动的安装，macOS 与 Linux 安装 openmv ide 与 Windows 安装 openmv ide 的方式类似。

第 4 课 OPENMV IDE 的使用

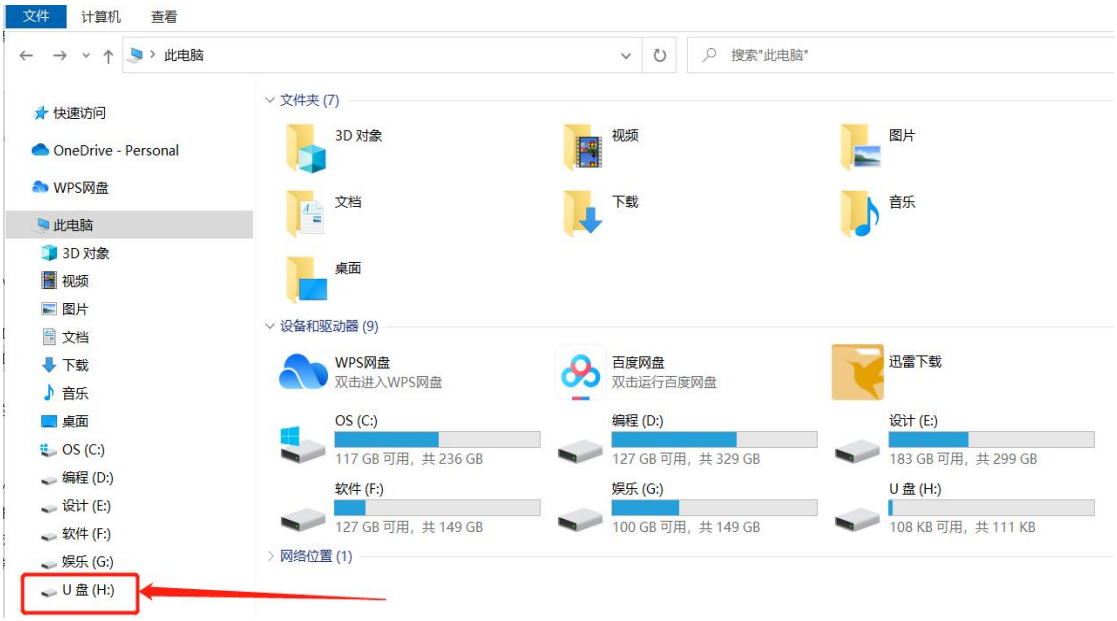
【本课目标】：

本课认识 OPENMV IDE 的操作界面并学习其使用。

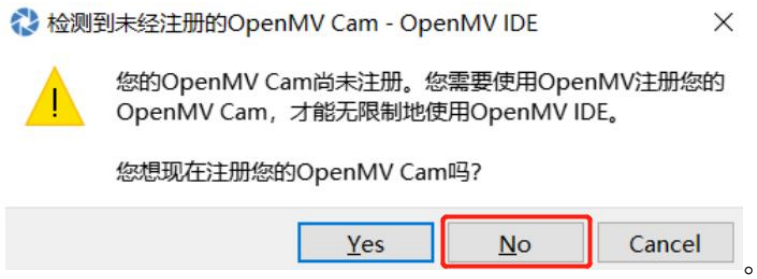
【操作步骤】：

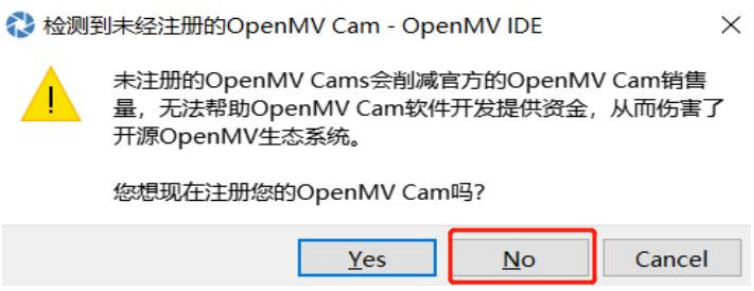
首先确保 openmv IDE 和驱动正常安装完成。

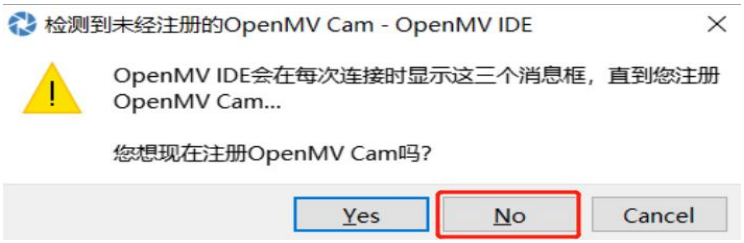
将 OpenMV 通过 USB 线插入电脑，此时会有一个 U 盘出现



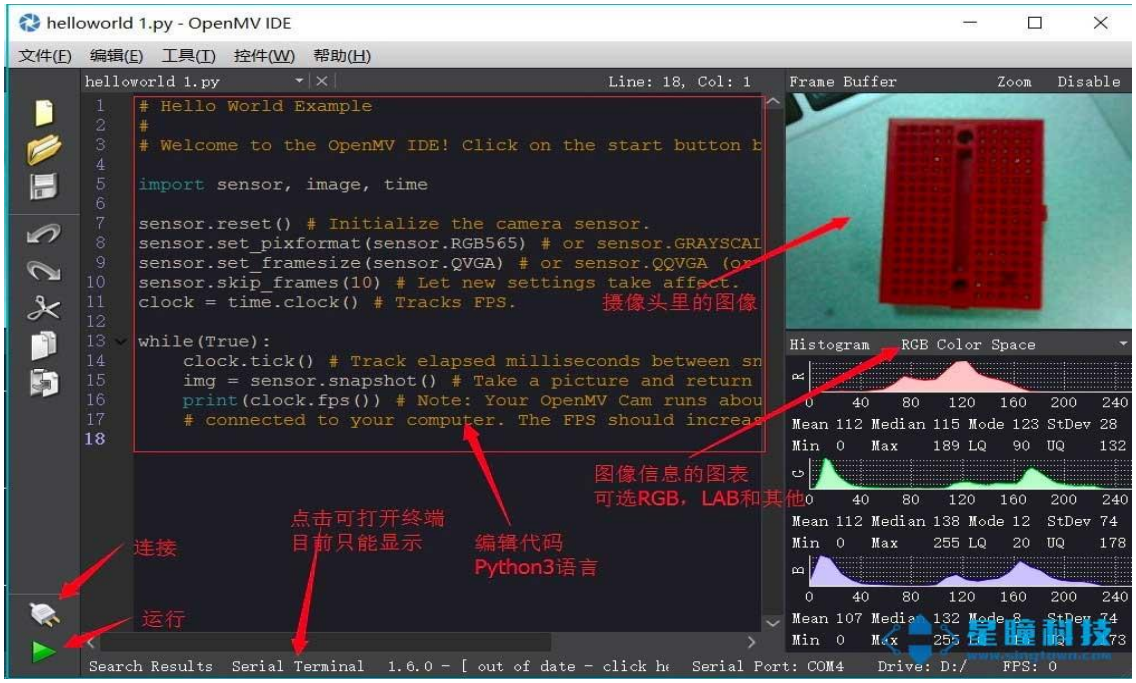
打开 OpenMV IDE，可能会连续弹出 3 个窗口提示，提示让你注册，此时可以点击 No 或 Cancel，然后就可以进入 openmv ide 界面进行操作了：







然后会看到 openmv ide 的操作界面如下：



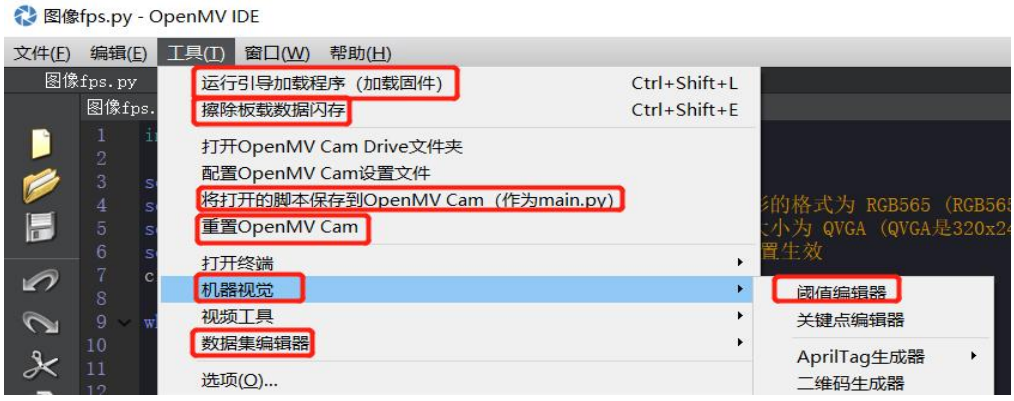
重点看上面这张图（此处为英文版，和中文版的对应位置是一样的）。

各部分功能介绍如下：

1.官方示例程序可以在'文件'-->'示例'-->'OpenMV'中找到；

2.在'帮助'中可以看到 openmv 官方文档及论坛等，可以点击进入查看更详细的介绍。

3.在'工具'中可以看到如下几项，比较重要的几个都已框出来。



'运行引导加载程序'会在升级固件时用到。

'擦除板载数据闪存'可以将 openmv 闪存清空。

‘将打开的脚本保存到 openmv 中’可以将在 openmv ide 中打开的文件保存到 openmv 的盘符中，并以 main.py 命名，可以将已保存的文件改为其他名称。

‘重置 openmv cam’可以使 openmv 板子重启，注意：使用上述方法将某些文件保存到 openmv 的盘符中时，需要点击‘重置 openmv cam’来刷新 openmv 的 flash 闪存。

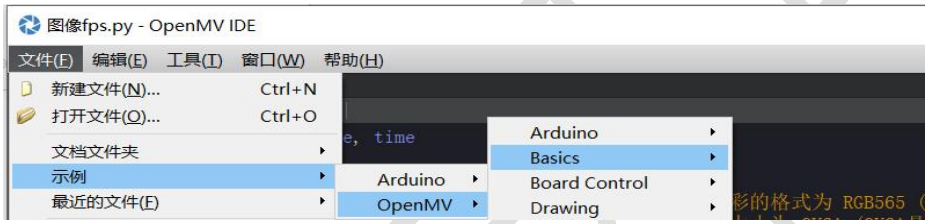
‘机器视觉’-->‘阈值编辑器’可以用来调节颜色阈值，获得自己想要的颜色阈值，然后复制出来编辑到代码中使用。

‘数据集编辑器’-->‘新数据集’可以用来采集数据，进行神经网络模型的训练，注意只适用于 openmv4 h7 plus

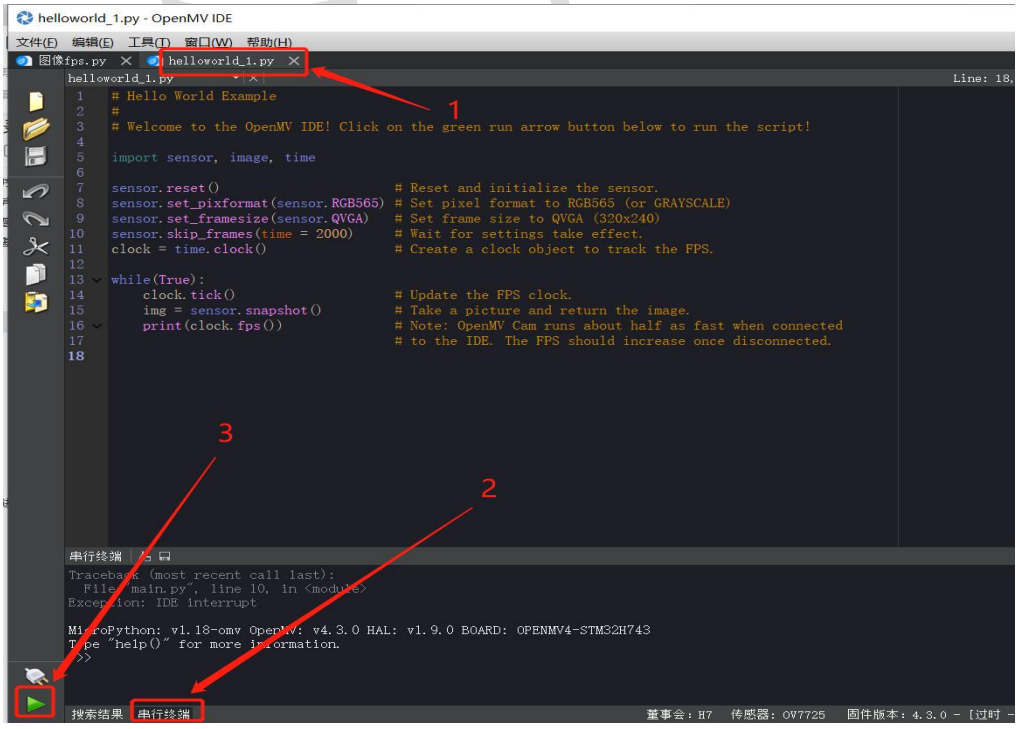
‘直方图色彩空间 LAB’也可用来配合**‘帧缓冲区’**调节颜色阈值，获得自己想要的颜色阈值。

‘帧缓冲区’可以显示 openmv 摄像头拍到的画面，当点击‘禁用’后，帧率会更快，画面会更流畅，‘缩放’可以缩放图像。

运行 hello world 示例程序：



在‘文件’-->‘示例’-->‘OpenMV’-->‘Basics’中可以找到 helloworld.py 文件，打开，然后先点一下‘串行终端’，然后点击左下角的绿色运行按钮即可运行程序。



运行程序后，可以在串行终端窗口中看到打印的帧率，如下图所示：



点击红色×号可以停止运行程序。

【本课小结】：

本节我们学习了 openmv ide 软件的使用。

第 5 课 升级固件

【本课目标】：

本课了解 openmv 升级固件的方式。

【操作步骤】：

OpenMV 项目是一直更新的，用户可以升级固件来保持更新。
每次更新，都会修复 bug，增加新的功能。

有两种方式升级固件，一种是 IDE 自带的功能，使用 bootloader 升级固件，我们称之为常规升级。另一种是使用 STM32 Cube Programmer 升级，使用 STM 芯片的升级固件功能。

注意：IDE 自带的升级固件功能适用于 bootloader 未损坏的情况，STM32 Cube Programmer 升级固件的方法适用于 bootloader 也损坏了的情况。一般来说使用 IDE 自带升级固件功能进行升级即可。

请优先使用常规升级，如果是用不了，或者是新的芯片，可以使用 STM32 Cube Programmer 升级固件。

常规升级

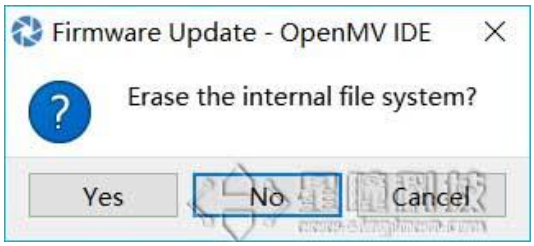
OpenMV 的 IDE 有更新固件的功能，非常好用，优先使用这个（Windows，MacOS，Ubuntu 通用）。



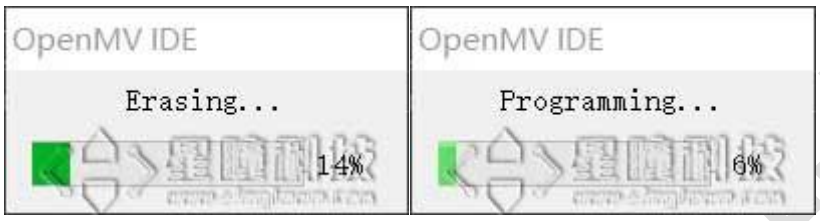
如果你的固件不是最新版本的话，在 IDE 下方会有[过时，点击更新]。如果需要更新固件时，点击这里，会弹出一个对话框：是否升级到最新固件？



选择 OK

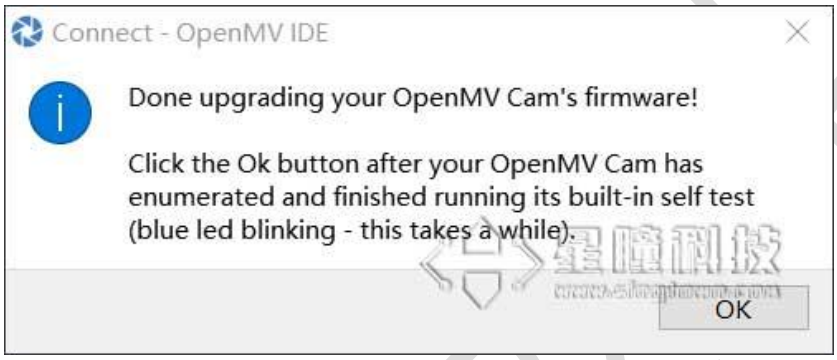


如果想擦除文件系统，选择 Yes，如果想保留文件，选择 No



这个过程大概会持续 1 分钟，OpenMV 的灯会闪。

最后会提示：升级成功。



然后，OpenMV 就升级到最新版了。

STM32 Cube Programmer 升级固件：

可参考博客：https://www.it610.com/article/1290891953084375040.htm

可参考官方视频教程：https://singtown.com/learn/50903/

【本课小结】：

本节课了解了 openmv 升级固件的方法。

注意我们一般是不需要升级固件的。

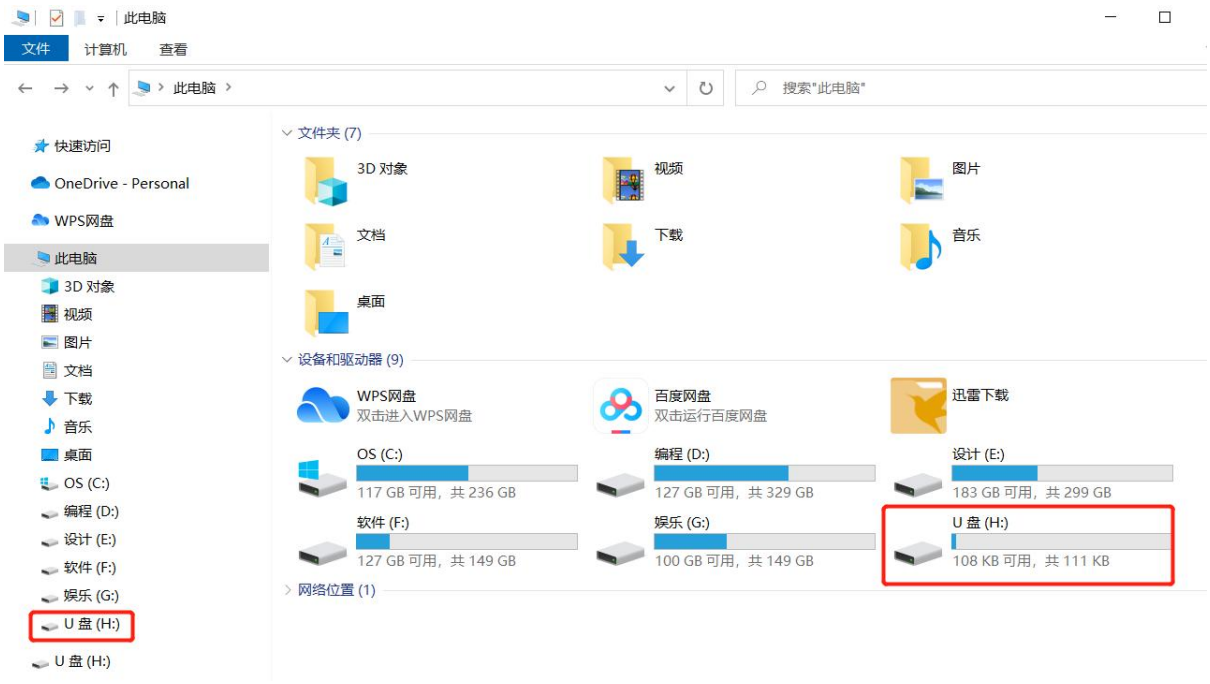
第 6 课 SD 内存卡的使用

【本课目标】：

本课了解 openmv SD 内存卡的使用。

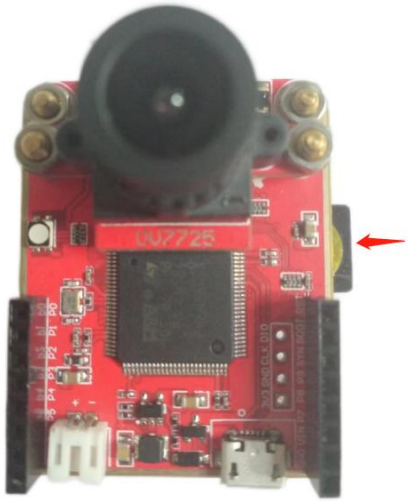
【操作步骤】：

前面我们已经知道，当 openmv 用 usb 数据线连到电脑上之后，会弹出一个盘符，那就是 openmv 的内置 flash 闪存空间，可以将代码存到 flash 中使用，如下所示：

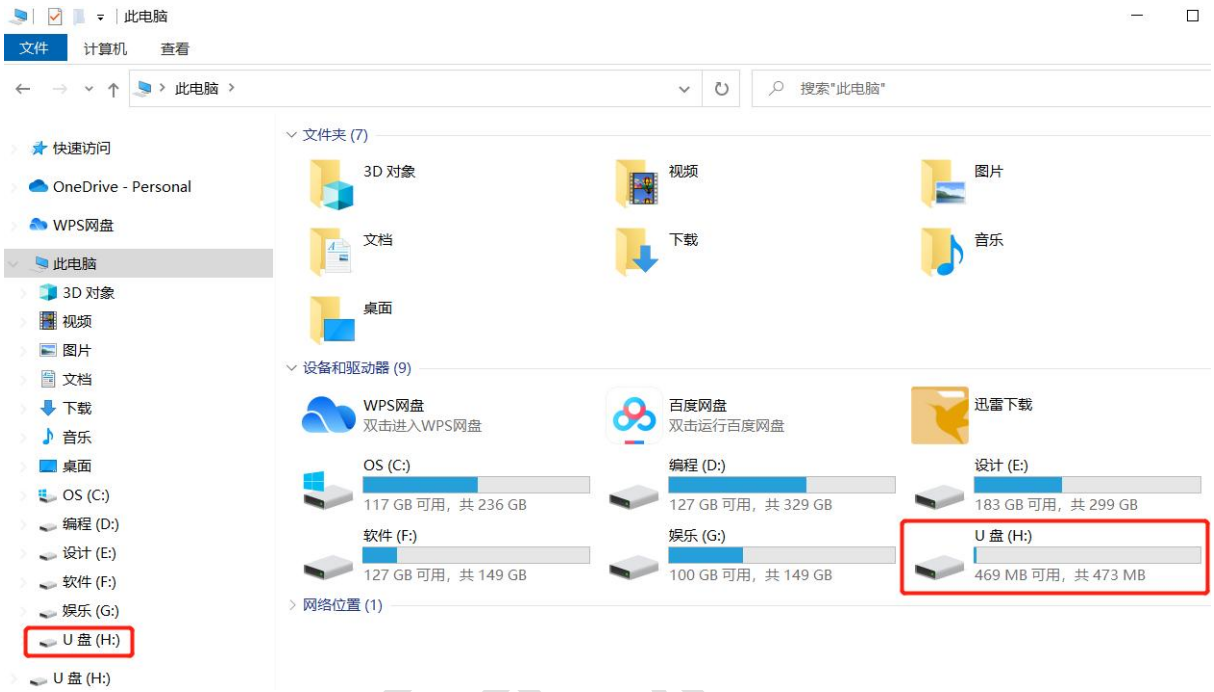


那如果代码有 10 万行，内置的 flash 不够用怎么办？我想保存图片怎么办？

答案是，可以使用 SD 卡！



那么 SD 卡怎么用呢？SD 卡也是一个文件系统，当上电的时候，如果插入 SD 卡，那么 SD 卡的文件系统就会自动取代内置的 Flash 文件系统，每次上电，就会运行 SD 卡中的 main.py 啦，是不是很直观，很方便。SD 卡最大支持 32G 的容量。



【本课小结】：

本节我们学习了 OpenMV SD 卡的使用方法，注意 openmv 内置 flash 闪存不够时可以增加 SD 卡来使用。

第 7 课 脱机运行

【本课目标】：

本课学习 OpenMV 脱机运行程序的方法。

【本课内容】：

OpenMV 运行程序有在线运行和脱机运行两种方式。

注意：在线运行适用于 openmv 不需要移动的情况下，脱机运行则适用于 openmv 需要移动的情况下，比如应用于巡线小车等。

一、在线运行方式

当编写好代码之后，点击 ide 界面左下角的绿色运行图标即可在线运行。

注意：点击红色×号可停止运行程序。

二、脱机运行方式

当编写好代码之后，点击'工具'-->'将打开的脚本保存到 openmv cam 中'（也可以直接将编辑好的源码文件复制粘贴到 openmv 的盘符中，然后将文件名改为 main.py），然后拔掉 usb 数据线，另外重新给 openmv 供电即可脱机运行程序。

三、代码

```
from pyb import LED
from time import sleep

# 初始化函数
def init_setup():
    global led              # 设置为全局变量
    led = LED(1)            # 1 为红色，2 为绿色，3 为蓝色
    led.off()               # 灯灭

# 主函数
def main(delay_time=1):
    led.toggle()            # 灯亮灭反转
    sleep(delay_time)       # 默认延时 1 秒

# 程序入口
if __name__ == '__main__':
    init_setup()            # 执行初始化函数
    try:                    # 异常处理
        while 1:            # 无限循环
            main()          # 执行主函数
    except:
        led.off()           # 程序异常时让灯灭
```

四、脱机运行

在我们的资料中的'源码例程'文件夹中找到 offline_run_led.py 文件，在 openmv ide 中打开，然后保存到 openmv 的盘符中，脱机运行即可（注意：如果您购买的套餐中包含

我们的扩展板，可以使用我们的 zlink 模块+扩展板通过总线对接线来给 openmv 供电)。

五、运行效果

板载 LED 灯每隔 1 秒亮一次红灯。

六、知识点

LED 类 – LED 对象

LED 对象控制一个独立的 LED（发光二极管）。

构造函数

classpyb.LED(id)

创建一个与给定 LED 相关联的 LED 对象：`id` 是 LED 的编号，取值 1-4。

方法

LED.off()

关闭 LED。

LED.on()

打开 LED，达到最大强度。

LED.toggle()

切换 LED 的开（最大强度）关。若 LED 在非零强度下，则会被认为已打开，随之被切换为关。

附：MicroPython 和 OpenMV Cam 中文文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/index.html

【本课小结】：

本节我们学习了 OpenMV 在线运行和脱机运行程序的方式，在后面的课程中我们这两种运行方式都可以灵活选择。

第 8 课 测距

【本课目标】：

本课学习使用 openmv 通过代码测量物体与 openmv 之间的距离。

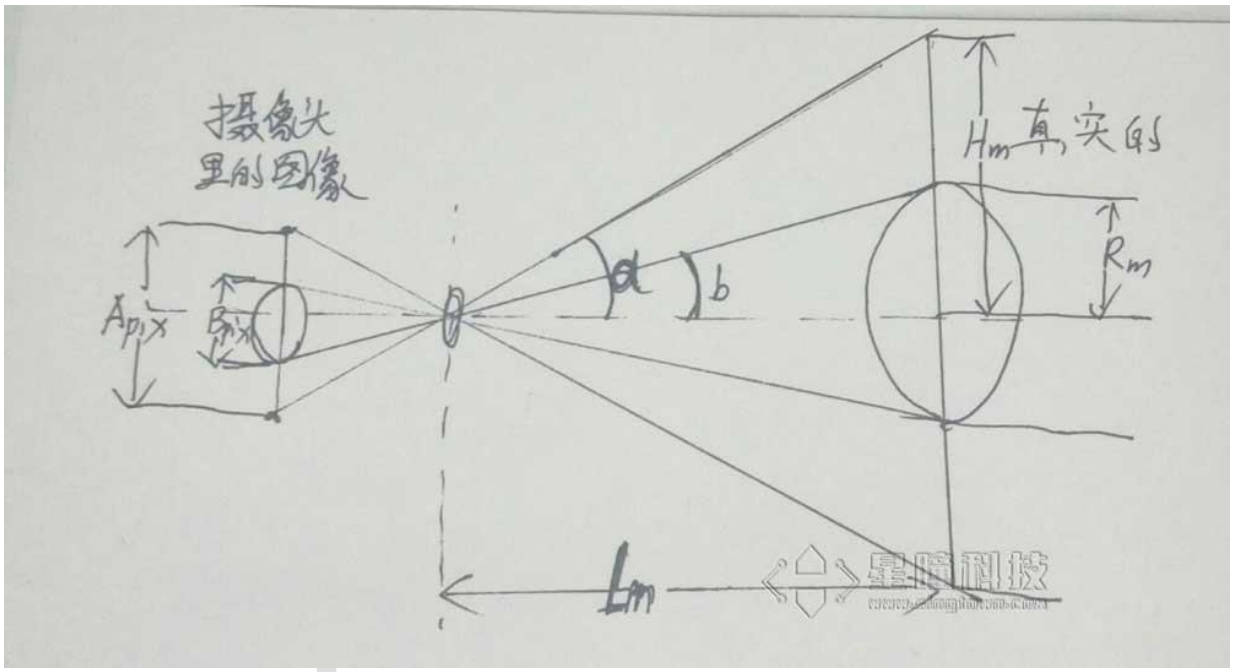
【本课内容】：

一、测距原理：

OpenMV 采用的是单目摄像头，想要实现测距，就需要选参照物，利用参照物的大小比例来计算距离。众所周知，物体距离摄像头越远，摄像头里物体的大小就越小，那么问题来了？这个关系到底是什么呢？

（注：此处的数学几何问题，仅涉及到高中数学三角函数部分，不想看的，直接看结论也可）

为了简化问题，我们看一下图：



由左边的摄像头里的几何关系可得知：

$$tan(a) = \frac{Apix}{2L'}$$

$$tan(b) = \frac{Bpix}{2L'}$$

所以有（1 式）

$$\frac{tan(a)}{tan(b)} = \frac{Apix}{Bpix}$$

由右边的真实环境里的几何关系得知：

$$tan(b) = \frac{Rm}{Lm}$$

带入（1 式），可得（结论公式）：

$$Lm * Bpix = \frac{Rm * Apix}{tan(a)}$$

上面就是最终我们想知道的关系啦！

等号左边的 Lm 是长度，Bpix 是摄像头中，球所占的像素（直径的像素）。等号右边呢，Rm 是球真实的半径，Apix 是是固定的像素，a 是视角的一半。

所以这个公式告诉我们的就是：

实际长度和摄像头里的像素成反比

简化就是

距离 = 一个常数/直径的像素

具体操作步骤呢，就是先测出这个常数的值，就是先让物体距离摄像头 10cm，打印出摄像头里直径的像素值，然后相乘，就得到了 k 的值！

然后 距离=这个常数/摄像头里像素点.

注意：此处是利用参照物的大小比例来计算距离，此种方法只适用于特定大小物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了。

二、代码

```
# 测距
# 注意：此处是利用参照物的大小比例来计算距离。
# 此种方法只适用于特定大小的物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了
import sensor, image, time

# 此处通过识别物体的颜色来测距，当然也可换成其他方式来识别
target_threshold = (33, 98, 37, 94, 75, -27)
#target_threshold = (10, 63, 9, 127, -20, 55)

K = 430                                 # 对某个物体进行测距的第一次需要先确定比例系数 K 的大小
# K = 直径的像素 * 距离。
```

```
# 第一次时，可以将小球放在距离摄像头 10cm 处的位置，测得 Lm 的大小之后，K = 10 * Lm
# 比如在 10cm 处时，直径的像素为 43，则 K = 10 * 43 = 430，具体情况以你的实际情况为准

# 初始化函数
def init_setup():
    global sensor, clock                    # 设置为全局变量
    sensor.reset()                          # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)     # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)      # 将图像大小设置为 160x120
    sensor.skip_frames(10)                  # 跳过 10 帧，使以上设置生效
    sensor.set_auto_whitebal(False)         # 关闭白平衡
    clock = time.clock()                    # 创建时钟对象

# 主函数
def main():
    while 1:
        clock.tick()                        # 更新 FPS 帧率时钟
        img = sensor.snapshot()             # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的红色区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])      # 将红色区域框出来
            img.draw_cross(b[5], b[6])      # 在红色中心位置画十字
            Lm = (b[2]+b[3])/2              # 获取像素点
            length = K/Lm                   # 通过比例系数 K 得到物体距离
            print('摄像头距离物体：%s cm' % length)  # 打印距离值
            #print(Lm)                       # 打印像素点数，在你对一个新的物体进行测距时，需要先获取它的像素点数来得到比例系数 K
        #print(clock.fps())                 # 打印帧率，注:实际 FPS 更高，流 FB 使它更慢。

# 程序入口
if __name__ == '__main__':
    init_setup()  # 执行初始化函数
    try:
        main()    # 执行主函数
    except:
        pass
```

三、代码使用方法

1.在资料中的'源码例程'文件夹中找到 distance_detection.py，然后在 openmv ide 中打开。

2.确定要测距的物体，比如这里我要测一个红色木块与 openmv 之间的距离，首先需要将代码中的 target_threshold = (33, 98, 37, 94, 75, -27) 处的阈值范围替换成红色木块的阈值。

阈值查找方式这里主要介绍两种，都是在 openmv ide 中来操作的：

阈值查找方式一：

（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）

（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



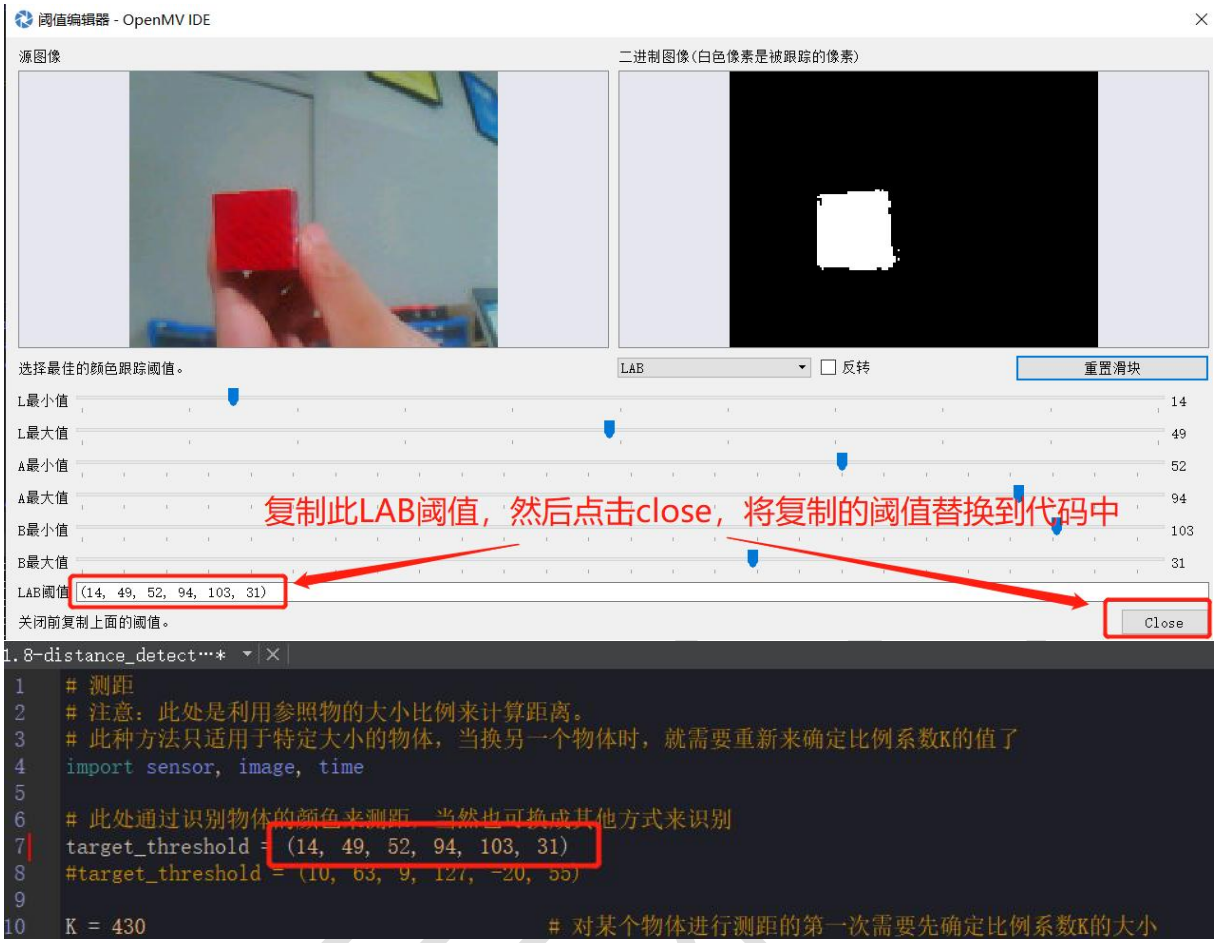
（4）然后点击'工具'-->'机器视觉'-->'阈值编辑器'，选择'帧缓冲区'



（5）看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。

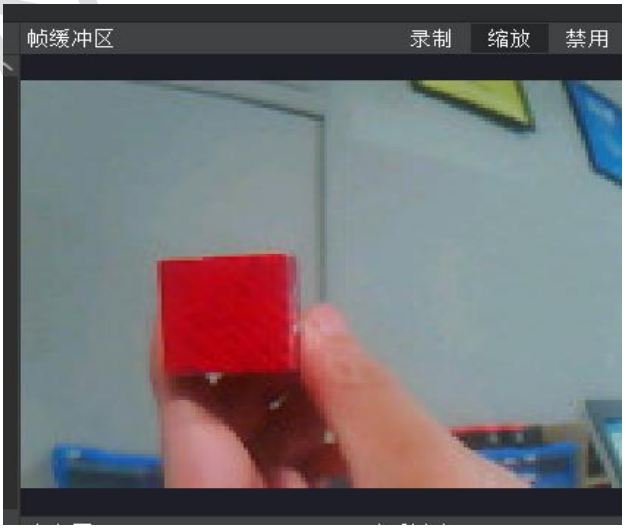


阈值查找方式二：

（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）

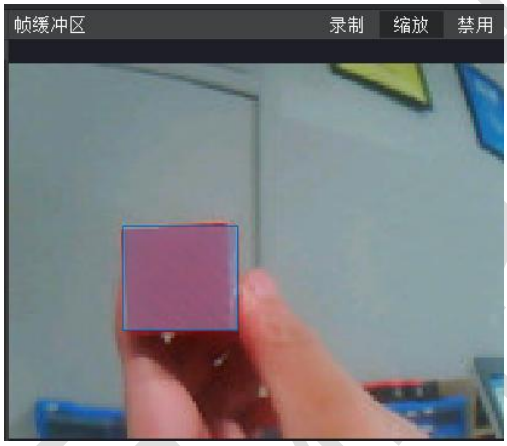
（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



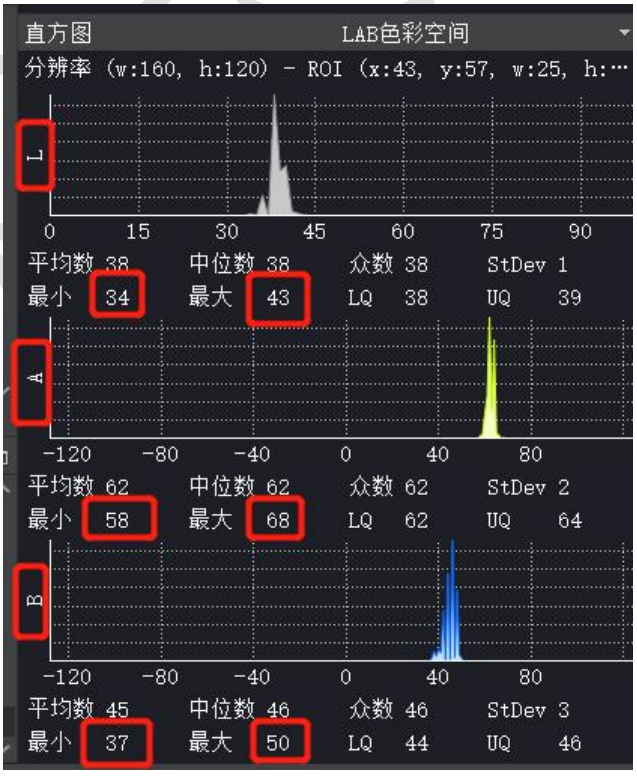
（4）在'直方图'中选择'LAB 色彩空间'，



（5）在'帧缓冲区'中用鼠标左键将要识别的颜色框出来，比如这里我将红色木块框出来了



（6）观察'直方图'中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值)，如下图 L 最小 34，最大 43，A 最小 58，最大 68，B 最小 37，最大 50，则 LAB 阈值为(34,43,58,68,37,50)，替换到代码中使用就可以了。



```
1.8-distance_detect…*
# 测距
# 注意：此处是利用参照物的大小比例来计算距离。
# 此种方法只适用于特定大小的物体，当换另一个物体时，就需要重新来确定比例系数K的值了
import sensor, image, time

# 此处通过识别物体的颜色来测距，当然也可换成其他方式来识别
target_threshold = (34, 43, 58, 68, 37, 50)
#target_threshold = (10, 63, 9, 127, -20, 55)

K = 430                                  # 对某个物体进行测距的第一次需要先确定比例系数K
# K = 直径的像素 * 距离。
# 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm的大小之后，K = 10 * Lm
# 比如在10cm处时，直径的像素为43，则K = 10 * 43 = 430，具体情况以你的实际情况为准
```

3.确定并替换好代码中的颜色阈值之后，将代码中的 print('摄像头距离物体：%s' % length)注释掉，将#print(Lm)取消注释，也就是将下面第一张图改为第二张图的样子：

```
def main():
    while 1:
        clock.tick()                     # 更新FPS帧率时钟
        img = sensor.snapshot()          # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的红色区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])   # 将红色区域框出来
            img.draw_cross(b[5], b[6])   # 在红色中心位置画十字
            Lm = (b[2]+b[3])/2           # 获取像素点
            length = K/Lm                # 通过比例系数K得到物体距离
            print('摄像头距离物体：%s cm' % length)  # 打印距离值
            #print(Lm)                    # 打印像素点数，在你对一个新的物体进行测距时，需
        #print(clock.fps())               # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

```
        img = sensor.snapshot()          # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的红色区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])   # 将红色区域框出来
            img.draw_cross(b[5], b[6])   # 在红色中心位置画十字
            Lm = (b[2]+b[3])/2           # 获取像素点
            length = K/Lm                # 通过比例系数K得到物体距离
            #print('摄像头距离物体：%s cm' % length)  # 打印距离值
            print(Lm)                    # 打印像素点数，在你对一个新的物体进行测距时，需要先获取它的像
        #print(clock.fps())               # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

4.再次运行此代码，然后将红色木块放在距离 openmv 摄像头正前方 10cm 处，观察此时'串行端口'中的输出值，当输出值稳定后，然后停止运行程序。

```
串行终端
43.0
43.0
43.0
43.0
43.0
```

此时就可以计算比例系数 K 的值了，此处 K=10*43=430，（每个物体的比例系数可能

都不一样，视具体情况来定）

5.将代码中的 K 赋值为 430（你上一步计算的 K 是多少，这里就替换为多少，我这里是 430）

```
#target_threshold = (10, 63, 9, 127, -20, 55)

K = 430                                    # 对某个物体进
# K = 直径的像素 * 距离。
# 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm
# 比如在10cm处时，直径的像素为43，则K = 10 * 43 = 430，
```

6.将第 3 步还原，还原到如下图所示：

```
def main():
    while 1:
        clock.tick()                    # 更新FPS帧率时钟
        img = sensor.snapshot()         # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的红色区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])      # 将红色区域框出来
            img.draw_cross(b[5], b[6])      # 在红色中心位置画十字
            Lm = (b[2]+b[3])/2              # 获取像素点
            length = K/Lm                   # 通过比例系数K得到物体距离
            print('摄像头距离物体：%s cm' % length)  # 打印距离值
            #print(Lm)                       # 打印像素点数，在你对一个新的物体进行测距时，需
        #print(clock.fps())              # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

7.然后准备工作就完成了，现在可以开始运行程序了，程序运行后，将要测距的物体在 openmv 摄像头正前方任意位置，然后观察串行端口中的输出值是否正确。

【本课小结】：

本节我们学习了使用 openmv 通过代码测量物体与 openmv 之间的距离。

注意：此处是利用参照物的大小比例来计算距离，此种方法只适用于特定大小物体，当换另一个物体时，就需要重新来确定比例系数 K 的值了。

第 9 课 测量物体大小

【本课目标】：

本课学习通过代码测量物体实际大小。

【操作步骤】：

一、原理

利用固定距离（此处我们使用的是 10cm）和比例系数 K 来计算物体实际大小。

实际大小 = 直径的像素 * 比例系数 K

注意：当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处。

二、代码

```
# 测量物体实际大小
# 注意：此处是利用固定距离和比例系数来计算物体大小。
# 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处
import sensor, image, time

# 此处通过识别物体的颜色来测大小
target_threshold = (13, 58, 37, 94, 75, -27)

K = 0.07                                # 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处
# 实际大小 = 直径的像素 * K。
# 第一次时，可以将小球放在距离摄像头 10cm 处的位置，测得 Lm 的大小之后，K = 实际大小 / Lm
# 比如在 10cm 处时，直径的像素为 43，小球实际大小为 3cm，则 K = 3 / 43 = 0.07，具体情况以你的实际情况为准

# 初始化函数
def init_setup():
    global sensor, clock                # 设置为全局变量
    sensor.reset()                      # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)  # 将图像大小设置为 160x120
    sensor.skip_frames(10)              # 跳过 10 帧，使以上设置生效
    sensor.set_auto_whitebal(False)     # 关闭白平衡
    clock = time.clock()                # 创建时钟对象

# 主函数
def main():
    while 1:
        clock.tick()                    # 更新 FPS 帧率时钟
        img = sensor.snapshot()         # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的目标区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])    # 将目标区域框出来
```

```
34.             img.draw_cross(b[5], b[6])    # 在目标中心位置画十字
35.             Lm = (b[2]+b[3])/2            # 获取像素点
36.             w = K * b[2]                  # 通过比例系数 K 和像素值得到物体实际宽度
37.             h = K * b[3]                  # 通过比例系数 K 和像素值得到物体实际高度
38.             print('物体实际宽度为 %s cm' % w)  # 打印物体实际宽度值
39.             print('物体实际高度为 %s cm' % h)  # 打印物体实际高度值
40.             #print(Lm)                    # 打印像素点数，在你对一个新的物体进行测大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处
41.         #print(clock.fps())              # 打印帧率，注:实际 FPS 更高，流 FB 使它更慢。
42.
43.  # 程序入口
44.  if __name__ == '__main__':
45.      init_setup()  # 执行初始化函数
46.      try:
47.          main()    # 执行主函数
48.      except:
49.          pass
```

三、代码使用方法

1.在资料中的'源码例程'文件夹中找到 size_detection.py，然后在 openmv ide 中打开。

2.先确定比例系数 K 的大小，所以此时我这里找了一个红色木块，长宽均为 3cm（你也可以找其他颜色和大小的物体），这里同样是需要先将代码中的 target_threshold = (33, 98, 37, 94, 75, -27) 处的阈值范围替换成红色木块的阈值。

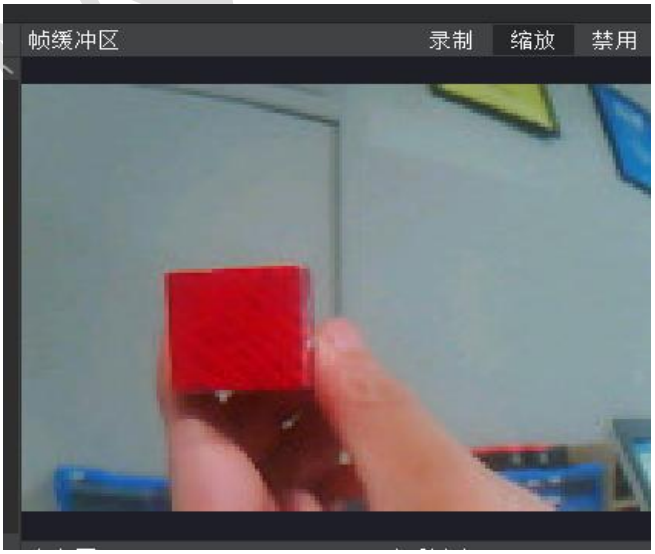
阈值查找方式这里主要介绍两种，都是在 openmv ide 中来操作的：

阈值查找方式一：

（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）

（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



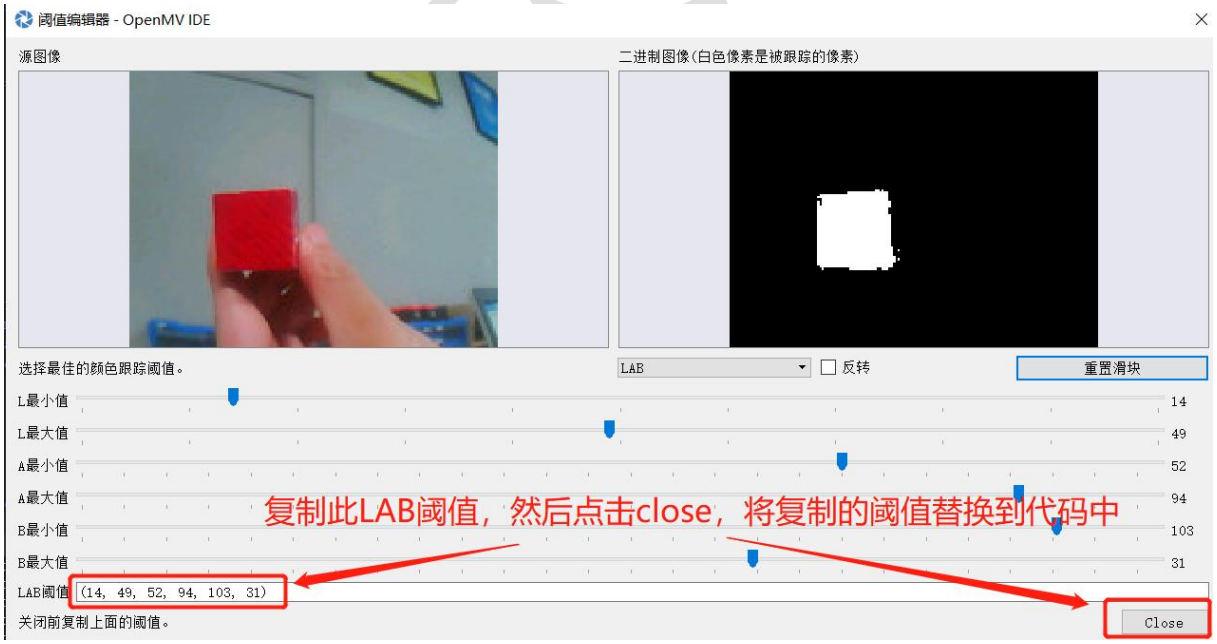
（4）然后点击'工具'-->'机器视觉'-->'阈值编辑器'，选择'帧缓冲区'



（5）看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。



```
# 测量物体实际大小
# 注意：此处是利用固定距离和比例系数来计算物体大小。
# 当测另一个物体的大小时，需要将该物体放在求比例系数K时的那个距离处
import sensor, image, time

# 此处通过识别物体的颜色来测大小
target_threshold = (14, 49, 52, 94, 103, 31)

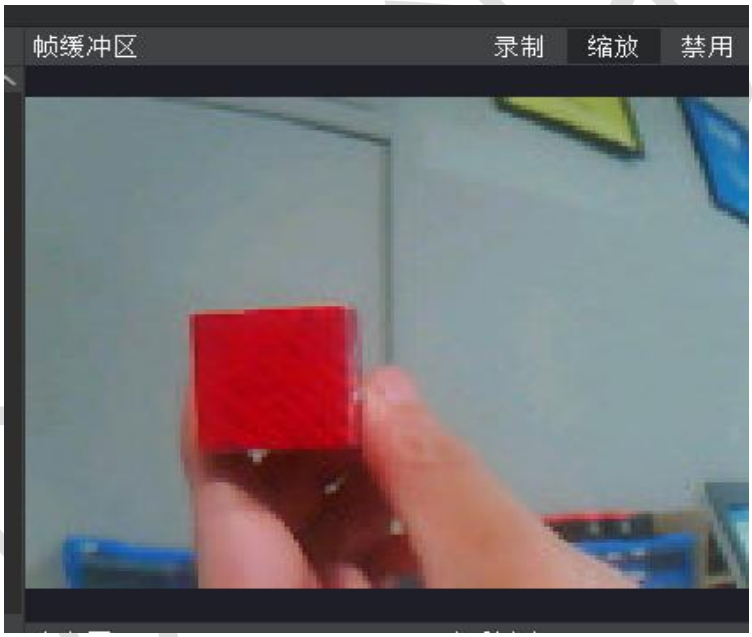
K = 0.07                                    # 当测另一个物体的大小时,
# 实际大小 = 直径的像素 * K。
```

阈值查找方式二：

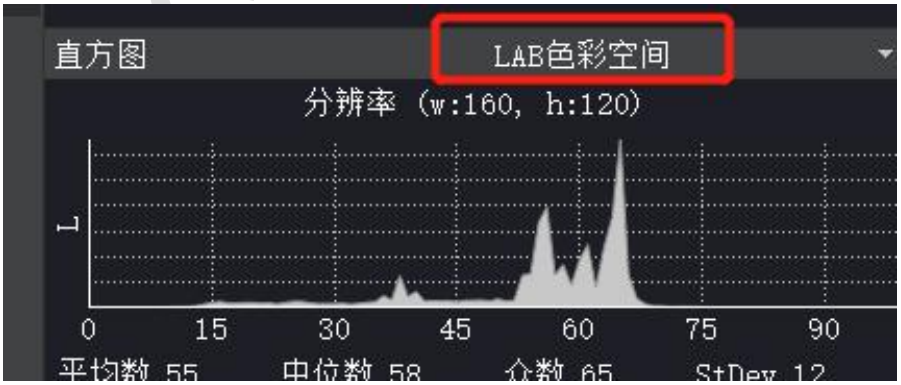
（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要测距的物体放在摄像头画面内，（注意不要有相近颜色的物体干扰，否则效果不好）

（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，

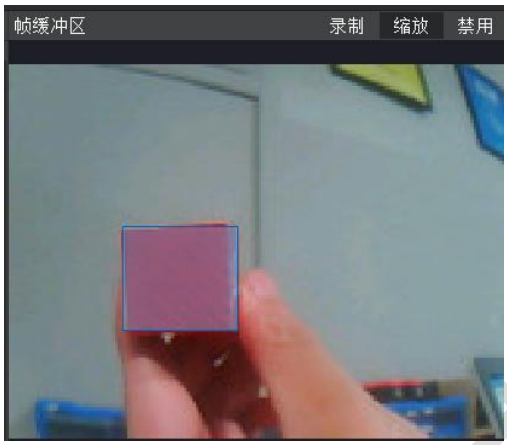


（4）在'直方图'中选择'LAB 色彩空间'，

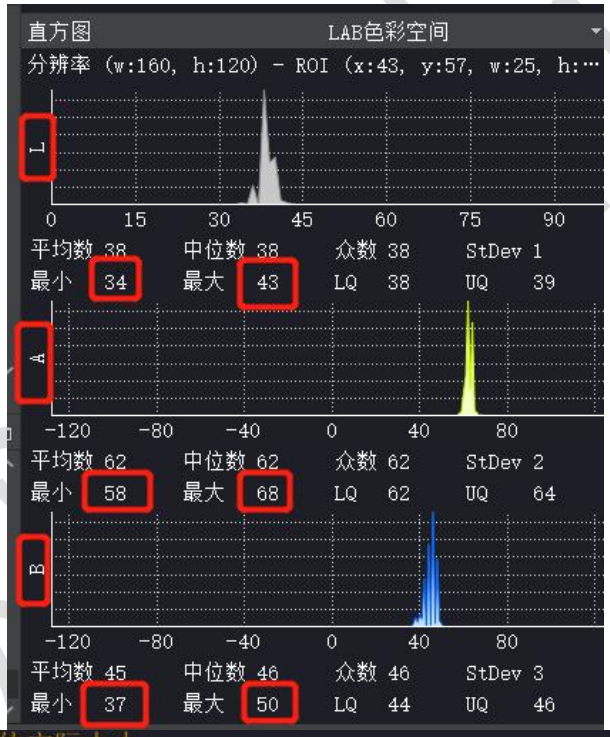


（5）在'帧缓冲区'中用鼠标左键将要识别的颜色框出来，比如这里我将红色木块框出

来了



（6）观察'直方图'中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值)，如下图 L 最小 34，最大 43，A 最小 58，最大 68，B 最小 37，最大 50，则 LAB 阈值为(34,43,58,68,37,50)，替换到代码中使用就可以了。




```
# 测量物体实际大小
# 注意：此处是利用固定距离和比例系数来计算物体大小。
# 当测另一个物体的大小时，需要将该物体放在求比例系数K时的那个距离
import sensor, image, time

# 此处通过识别物体的颜色来测大小
target_threshold = (34, 43, 58, 68, 37, 50)

K = 0.07                                    # 当测另一个物体的大小
# 实际大小 = 直径的像素 * K。
```

3.确定并替换好代码中的颜色阈值之后，将代码中的 print('物体实际宽度为%s cm' %

w)和 print('物体实际高度为%s cm' % h)注释掉，将#print(Lm)取消注释，也就是将下面第一张图改成第二张图的样子：

```
# 主函数
def main():
    while 1:
        clock.tick()                    # 更新FPS帧率时钟
        img = sensor.snapshot()         # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的目标区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])     # 将目标区域框出来
            img.draw_cross(b[5], b[6])     # 在目标中心位置画十字
            Lm = (b[2]+b[3])/2             # 获取像素点
            w = K * b[2]                   # 通过比例系数K和像素值得到物体实际宽度
            h = K * b[3]                   # 通过比例系数K和像素值得到物体实际高度
            print('物体实际宽度为 %s cm' % w)  # 打印物体实际宽度值
            print('物体实际高度为 %s cm' % h)  # 打印物体实际高度值
            #print(Lm)                      # 打印像素点数，在你对一个新的物体进行测大小
        #print(clock.fps())                 # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

```
            b = blobs[0]
            img.draw_rectangle(b[0:4])     # 将目标区域框出来
            img.draw_cross(b[5], b[6])     # 在目标中心位置画十字
            Lm = (b[2]+b[3])/2             # 获取像素点
            w = K * b[2]                   # 通过比例系数K和像素值得到物体实际宽度
            h = K * b[3]                   # 通过比例系数K和像素值得到物体实际高度
            #print('物体实际宽度为 %s cm' % w)  # 打印物体实际宽度值
            #print('物体实际高度为 %s cm' % h)  # 打印物体实际高度值
            print(Lm)                      # 打印像素点数，在你对一个新的物体进行测大小时，需要将
        #print(clock.fps())                 # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

4.再次运行此代码，然后将红色木块放在距离 openmv 摄像头正前方 10cm 处，观察此时'串行端口'中的输出值，当输出值稳定后，然后停止运行程序。

```
串行终端
43.0
43.0
43.0
43.0
43.0
```

此时就可以计算比例系数 K 的值了，此处 K=3 /43=0.07，（以你测得的实际为准）

5.将代码中的 K 赋值为 0.07（你上一步计算的 K 是多少，这里就替换为多少，我这里是 0.07）

```
# 当测另一个物体的大小时，需要将该物体放在求比例系数K时的那个距离处，此处我
import sensor, image, time

# 此处通过识别物体的颜色来测大小
target_threshold = (14, 49, 52, 94, 103, 31)

K = 0.07                                # 当测另一个物体的大小时，需要将
# 实际大小 = 直径的像素 * K。
# 第一次时，可以将小球放在距离摄像头10cm处的位置，测得Lm的大小之后，K = 实
```

6.将第 3 步还原，还原到如下图所示：

```
# 主函数
def main():
    while 1:
        clock.tick()                    # 更新FPS帧率时钟
        img = sensor.snapshot()         # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 找到图像中的目标区域
        if len(blobs) == 1:
            b = blobs[0]
            img.draw_rectangle(b[0:4])     # 将目标区域框出来
            img.draw_cross(b[5], b[6])     # 在目标中心位置画十字
            Lm = (b[2]+b[3])/2             # 获取像素点
            w = K * b[2]                   # 通过比例系数K和像素值得到物体实际宽度
            h = K * b[3]                   # 通过比例系数K和像素值得到物体实际高度
            print('物体实际宽度为 %s cm' % w)  # 打印物体实际宽度值
            print('物体实际高度为 %s cm' % h)  # 打印物体实际高度值
            #print(Lm)                      # 打印像素点数，在你对一个新的物体进行测大小
        #print(clock.fps())                 # 打印帧率，注:实际FPS更高，流FB使它更慢。
```

7.然后准备工作就完成了，现在可以开始运行程序了，程序运行后，将要测大小的物体（注意：如果要测的物体和确定比例系数时的颜色不一样，需要重新把被测物体的颜色阈值替换到代码中）放在 openmv 摄像头正前方 10cm 处，然后观察串行端口中的输出值是否正确。

【本课小结】：

本节我们学习了使用 openmv 通过代码测量物体的实际大小。

注意：

1. 当测另一个物体的大小时，需要将该物体放在求比例系数 K 时的那个距离处，此处我们使用的是 10cm 处。
2. 当被测物体颜色不同于确定比例系数 K 时的那个颜色时，需要在代码中将被测物体的颜色阈值替换上去。

第 10 课 串口收发数据

【本课目标】：

本课学习通过 openmv 的串口进行收发数据。

【操作步骤】：

为什么要用串口呢？因为到时候需要把信息传给其他 MCU，串口简单，通用，基本每一个 MCU 都会有串口。

TTL 串口至少需要 3 根线：TXD，RXD，GND。TXD 是发送端，RXD 是接收端，GND 是地线。 连线的时候，需要把 OpenMV 的 RXD 连到另一个 MCU 的 TXD，TXD 连到 RXD。

注意：我们需要使用的是串口 3，因为串口 1 是用来通过 USB 和电脑通信的。

注意：我们的总线设备都是通过串口来收发数据的。

一、示例代码

```
# 串口收发数据
#
# This example shows how to use the serial port on your OpenMV Cam. Attach pin
# P4 to the serial input of a serial LCD screen to see "Hello World!" printed
# on the serial LCD display.

from pyb import UART
import time

def init_setup(baud=115200):
    global uart3
    uart3 = UART(3, baud)                   # 初始化串口 3，设置波特率为 115200
    uart3.init(115200,8,None,1)
    print('串口初始化完成')

# 串口接收数据
def uart3_recv():
    recv_data = uart3.read()
    print('recv data: %s' % recv_data)
    return recv_data

# 串口发送数据
def uart3_send(send_data):
    global uart3
    uart3.write(send_data)

# 主函数
def main():
    if uart3.any():
        data = uart3_recv()             # 串口接收数据
```

```
31.         print('uart3 recv: %s' % data)   # 打印串口接收到的数据
32.         print(1)
33.         uart3_send(data)                 # 串口将接收到的数据再通过串口发送出去了
34.
35. # 程序入口
36. if __name__ == '__main__':
37.     init_setup()                         # 执行初始化函数
38.     try:
39.         while 1:
40.             main()                       # 执行主函数
41.     except:
42.         pass
```

注意：此代码在运行时可能没有显示，因为我们只是开启了串口 3，还并未用别的设备给它发数据，此处只是示例代码，教大家串口程序的编写，当你用别的设备给它发数据时，此程序的串行端口窗口中就会显示接收到的数据，同时会将接收到的数据再通过串口 3 发给别的设备。在后面马达控制时就用到了串口 3 发数据。

二、知识点

关于代码中 UART 的使用可以参考官方文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/pyb.UART.html#pyb-uart

【本课小结】：

本节课我们学习了通过 openmv 的串口进行收发数据，为后面使用我们的总线设备奠定基础。

第 11 课 PWM 舵机的控制

【本课目标】：

本课学习使用 OpenMV 和扩展总线板控制 pwm 舵机。

【操作步骤】：

在前面的课程中我们已经讲过，openmv4 h7 有 p7、p8、p9 三个用于控制舵机的引脚，我们的示例代码中使用的是 p7 和 p8。需要注意的是：如果你使用的是 openmv4 h7 plus，则它本身只有 p7、p8 两个用于控制舵机的引脚。对本示例代码无影响。

一、示例代码

```
'''
pwm 舵机控制
度数为 180 度（-90 度-90 度）
可用的 pwm 口有 P7 P8 P9，在我们的扩展板上使用的是 P7 和 P8 引脚
'''
from pyb import Servo
from time import sleep

# 实例化两个舵机
s1 = Servo(1)               # 使用的 P7 引脚
s2 = Servo(2)               # 使用的 P8 引脚

# 舵机回归到中位，即 90 度位置
def middle():
    s1.angle(0, 1000)       # s1 舵机在 1 秒内转动到 0 度位置
    s2.angle(0, 1000)       # s2 舵机在 1 秒内转动到 0 度位置
    sleep(1)                # 延时 1 秒

# 主函数
def main():
    global s1, s2
    for i in range(3):      # 循环执行 3 次
        s1.angle(-45, 1500) # s1 舵机在 1.5 秒内转动到 0 度位置
        s2.angle(-45, 1500) # s2 舵机在 1.5 秒内转动到 0 度位置
        sleep(1.5)          # 延时 1.5 秒
        s1.angle(45, 1500)  # s1 舵机在 1.5 秒内转动到 0 度位置
        s2.angle(45, 1500)  # s2 舵机在 1.5 秒内转动到 0 度位置
        sleep(1.5)          # 延时 1.5 秒

# 程序入口
if __name__ == '__main__':
    try:                    # 异常处理
        main()              # 执行主函数
    finally:
        middle()            # 程序结束时使舵机回到 0 度位置
```

注意：此代码中主要是控制的云台上的 pwm 舵机，s1 接的是 openmv 引脚 p7 控制的

那个 pwm 接口，s2 接的是 openmv 引脚 p8 控制的那个 pwm 接口。s1 是云台底部的 pwm 舵机，s2 是云台上方的 pwm 舵机。

二、代码效果

云台从右下方运动到左上方，循环三次。

三、知识点

代码中 Servo 类的使用参考官方文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/pyb.Servo.html#pyb-servo

【本课小结】：

本节我们学习了使用 OpenMV 和扩展总线板控制 pwm 舵机，可以用来控制我们的云台。

第 12 课 总线舵机的控制

【本课目标】：

本课学习使用 OpenMV+扩展总线板+电源+zlink 模块+总线对接线控制我们店里的总线舵机。

【操作步骤】：

需要注意：此处控制我们的总线舵机是通过串口发送指令字符串来控制的。

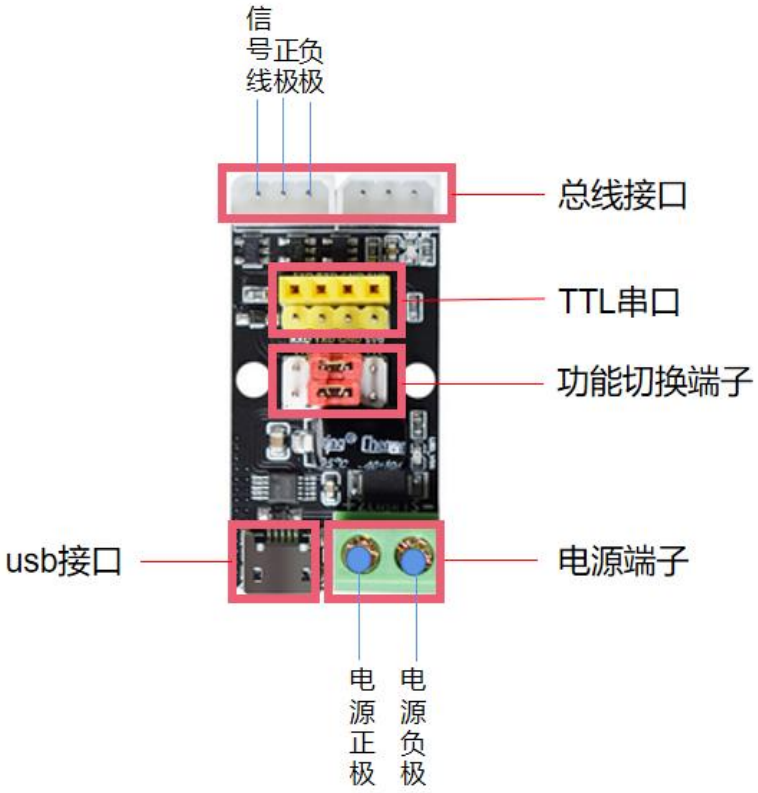
一、认识各模块

1.总线对接线



关于总线对接线具体可参考淘宝详情页 https://item.taobao.com/item.htm?spm=a1z10.5-c-s.w4002-18555111852.30.541224b6PScBKc&id=552997666098

2.zlink 模块



注意：

（1）我们使用时，功能切换端子处的跳线帽在上图中是接到中间的，我们这里要接到 TTL 和 ZX，也就是左移一下，接到偏左的位置。

（2）电源端子处接电源对接线，电源对接线再连接电源。

关于 zlink 模块也可参考淘宝详情页

https://item.taobao.com/item.htm?spm=a1z10.5-c-s.w4002-18555111852.40.541224b6PScBKc&id=570100064201

二、接线说明

1.电源---->电源对接线---->zlink 模块

2.zlink 模块---->总线对接线---->扩展总线板（openmv 是插在我们的扩展总线板上的）

3.扩展总线板---->总线对接线---->可串联两个总线舵机

三、示例代码

```
'''
总线舵机的控制
此处控制总线舵机之前需要先给每个舵机设定 ID，我们建议云台底部舵机 ID 设为 000，云台上方 ID 设为 001
使用 openmv 的串口 3，波特率为 115200
通过串口发送指令字符串来控制
指令格式为 #idPpwmTtime!
其中 id：云台底部舵机 ID 设为 000，云台上方 ID 设为 001
pwm 范围为 0500-2500，复位状态为 1500
time 为时间（毫秒为单位）

'''
import time
from pyb import UART

# 初始化函数
def init_setup():
    global uart3              # 设置为全局变量
    uart3 = UART(3, 115200)   # 实例化串口 3，波特率设置为 115200

# 云台复位
def middle():
    uart3.write('#000P1500T1000!#001P1500T1000!')
    time.sleep(1)             # 延时 1 秒

# 主函数
def main():
    for i in range(3):
        uart3.write('#000P1000T1000!#001P1000T1000!')  # 后退 1 秒
        time.sleep(1)         # 延时 1 秒
        uart3.write('#000P2000T1000!#001P2000T1000!')  # 前进 1 秒
        time.sleep(1)         # 延时 1 秒

# 程序入口
if __name__ == '__main__':
    init_setup()              # 执行初始化函数初始化串口
    try:                      # 异常处理
```

```
37.         main()              # 执行主函数
38.     finally:
39.         middle()
```

四、运行效果

云台从右下角移动到左上角，循环 3 次。

五、知识点

在代码中可以看到，我们发送的指令格式为#IndexPpwmTtime!

指令格式解析：单个舵机指令，Index 为 3 位，000-254；pwm 为 4 位，0500-2500；time 为 4 位，0000-9999，单位毫秒，总共 15 位数据，不足的位数补 0

其中：pwm 为 1500 时，舵机呈复位状态；

【本课小结】：

本课学习了使用 OpenMV+扩展总线板+电源+zlink 模块+总线对接线来控制总线舵机。

需要注意：此处控制我们的总线舵机是通过串口发送指令字符串来控制的。

第 13 课 马达的控制

【本课目标】：

本课学习使用 OpenMV+扩展总线板+电源+zlink 模块+总线对接线+总线双路驱动控制 TT 马达。

【操作步骤】：

需要注意：此处控制我们的马达是通过串口发送指令字符串来控制的。

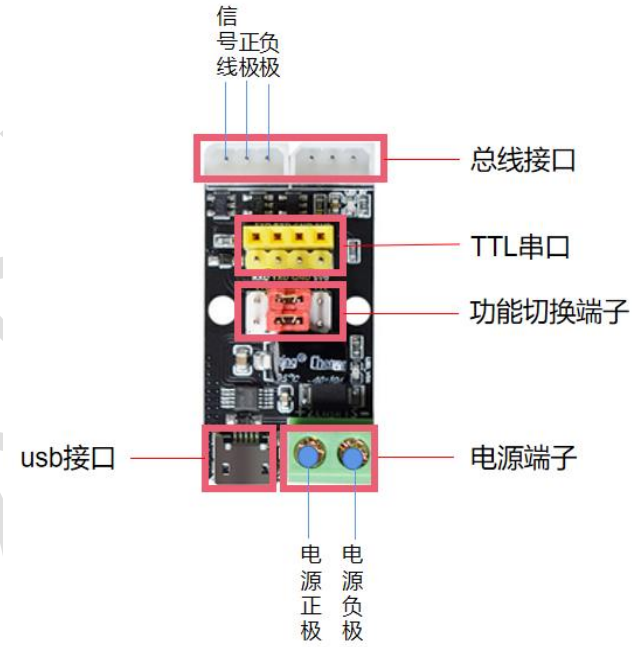
一、认识各模块

1.总线对接线



关于总线对接线具体可参考淘宝详情页 https://item.taobao.com/item.htm?spm=a1z10.5-c-s.w4002-18555111852.30.541224b6PScBKc&id=552997666098

2.zlink 模块



注意：

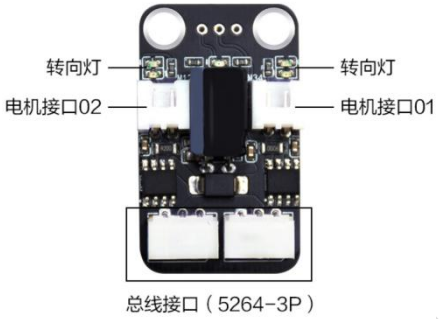
（1）我们使用时，功能切换端子处的跳线帽在上图中是接到中间的，我们这里要接到 TTL 和 ZX，也就是左移一下，接到偏左的位置。

（2）电源端子处接电源对接线，电源对接线再连接电源。

关于 zlink 模块也可参考淘宝详情页 https://item.taobao.com/item.htm?spm=a1z10.5-c-s.w4002-18555111852.40.541

224b6PScBKc&id=570100064201

3.总线双路驱动



注意：我们使用时，总线接口随便接哪一个都可以，但是需要注意两个马达的接法，此时以上图的角度来说，小车的左侧马达接电机接口 02，右侧马达接电机接口 01

二、接线说明

1.电源---->电源对接线---->zlink 模块

2.zlink 模块---->总线对接线---->扩展总线板（openmv 是插在我们的扩展总线板上的）

3.扩展总线板---->总线对接线---->总线双路驱动

4.总线双路驱动---->马达

三、示例代码

```
'''
马达的控制
此处控制马达用到了我们的总线双路驱动模块
使用 openmv 的串口 3，波特率为 115200
通过串口发送指令字符串来控制
指令格式为 #idPpwmTtime!
其中 id 我们需要设置总线双路驱动的 id 为 006，以马达接口朝上的方向，则左轮马达接左侧马达接口，右轮马达接右侧马达接口
pwm 范围为 0500-2500，中位为 1500,当 pwm 值小于 1500 或大于 1500 时为正转或反转，具体视马达安装方向而定
time 为时间（毫秒为单位），如果是 0000 则电机一直转

'''
import time
from pyb import UART

# 初始化函数
def init_setup():
    global uart3              # 设置为全局变量
    uart3 = UART(3, 115200)   # 实例化串口 3，波特率设置为 115200

# 马达停止转动
def middle():
    uart3.write('#006P1500T0000!#007P1500T0000!')

# 主函数
def main():
```

```
    for i in range(3):
        uart3.write('#006P1200T1000!#007P1800T1000!')  # 后退 1 秒
        time.sleep(1)          # 延时 1 秒
        uart3.write('#006P1800T1000!#007P1200T1000!')  # 前进 1 秒
        time.sleep(1)          # 延时 1 秒

# 程序入口
if __name__ == '__main__':
    init_setup()               # 执行初始化函数初始化串口
    try:                       # 异常处理
        main()                 # 执行主函数
    finally:
        middle()
```

四、运行效果

小车先前进 1 秒，再后退 1 秒，循环 3 次。

五、知识点

在代码中可以看到，我们发送的指令格式为#IndexPpwmTtime!

指令格式解析：单个马达指令，Index 为 3 位，000-254；pwm 为 4 位，0500-2500；time 为 4 位，0000-9999，单位毫秒，总共 15 位数据，不足的位数补 0

其中：pwm 为 1500 时，马达停止转动；time 为 0 时，马达会一直以 pwm 的值转动。

【本课小结】：

本课学习了使用 OpenMV+扩展总线板+电源+zlink 模块+总线对接线+总线双路驱动控制 TT 马达，为我们后面控制小车做准备。

需要注意：此处控制我们的马达是通过串口发送指令字符串来控制的。

进阶篇

第 1 课 颜色识别

【本课目标】：

本课学习使用 openmv 实现识别颜色的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

注意：在本节的源码例程中会有两个源码文件：multi_color_detection.py 和 single_color_detection.py

multi_color_detection.py 是可以同时识别多种颜色

single_color_detection.py 是只能识别一种颜色

根据自己的需要来选择一个使用即可，当然也可以两个都试一下。

注意在使用代码之前需要先将阈值范围调整一下，在下面有具体调节阈值的方法。

一、代码

单颜色识别示例代码

```
'''''
单种颜色识别
每次识别一种颜色，这里有红绿蓝三种颜色可以选择，还可自行添加其他颜色阈值来识别其他颜色
颜色阈值采集方法：
方法一：
    1.可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集，
    2.拖动 6 个滑块来调整图像，直到要采集的颜色都呈白色，其他都呈黑色为止，
    3.采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
方法二：
    1.可以在右侧‘帧缓冲区’和‘直方图’的’LAB 色彩空间‘配合采集
    2.先在帧缓冲区中用鼠标左键框出要采集的颜色区域，
    3.然后在直方图中的 LAB 色彩空间中，分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
'''
import sensor, time

# 颜色阈值索引，可以改变索引值来识别不同颜色此处 0 为红色，1 为绿色，2 为蓝色
threshold_index = 0

# 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
thresholds = [(35, 56, 96, 48, -81, 96),   # 红色阈值
              (21, 96, -49, -33, 20, 47),  # 绿色阈值
              (16, 46, 9, 31, -15, -115)]  # 蓝色阈值

# 初始化函数
def init_setup():
    global sensor, clock                    # 设置为全局变量
```

```
27.     sensor.reset()                          # 初始化感光元件
28.     sensor.set_pixformat(sensor.RGB565)     # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图，GRAYSCALE 为灰度图)
29.     sensor.set_framesize(sensor.QVGA)       # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
30.     sensor.skip_frames(time=2000)           # 等待 2 秒，使以上的设置生效
31.     sensor.set_auto_gain(False)             # 关闭自动增益。在颜色识别中，需要关闭自动增益
32.     sensor.set_auto_whitebal(False)         # 关闭白平衡。白平衡是默认开启的，在颜色识别中，需要关闭白平衡
33.     clock = time.clock()                    # 创建时钟对象
34.
35. # 主函数
36. def main():
37.     while 1:                                # 无限循环
38.         clock.tick()                        # 更新 FPS 帧率时钟
39.         img = sensor.snapshot()             # 拍一张照片并返回图像
40.         # 循环 3 种颜色阈值，在图像中查看是否有对应颜色并做一些操作，[thresholds[threshold_index]]为列表格式
41.         for blob in img.find_blobs([thresholds[threshold_index]], pixels_threshold=200, area_threshold=200):
42.             img.draw_rectangle(blob.rect())         # 将目标颜色画框
43.             img.draw_cross(blob.cx(), blob.cy())    # 在目标颜色中心位置画个十字
44.         print("FPS %f" % clock.fps())  # 打印帧率
45.
46. # 程序入口
47. if __name__ == '__main__':
48.     init_setup()                            # 执行初始化函数
49.     try:                                    # 异常处理
50.         main()                              # 执行主函数
51.     except:
52.         pass
```

多颜色同时识别示例代码

```
1.  '''''
2.  多颜色同时识别
3.  红绿蓝三种颜色可以同时识别，还可自行添加其他颜色阈值来识别其他颜色
4.  颜色阈值采集方法：
5.  方法一：
6.      1.可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集，
7.      2.拖动 6 个滑块来调整图像，直到要采集的颜色都呈白色，其他都呈黑色为止，
8.      3.采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
9.  方法二：
10.     1.可以在右侧‘帧缓冲区’和‘直方图’的’LAB 色彩空间‘配合采集
11.     2.先在帧缓冲区中用鼠标左键框出要采集的颜色区域，
12.     3.然后在直方图中的 LAB 色彩空间中，分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
13. '''
14. import sensor, time
15.
16. # 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
17. thresholds = [(0, 23, 50, 20, 29, 1),    # 红色阈值
18.               (21, 96, -49, -33, 20, 47),  # 绿色阈值
19.               (6, 13, -13, 11, -13, -97)]  # 蓝色阈值
20.
21. # 初始化函数
```

```
def init_setup():
    global sensor, clock                    # 设置为全局变量
    sensor.reset()                          # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)     # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图，GRAYSCALE 为灰度图)
    sensor.set_framesize(sensor.QVGA)       # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
    sensor.skip_frames(10)                  # 跳过一些帧，使以上的设置生效
    sensor.set_auto_gain(False)             # 关闭自动增益。在颜色识别中，需要关闭自动增益
    sensor.set_auto_whitebal(False)         # 关闭白平衡。白平衡是默认开启的，在颜色识别中，需要关闭白平衡
    clock = time.clock()                    # 创建时钟对象

# 主函数
def main():
    while 1:                                # 无限循环
        clock.tick()                        # 更新 FPS 帧率时钟
        img = sensor.snapshot()             # 拍一张照片并返回图像
        # 循环 3 种颜色阈值，在图像中查看是否有对应颜色并做一些操作，thresholds 需为列表格式
        for blob in img.find_blobs(thresholds, pixels_threshold=200, area_threshold=200):
            img.draw_rectangle(blob.rect())         # 将目标颜色画框
            img.draw_cross(blob.cx(), blob.cy())    # 在目标颜色中心位置画个十字
        print("FPS %f" % clock.fps())  # 打印帧率

# 程序入口
if __name__ == '__main__':
    init_setup()                            # 执行初始化函数
    try:                                    # 异常处理
        main()                              # 执行主函数
    except:
        pass
```

二、代码使用方法

1.在资料中的'源码例程'文件夹中找到颜色识别文件夹，然后选择单颜色识别或多颜色识别代码在 openmv ide 中打开。此处以单颜色识别为例

2.替换颜色阈值。

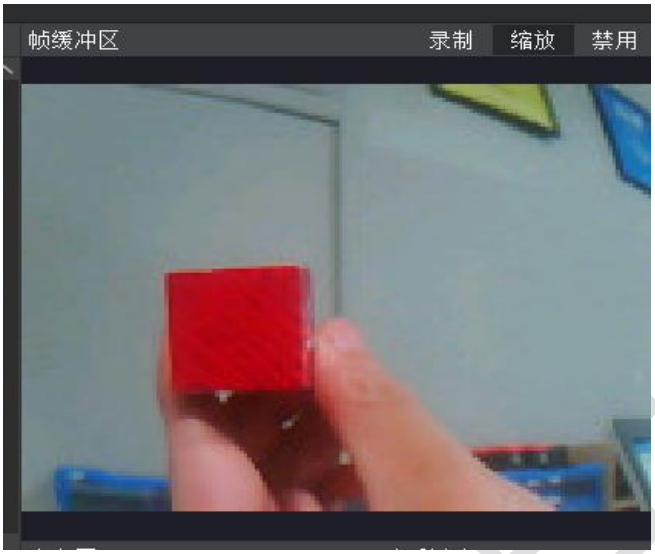
阈值查找方式这里主要介绍两种，都是在 openmv ide 中来操作的：

阈值查找方式一：

（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要识别颜色的物体放在摄像头画面内，(注意不要有相近颜色的物体干扰，否则效果不好)

（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



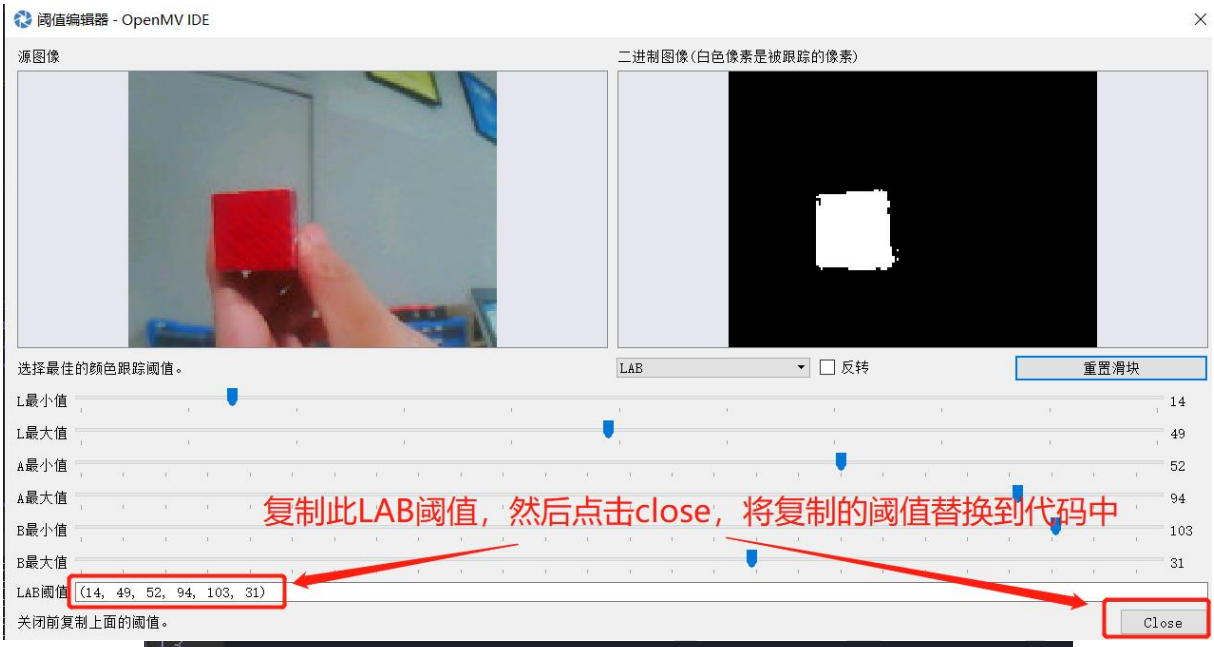
（4）然后点击'工具'-->'机器视觉'-->'阈值编辑器'，选择'帧缓冲区'



（5）看到如下界面



调整到如下图所示情况时即可复制 LAB 阈值到代码中使用了。



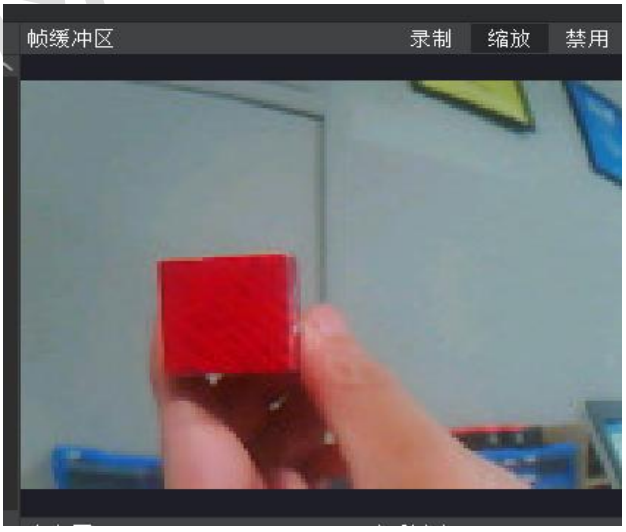

```
14  import sensor, time
15
16  # 颜色阈值索引，可以改变索引值来识别不同颜色此处0为红色，1为绿
17  threshold_index = 0
18
19  # 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
20  thresholds = [(14,  49,  52,  94,  103,  31)    # 红色阈值
21                (21,  96,  -49,  -33,  20,  47),   # 绿色阈值
22                (16,  46,  9,  31,  -15,  -115)]   # 蓝色阈值
```

阈值查找方式二：

（1）在 openmv ide 中运行刚才打开的源码文件，

（2）然后将要识别颜色的物体放在摄像头画面内，(注意不要有相近颜色的物体干扰，否则效果不好)

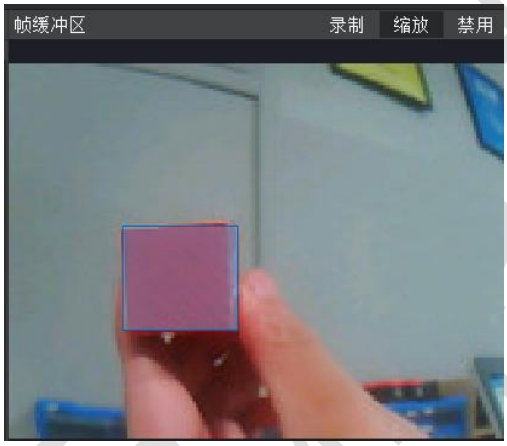
（3）然后点击红色×号按钮停止运行程序，此时就可以在右上方的帧缓冲区中看到图像画面，



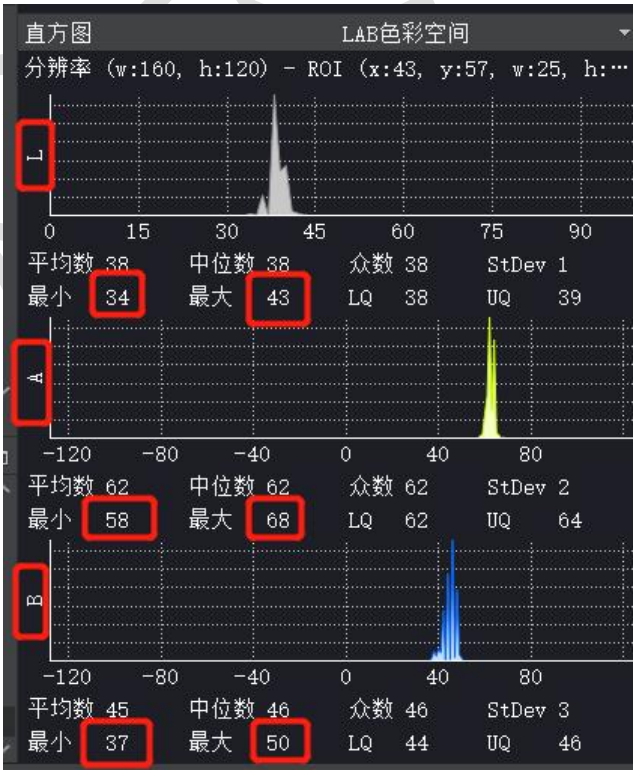
（4）在'直方图'中选择'LAB 色彩空间'，



（5）在'帧缓冲区'中用鼠标左键将要识别的颜色框出来，比如这里我将红色木块框出来了



（6）观察'直方图'中的 L、A、B 的最小值与最大值(也可将鼠标左键分别悬停在 L、A、B 的 x 轴上查看具体数值)，如下图 L 最小 34，最大 43，A 最小 58，最大 68，B 最小 37，最大 50，则 LAB 阈值为(34,43,58,68,37,50)，替换到代码中使用就可以了。



```
14  import sensor, time
15
16  # 颜色阈值索引，可以改变索引值来识别不同颜色此处0为红色，1为
17  threshold_index = 0
18
19  # 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
20  thresholds = [(34, 43, 58, 68, 37, 50)     # 红色阈值
21                (21,  96, -49, -33,  20,  47),  # 绿色阈值
22                (16,  46,  9,  31, -15, -115)]  # 蓝色阈值
23
```

3.确定并替换好代码中的颜色阈值之后，就可以运行此代码了。然后将要识别的物体放在 openmv 摄像头前面随便移动，看看 ide 中的帧缓冲区是否会将该物体框出来即可。

三、运行效果

将要识别的物体放在 openmv 摄像头前面随便移动，ide 中的帧缓冲区里会将识别到的目标颜色的物体框出来。

单颜色识别只会框出要识别的那一种颜色的物体，而多颜色识别可以将要识别的多种颜色的多个物体同时框出来。

四、知识点

关于 img.find_blobs 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_blobs，可以查找到 img.find_blobs 的使用方法及其对象 blob 的方法属性的使用。

【本课小结】：

本课学习了使用 OpenMV 进行颜色识别，注意调节颜色阈值

第 2 课 颜色跟随

【本课目标】：

本课学习使用 openmv 实现云台跟随目标颜色移动的功能

【课前准备】：

OpenMV、USB 数据线、扩展板、云台、zlink 模块、总线对接线

【本课内容】：

一、代码

```
import sensor, image, time

from pid import PID
from pyb import Servo

target_threshold  = (2, 88, -23, 82, 34, 127)    # 此处以黄色为例

# 初始化函数
def init_setup():
    global pan_servo, tilt_servo, pan_pid, tilt_pid, sensor, clock
    pan_servo = Servo(1)                         # 实例化云台靠下的舵机
    tilt_servo = Servo(2)                        # 实例化云台靠上的舵机

    pan_servo.calibration(500,2500,1500)         # 设置舵机脉冲范围为 500-2500，中心位置为 1500
    tilt_servo.calibration(500,2500,1500)        # 设置舵机脉冲范围为 500-2500，中心位置为 1500

    #pan_pid = PID(p=0.07, i=0, imax=90)          # 脱机运行或者禁用图像传输，使用这个 PID
    #tilt_pid = PID(p=0.05, i=0, imax=90)         # 脱机运行或者禁用图像传输，使用这个 PID
    pan_pid = PID(p=0.12, i=0, imax=90)          # 在线调试使用这个 PID
    tilt_pid = PID(p=0.12, i=0, imax=90)         # 在线调试使用这个 PID

    sensor.reset()                               # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)          # 图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)           # 图像大小设置为 QQVGA，即 160x120
    sensor.skip_frames(10)                       # 跳过 10 帧，使以上设置生效
    sensor.set_auto_whitebal(False)              # 关闭白平衡
    clock = time.clock()                         # 创建时钟对象

# 舵机回归中位
def middle():
    pan_servo.angle(0, 1000)                     # 舵机在 1 秒内转到 0 度位置
    tilt_servo.angle(0, 1000)                    # 舵机在 1 秒内转到 0 度位置
    time.sleep(1)                                # 延时 1 秒

# 查找识别到的目标颜色区域中最大区域的函数
def find_max(blobs):
    max_size = 0
    for blob in blobs:
        if blob[2] * blob[3] > max_size:
```

```
            max_blob = blob
            max_size = blob[2] * blob[3]
    return max_blob

# 主函数
def main():
    while 1:
        clock.tick()                            # 更新 FPS 帧率时钟
        img = sensor.snapshot()                 # 拍一张照片并返回图像

        blobs = img.find_blobs([target_threshold])  # 查找图像中的目标颜色
        if blobs:
            max_blob = find_max(blobs)          # 找到最大的目标颜色区域
            pan_error = max_blob.cx() - img.width() / 2
            tilt_error = max_blob.cy() - img.height() / 2

            #print("tilt_error: ", tilt_error)

            img.draw_rectangle(max_blob.rect())  # 将最大的目标颜色区域框出来
            img.draw_cross(max_blob.cx(), max_blob.cy())  # 在目标颜色区域的中心点处画十字

            pan_output = pan_pid.get_pid(pan_error, 1) / 2
            tilt_output = tilt_pid.get_pid(tilt_error, 1) / 2
            #print("tilt_output", tilt_output)
            pan_servo.angle(pan_servo.angle() - pan_output)
            tilt_servo.angle(tilt_servo.angle() - tilt_output)

            # 异常判断，定义舵机极限位置
            if pan_servo.angle() > 90:
                pan_servo.angle(90)
            if pan_servo.angle() < -90:
                pan_servo.angle(-90)
            if tilt_servo.angle() > 45:
                tilt_servo.angle(45)
            if tilt_servo.angle() < -60:
                tilt_servo.angle(-60)

# 程序入口
if __name__ == '__main__':
    init_setup()    # 执行初始化函数
    try:
        main()      # 执行主函数
    finally:
        middle()    # 程序结束时使云台舵机复位
```

二、代码使用方法

1. 用 openmv ide 打开 color_tracking.py，调节代码中的颜色阈值，具体调节方法可参考颜色识别那一节。

2.将 pid.py 文件复制粘贴到 openmv 的盘符中，然后用 openmv ide 打开

color_tracking.py 运行使用即可

三、运行效果

运行代码后，openmv 摄像头可以识别到红绿蓝三种颜色，当然你也可以继续增加其他颜色的阈值到代码中。

【本课小结】：

本课学习了使用 openmv 实现云台跟随目标颜色移动的功能

众灵科技

第 3 课 人脸识别

【本课目标】：

本课学习使用 openmv 实现识别人脸的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

这个例子展示了 OpenMV Cam 的内置人脸检测功能。

人脸检测通过在图像上使用 Haar Cascade 特征检测器来工作。 haar 级联是一系列简单的区域对比检查。对于内置的前表面探测器，有 25 个阶段的检查，每个阶段有数百个检查一块。 Haar Cascades 运行速度很快，因为只有在以前的阶段过去后才会评估后期阶段。 此外，您的 OpenMV 使用称为整体图像的数据结构来在恒定时间内快速执行每个区域对比度检查（特征检测仅为灰度的原因是因为整体图像的空间需求）。

一、代码

```
"""
人脸识别例程
这个例子展示了 OpenMV Cam 的内置人脸检测功能。
人脸检测通过在图像上使用 Haar Cascade 特征检测器来工作。 haar 级联是一系列简单的区域对比检查。
对于内置的前表面探测器，有 25 个阶段的检查，每个阶段有数百个检查一块。 Haar Cascades 运行速度
很快，因为只有在以前的阶段过去后才会评估后期阶段。 此外，您的 OpenMV 使用称为整体图像的数据
结构来在恒定时间内快速执行每个区域对比度检查
（特征检测仅为灰度的原因是因为整体图像的空间需求）。
"""
import sensor, image, time

# 初始化函数
def init_setup():
    # 设置为全局变量
    global sensor, face_cascade, clock
    # 重置感光元件
    sensor.reset()
    # 感光元件设置
    sensor.set_contrast(3)
    sensor.set_gainceiling(16)
    # HQVGA 和灰度对于人脸识别效果最好，240x160
    sensor.set_framesize(sensor.HQVGA)
    # 注意人脸识别只能用灰度图哦
    sensor.set_pixformat(sensor.GRAYSCALE)
    # 创建时钟对象
    clock = time.clock()

    # 加载 Haar 算子
```

```
    # 默认情况下，这将使用所有阶段，更低的 satges 更快，但不太准确。
    face_cascade = image.HaarCascade("frontalface", stages=25)
    # image.HaarCascade(path, stages=Auto)加载一个 haar 模型。haar 模型是二进制文件，
    # 这个模型如果是自定义的，则引号内为模型文件的路径；也可以使用内置的 haar 模型，
    # 比如“frontalface” 人脸模型或者“eye”人眼模型。
    # stages 值未传入时使用默认的 stages。stages 值设置的小一些可以加速匹配，但会降低准确率。
    # print(face_cascade)

# 主函数
def main():
    clock.tick()                    # 更新 FPS 帧率时钟
    img = sensor.snapshot()         # 拍摄一张照片

    objects = img.find_features(face_cascade, threshold=1, scale=1.25)
    # image.find_features(cascade, threshold=0.5, scale=1.5)
    # thresholds 越大，匹配速度越快，错误率也会上升。scale 可以缩放被匹配特征的大小。
    if objects:                     # 检测到人脸时
        for r in objects:
            img.draw_rectangle(r)   # 在找到的目标上画框，标记出来
            print(r)                # 打印人脸的左上角坐标、人脸宽度高度
    else:                           # 未检测到人脸时
        print("FPS %f" % clock.fps())   # 打印帧率

# 程序入口
if __name__ == '__main__':
    init_setup()                    # 初始化函数
    try:                            # 异常处理
        while 1:                    # 无限循环
            main()                  # 执行主函数
    except:
        pass
```

二、运行效果

运行之后，将人脸暴露在 openmv 摄像头画面中时，它会将人脸框出来。

三、知识点

关于 img.find_features 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_features，可以查找到 img.find_features 的使用方法及 HaarCascade 类 – 特征描述符的使用。

【本课小结】：

本课学习了使用 openmv 实现人脸识别

第 4 课 人脸跟随

【本课目标】：

本课学习使用 openmv 实现云台跟随人脸移动的功能

【课前准备】：

OpenMV、USB 数据线、扩展板、云台、zlink 模块、总线对接线

【本课内容】：

一、代码

```
import sensor, image, time

from pid import PID
from pyb import Servo


# 初始化函数
def init_setup():
    global pan_servo, tilt_servo, pan_pid, tilt_pid, sensor, clock, face_cascade
    pan_servo = Servo(1)                        # 实例化云台靠下的舵机
    tilt_servo = Servo(2)                       # 实例化云台靠上的舵机

    pan_servo.calibration(500,2500,1500)        # 设置舵机脉冲范围为 500-2500，中心位置为 1500
    tilt_servo.calibration(500,2500,1500)       # 设置舵机脉冲范围为 500-2500，中心位置为 1500

    #pan_pid = PID(p=0.07, i=0, imax=90)         # 脱机运行或者禁用图像传输，使用这个 PID
    #tilt_pid = PID(p=0.05, i=0, imax=90)        # 脱机运行或者禁用图像传输，使用这个 PID
    pan_pid = PID(p=0.12, i=0, imax=90)         # 在线调试使用这个 PID
    tilt_pid = PID(p=0.12, i=0, imax=90)        # 在线调试使用这个 PID

    sensor.reset()                              # 初始化感光元件
    sensor.set_contrast(3)                      # 对比度
    sensor.set_gainceiling(16)                  # 自动增益
    sensor.set_pixformat(sensor.GRAYSCALE)      # 图像格式设置为灰度
    sensor.set_framesize(sensor.HQVGA)          # 图像大小设置为 HQVGA，即 240x160
    sensor.skip_frames(10)                      # 跳过 10 帧，使以上设置生效
    sensor.set_auto_whitebal(False)             # 关闭白平衡
    clock = time.clock()                        # 创建时钟对象
    face_cascade = image.HaarCascade("frontalface", stages=25)

# 舵机回归中位
def middle():
    pan_servo.angle(0, 1000)                    # 舵机在 1 秒内转到 0 度位置
    tilt_servo.angle(0, 1000)                   # 舵机在 1 秒内转到 0 度位置
    time.sleep(1)                               # 延时 1 秒

# 查找识别到的目标颜色区域中最大区域的函数
def find_max(blobs):
```

```
    max_size = 0
    for blob in blobs:
        if blob[2] * blob[3] > max_size:
            max_blob = blob
            max_size = blob[2] * blob[3]
    return max_blob

# 主函数
def main():
    while 1:
        clock.tick()                            # 更新 FPS 帧率时钟
        img = sensor.snapshot()                 # 拍一张照片并返回图像

        faces = img.find_features(face_cascade, threshold=0.75, scale=1.25)
        if faces:
            face = find_max(faces)
            cx = int(face[0] + face[2] / 2)
            cy = int(face[1] + face[3] / 2)
            pan_error = cx - img.width() / 2
            tilt_error = cy - img.height() / 2

            #print("tilt_error: ", tilt_error)

            img.draw_rectangle(face)            # 将最大的目标区域框出来
            img.draw_cross(cx, cy)              # 在目标区域的中心点处画十字

            pan_output = pan_pid.get_pid(pan_error, 1) / 2
            tilt_output = tilt_pid.get_pid(tilt_error, 1) / 2
            #print("tilt_output", tilt_output)
            pan_servo.angle(pan_servo.angle() - pan_output)
            tilt_servo.angle(tilt_servo.angle() - tilt_output)

            # 异常判断，定义舵机极限位置
            if pan_servo.angle() > 90:
                pan_servo.angle(90)
            if pan_servo.angle() < -90:
                pan_servo.angle(-90)
            if tilt_servo.angle() > 45:
                tilt_servo.angle(45)
            if tilt_servo.angle() < -60:
                tilt_servo.angle(-60)

# 程序入口
if __name__ == '__main__':
    init_setup()    # 执行初始化函数
    try:
        main()      # 执行主函数
    finally:
        middle()    # 程序结束时使云台舵机复位
```

二、代码使用方法

先将 pid.py 文件保存到 openmv 的盘符中，然后用 openmv ide 打开 face_tracking.py 运行使用即可

三、运行效果

人脸在 openmv 摄像头的视野中移动时，云台会跟着人脸移动

四、知识点

关于 img.find_features 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_features，可以查找到 img.find_features 的使用方法及 HaarCascade 类 – 特征描述符的使用。

【本课小结】：

本课学习了使用 openmv 实现人脸跟随的功能

第 5 课 分辨不同人脸

【本课目标】：

本课学习使用 openmv 实现人脸面部特征识别的功能，即分辨不同人脸

【课前准备】：

OpenMV（需要插上 SD 卡）、USB 数据线

【本课内容】：

注意：

1. 本节课学习的是使用 openmv 实现分辨不同人脸的功能，openmv 需要插上 SD 卡来使用，否则内置 flash 可能不够用，因为我们要将一些人脸图片保存到 openmv 中。

2. 本节对应的源码例程中有两个文件：face_collection.py 和 face_recognition.py face_collection.py 是采集人脸，face_recognition.py 是分辨不同人脸。

要先根据教程采集人脸，然后再来分辨人脸

一、代码

采集人脸的代码：

```
# 人脸图片采集
# 采集人脸图片到 openmv 中
# 先在 openmv 的盘符中新建文件夹 ZLTech，然后在该文件夹中新建文件夹 s1，s2，要采集几个人就新建到 s 几
# 红灯亮做准备，蓝灯亮是正在拍照
import sensor, image, pyb

RED_LED_PIN = 1         # 红灯引脚
BLUE_LED_PIN = 3        # 蓝灯引脚
SAVE_PATH = 'ZLTech'    # 图片保存的位置
num = 1 # 设置被拍摄者序号，第一个人的图片保存到 s1 文件夹，第二个人的图片保存到 s2 文件夹，以此类推。每次更换拍摄者时，修改 num 值。
n = 20  # 设置每个人拍摄图片数量。

# 初始化函数
def init_setup():
    global sensor
    sensor.reset()                          # 初始化感光元件
    sensor.set_pixformat(sensor.GRAYSCALE)  # 设置图像格式为灰度图
    sensor.set_framesize(sensor.B128X128)   # 设置图像大小为 128x128
    sensor.set_windowing((92,112))          # 设置窗口大小为 92x112
    sensor.skip_frames(10)                  # 使以上设置生效
    sensor.skip_frames(time=2000)

# 主函数
```

```
def main():
    global n
    #连续拍摄 n 张照片，每间隔 3s 拍摄一次。
    while (n):
        #红灯亮
        pyb.LED(RED_LED_PIN).on()
        sensor.skip_frames(time=3000)    # 等待 3s，准备一下表情。

        #红灯灭，蓝灯亮
        pyb.LED(RED_LED_PIN).off()
        pyb.LED(BLUE_LED_PIN).on()

        #保存截取到的图片到 SD 卡
        print(n)
        sensor.snapshot().save("%s/s%s/%s.pgm" % (SAVE_PATH, num, n) )  # or "example.bmp" (or others)

        n -= 1

        pyb.LED(BLUE_LED_PIN).off()     # 蓝灯灭
        print("Done! Reset the camera to see the saved image.")

# 程序入口
if __name__ == '__main__':
    init_setup()  # 执行初始化函数
    try:          # 异常处理
        main()    # 执行主函数
    except:
        pass
```

分辨人脸的代码：

```
# 使用 LBP 分辨不同人脸
# 运行此代码前请先运行采集人脸的代码来采集人脸，如果已采集完成，可以运行此代码

import sensor, time, image, pyb

NUM_SUBJECTS = 2                              # 图像库中不同人数
NUM_SUBJECTS_IMGS = 20                        # 每人有 20 张样本图片
name_list = [None, '迪丽热巴', '鞠婧炜']      # 对应人的名字

# 初始化函数
def init_setup():
    global sensor,face_cascade                # 设置为全局变量
    sensor.reset()                            # 初始化感光元件
    sensor.set_pixformat(sensor.GRAYSCALE)    # 将图像格式设置为灰度
    sensor.set_framesize(sensor.B128X128)     # 将图像大小设置为 128x128
    face_cascade = image.HaarCascade("frontalface", stages=25)
    sensor.set_windowing((92,112))            # 将窗口大小设置为 92x112
    sensor.skip_frames(10)
    sensor.skip_frames(time=1000)             # 等待 1s，使以上设置生效

def min(pmin, a, s):
    global num
```

```
    if a < pmin:
        pmin = a
        num = s
    return pmin

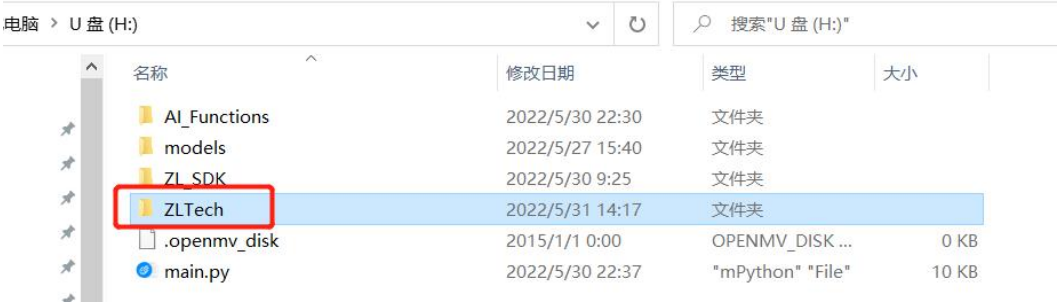
# 主函数
def main():
    global num
    # 拍摄当前人脸
    img = sensor.snapshot()
    # d0 为当前人脸的 lbp 特征
    d0 = img.find_lbp((0, 0, img.width(), img.height()))
    objects = img.find_features(face_cascade, threshold=1, scale=1.25)
    if objects:                    # 检测到人脸时
        img = None
        pmin = 999999
        num = 0
        for s in range(1, NUM_SUBJECTS+1):
            dist = 0
            for i in range(1, NUM_SUBJECTS_IMGS+1):
                img = image.Image("ZLTech/s%d/%d.pgm"%(s, i))
                d1 = img.find_lbp((0, 0, img.width(), img.height()))
                # d1 为第 s 文件夹中的第 i 张图片的 lbp 特征
                dist += image.match_descriptor(d0, d1)  # 计算 d0 d1 即样本图像与被检测人脸的特征差异度。
            print("Average dist for subject %d: %d"%(s, dist/NUM_SUBJECTS_IMGS))  # 打印与每个人的特征差异度
            pmin = min(pmin, dist/NUM_SUBJECTS_IMGS, s) # 特征差异度越小，被检测人脸与此样本更相似更匹配。

        print(name_list[num]) # num 为当前最匹配的人的编号,然后从名字列表中取出对应人的名字。

# 程序入口
if __name__ == '__main__':
    init_setup()      # 执行初始化函数
    try:              # 异常处理
        while 1:
            main()    # 执行主函数
    except:
        pass
```

二、代码使用方法

1.在 openmv 的盘符中新建文件夹 ZLTech，然后在该文件夹中新建文件夹 s1，s2...，要采集几个人就新建到 s 几，比如我要分辨两个人，我就在 ZLTech 文件夹中新建 s1 和 s2





2.在 openmv ide 中打开采集人脸的代码 face_collection.py

```
import sensor, image, pyb

RED_LED_PIN = 1          # 红灯引脚
BLUE_LED_PIN = 3         # 蓝灯引脚
SAVE_PATH = 'ZLTech'     # 图片保存的位置
num = 1 # 设置被拍摄者序号，第一个人的图片保存到s1文件夹，第二个人的图片保

n = 20  # 设置每个人拍摄图片数量。
```

运行程序即可开始拍照。num=1 表示拍第一个人的脸，当你拍完第一个人的 20 张照片后，可以将代码中的 num=1 改为 num=2，然后再次运行程序来拍第二个人的脸。

拍照时，红灯亮为做准备，蓝灯亮表示正在拍照。

3.在 openmv ide 中打开分辨人脸的代码 face_recognition.py

```
# 使用LBP分辨不同人脸
# 运行此代码前请先运行采集人脸的代码来采集人脸，如果已采集完成，可以运行此代码

import sensor, time, image, pyb

NUM_SUBJECTS = 2                                    # 图像库中不同人数
NUM_SUBJECTS_IMGS = 20                              # 每人有20张样本图片
name_list = [None, '迪丽热巴', '鞠婧炜']             # 对应人的名字
```

NUM_SUBJECTS = 2 表示采集人脸时一共采集了两个人的照片，根据实际情况修改即可，我这里是两个人，所以这里 NUM_SUBJECTS = 2

name_list = [None, '迪丽热巴', '鞠婧炜']是对应人的名字，比如我的 s1 中存的照片对应的是'迪丽热巴'，s2 中存的照片对应的是'鞠婧炜'，你可以根据你的实际情况进行增加修改等。

4.运行分辨人脸的代码 face_recognition.py 即可

三、运行效果

运行代码后，将已采集过图片的那个人的脸暴露在摄像头画面中时，ide 的串行端口窗口中就会打印出这个人的名字。我们有提供的测试图片。

四、知识点

关于 img.find_lbp 方法可以在官方文档找到
https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#
打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_lbp，可以查找到 img.find_lbp 的使用方法。

【本课小结】：

本课学习了使用 openmv 通过 LBP 分辨不同人脸

众灵科技

第 6 课 线段识别

【本课目标】：

本课学习使用 openmv 实现识别线段的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

一、代码

```
# 线段检测例程
#
# 在图像中查找线段。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。

# find_line_segments()找到有限长度的线（但是很慢）。
# 用 find_line_segments()找到非无限的线（而且速度很快）。

enable_lens_corr = False                # 畸变矫正，打开可以获得更直的线条…

import sensor, image, time

# 初始化函数
def init_setup():
    global sensor, clock                # 设置为全局变量
    sensor.reset()                      # 重置感光元件
    sensor.set_pixformat(sensor.RGB565) # 此处设置的是彩色，如果是灰度会更快
    sensor.set_framesize(sensor.QQVGA)  # 设置图像分辨率为 QQVGA，即 160x120
    sensor.skip_frames(time=2000)       # 等待 2 秒，使以上设置生效
    clock = time.clock()                # 创建时钟对象

# 所有线段都有 `x1()`, `y1()`, `x2()`, 和`y2()` 方法来获得他们的终点
# 一个 `line()` 方法来获得所有上述的四个元组值，可用于 `draw_line()`.

# 主函数
def main():
    while 1:
        clock.tick()                    # 更新 FPS 帧率时钟
        img = sensor.snapshot()         # 拍一张图片并返回图像
        if enable_lens_corr:
            img.lens_corr(1.8)          # 畸变矫正，for 2.8mm lens...

        # `merge_distance`控制附近行的合并。 在 0（默认），没有合并。
        # 在 1 处，任何距离另一条线一个像素点的线都被合并...等等，
        # 因为你增加了这个值。 您可能希望合并线段，因为线段检测会产生大量的线段结果。

        # `max_theta_diff` 控制要合并的任何两线段之间的最大旋转差异量。
        # 默认设置允许 15 度。

```

```
        for l in img.find_line_segments(merge_distance=0, max_theta_diff=5):
            img.draw_line(l.line(), color=(255, 0, 0))
            print(l)

        print("FPS %f" % clock.fps())    # 打印帧率，注:实际 FPS 更高，流 FB 使它更慢。

# 程序入口
if __name__ == '__main__':
    init_setup()  # 执行初始化函数
    try:
        main()    # 执行主函数
    except:
        pass
```

二、运行效果

运行代码后，openmv 识别到的线段都会被标出来

三、知识点

关于 img.find_line_segments 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_line_segments，可以查找到 image.find_line_segments 的使用方法及 Line 类 – 直线对象的使用。

【本课小结】：

本课学习了使用 openmv 识别线段。

第 7 课 直线识别

【本课目标】：

本课学习使用 openmv 实现识别直线的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

本课学习如何在图像中查找线条。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。

一、代码

```
# 识别直线例程
#
# 这个例子展示了如何在图像中查找线条。对于在图像中找到的每个线对象，都会返回一个包含线条旋转的线对象。

# 注意：线条检测是通过使用霍夫变换完成的：
# http://en.wikipedia.org/wiki/Hough_transform
# 请阅读以上关于“theta”和“rho”的更多信息。

# find_lines（）找到无限长度的线。使用 find_line_segments（）来查找非无限线。

enable_lens_corr = False                    # 畸变矫正，打开可以获得更直的线条…
import sensor, image, time

# 初始化函数
def init_setup():
    global sensor, clock, min_degree, max_degree   # 设置为全局变量
    sensor.reset()                          # 重置感光元件
    sensor.set_pixformat(sensor.RGB565)     # 此处设置的是彩色，如果是灰度会更快
    sensor.set_framesize(sensor.QQVGA)      # 设置图像分辨率为 QQVGA，即 160x120
    sensor.skip_frames(time=2000)           # 等待 2 秒，使以上设置生效
    clock = time.clock()                    # 创建时钟对象

    # 所有的线对象都有一个`theta（）`方法来获取它们的旋转角度。您可以根据旋转角度来过滤线条。
    min_degree = 0
    max_degree = 179

# 所有线段都有 `x1()`, `y1()`, `x2()`, and `y2()` 方法来获得他们的终点
# 一个 `line()` 方法来获得所有上述的四个元组值，可用于 `draw_line()`.

# 主函数
def main():
    while 1:
        clock.tick()                        # 更新 FPS 帧率时钟
        img = sensor.snapshot()             # 拍一张图片并返回图像
        if enable_lens_corr:
            img.lens_corr(1.8)              # for 2.8mm lens...
```

```
37.
38.         # `threshold`控制从霍夫变换中监测到的直线。只返回大于或等于阈值的直线。应用程序的阈值正确值取决于图像。
39.         # 注意：一条直线的大小是组成直线所有索贝尔滤波像素大小的总和。
40.
41.         # `theta_margin`和`rho_margin`控件合并相似的直线。如果两直线的 theta 和ρ值差异小于边际，则它们合并。
42.
43.         for l in img.find_lines(threshold=1000, theta_margin=25, rho_margin=25):
44.             if (min_degree <= l.theta()) and (l.theta() <= max_degree):
45.                 img.draw_line(l.line(), color=(255, 0, 0))
46.                 print(l)
47.
48.         print("FPS %f" % clock.fps())            # 打印帧率，注:实际 FPS 更高，流 FB 使它更慢。
49.
50.  # 程序入口
51.  if __name__ == '__main__':
52.      init_setup()  # 执行初始化函数
53.      try:
54.          main()    # 执行主函数
55.      except:
56.          pass
57.
58.  # 关于负 rho 值:
59.  #
60.  # A [theta+0:-rho] tuple is the same as [theta+180:+rho].
61.  # A [theta+0:-rho]元组与[theta+180:+rho]相同。
```

二、运行效果

运行代码后，openmv 会将识别到的直线都标出来。

注意：此处的直线包括一些线只要它有一端被隔断在画面之外也算是直线。

三、知识点

关于 img.find_lines 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_lines，可以查找到 image.find_lines 的使用方法及 Line 类 – 直线对象的使用。

【本课小结】：

本课学习了使用 openmv 识别直线。

第 8 课 圆形识别

【本课目标】：

本课学习使用 openmv 实现识别圆形的功能。

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

本节学习使用霍夫变换在图像中查找圆

一、代码

```
# 圆形检测例程
#
# 用霍夫变换在图像中找到圆。
# https://en.wikipedia.org/wiki/Circle_Hough_Transform
#
# 请注意，find_circles（）方法将只能找到完全在图像内部的圆。圈子之外的图像/ roi 被忽略...

import sensor, image, time

# 初始化函数
def init_setup():
    global sensor, clock                    # 设置为全局变量
    sensor.reset()                          # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)     # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)      # 将图像大小设置为 QQVGA (160x120)
    sensor.skip_frames(10)                  # 跳过一些帧，使以上设置生效。
    clock = time.clock()                    # 创建时钟对象

# 主函数
def main():
    # 更新 FPS 帧率时钟
    clock.tick()

    # # 拍一张照片并返回图像，lens_corr(1.8)为畸变矫正
    img = sensor.snapshot().lens_corr(1.8)

    # Circle 对象有四个值: x, y, r (半径), 和 magnitude。
    # magnitude 是检测圆的强度。越高越好

    # roi 是一个用以复制的矩形的感兴趣区域(x, y, w, h)。如果未指定，
    # ROI 即图像矩形。操作范围仅限于 roi 区域内的像素。

    # x_stride 是霍夫变换时需要跳过的 x 像素的数量。若已知圆较大，可增加 x_stride 。

    # y_stride 是霍夫变换时需要跳过的 y 像素的数量。若已知直线较大，可增加 y_stride 。

    # threshold 控制从霍夫变换中检测到的圆。只返回大于或等于阈值的圆。
```

```
    # 应用程序的阈值正确值取决于图像。注意：一个圆的大小是组成圆所有索贝尔滤波像素大小的总和。

    # x_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。

    # y_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。

    # r_margin 控制所检测的圆的合并。圆像素为 x_margin 、 y_margin 和 r_margin 的部分合并。

    # r_min，r_max 和 r_step 控制测试圆的半径。缩小测试圆半径的数量可以大大提升性能。
    # threshold=3500 比较合适。如果视野中检测到的圆过多，请增大阈值；相反，如果视野中检测到的圆过少，请减少阈值。
    for c in img.find_circles(threshold=3500, x_margin=10, y_margin=10, r_margin=10, r_min=2, r_max=100, r_s
tep=2):
        img.draw_circle(c.x(), c.y(), c.r(), color = (255, 0, 0))
        print(c)

    print("FPS %f" % clock.fps())  # 打印帧率

# 程序入口
if __name__ == '__main__':
    init_setup()      # 执行初始化函数
    try:
        while 1:
            main()    # 执行主函数
    except:
        pass
```

二、运行效果

运行代码后，openmv 会将识别到的圆都用红线画出来。

注意：黑色实心圆会更易识别。

三、知识点

关于 img.find_circles 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_circles，可以查找到 image.find_circles 的使用方法及 Circle 类 – 圆形对象的使用。

【本课小结】：

本课学习了使用 openmv 识别圆形。

注意黑色实心圆会更易识别。

第 9 课 矩形识别

【本课目标】：

本课学习使用 openmv 实现识别矩形的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

本节学习使用四元检测算法来检测矩形。

一、代码

```
# 矩形识别例程
#
# 使用 april 标签代码中的四元检测代码在图像中找到矩形。
# 四元检测算法以非常稳健的方式检测矩形，并且比基于霍夫变换的方法好得多。
# 例如，即使镜头失真导致这些矩形看起来弯曲，它仍然可以检测到矩形。 圆角矩形是没有问题的！
# (但是，这个代码也会检测小半径的圆)...

import sensor, image, time

# 初始化函数
def init_setup():
    global sensor, clock                # 设置为全局变量
    sensor.reset()                      # 重置感光元件
    sensor.set_pixformat(sensor.RGB565) # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)  # 设置图像分辨率为 QQVGA，即 160x120
    sensor.skip_frames(10)              # 跳过 10 帧，使以上设置生效
    clock = time.clock()                # 创建时钟对象

# 主函数
def main():
    clock.tick()                    # 更新 FPS 帧率时钟
    img = sensor.snapshot()         # 拍一张图片并返回图像

    # 下面的`threshold`应设置为足够高的值，以滤除在图像中检测到的具有低边缘幅度的噪声矩形。最适用与背景形成鲜明对比的矩形。
    for r in img.find_rects(threshold=15000):
        img.draw_rectangle(r.rect(), color=(255, 0, 0))        # 用红线框出矩形
        for p in r.corners():
            img.draw_circle(p[0], p[1], 5, color=(0, 255, 0))  # 在矩形端点处画绿色圆
        print(r)

    print("FPS %f" % clock.fps())    # 打印帧率，注:实际 FPS 更高，流 FB 使它更慢。

# 程序入口
if __name__ == '__main__':
    init_setup()  # 执行初始化函数
    try:
```

```
        while 1:
            main()    # 执行主函数
    except:
        pass
```

二、运行效果

代码运行后，openmv 会将识别到的矩形框出来，并且四个顶点也会用小圆圈框出来。

注意：此代码也会将小半径的圆识别成矩形。

黑色实心矩形会更易识别。

三、知识点

关于 img.find_rects 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_rects，可以查找到 image.find_rects 的使用方法及 Rect 类 – 矩形对象的使用。

【本课小结】：

本课学习了使用 openmv 识别矩形。

注意：有些小半径的圆形也会被识别成矩形。

第 10 课 AprilTag 标签识别

【本课目标】：

本课学习使用 openmv 实现识别 AprilTag 标签信息的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

一、代码

```
# AprilTag 标签识别示例
import sensor, time

# 初始化函数
def init_setup():
    global sensor, clock                        # 设置为全局变量
    sensor.reset()                              # 重置感光元件
    sensor.set_pixformat(sensor.RGB565)         # 此处设置的是彩色
    sensor.set_framesize(sensor.QQVGA)          # 设置图像分辨率为
QQVGA，即 160x120
    sensor.skip_frames(time=2000)
    sensor.set_auto_gain(False)                 # must turn this off to
prevent image washout...
    sensor.set_auto_whitebal(False)             # must turn this off to
prevent image washout...
    clock = time.clock()                        # 创建时钟对象

# 主函数
def main():
    clock.tick()                            # 更新 FPS 帧率时钟
    img = sensor.snapshot()                 # 拍一张图片并返回图像
    apriltags = img.find_apriltags()        # 查找图像中的 AprilTag
    for tag in apriltags:
        img.draw_rectangle(tag.rect, color=(255, 0, 0))    # 用红线将
AprilTag 框出来
        img.draw_cross(tag.cx, tag.cy, color=(0, 255, 0))
        #print(tag)                         # 打印 AprilTag 的所有信息
        apriltag_id = tag.id                # 获取 AprilTag 的 id
        apriltag_family = tag.name          # 获取 AprilTag 所属家族
        print('AprilTag 的 id 为 %s，所属家族为 %s' % (apriltag_id,
apriltag_family))  # 打印 AprilTag 的 id 及所属家族
```

```
    #if not apriltags:                          # 如果图像中未发现 AprilTag
时打印帧率，注:实际 FPS 更高，流 FB 使它更慢。
        #print("FPS %f" % clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()                                # 执行初始化函数
    try:
        while 1:
            main()                              # 执行主函数
    except:
        pass
```

二、运行效果

代码运行后，openmv 会将识别到的 AprilTag 标签的 id 和所属家族打印到串行端口窗口中。

三、知识点

关于 img.find_apriltags 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_apriltags，可以查找到 image.find_apriltags 的使用方法及 AprilTag 类 – AprilTag 对象的使用。

【本课小结】：

本课学习了使用 openmv 实现识别 AprilTag 标签信息的功能。

第 11 课 二维码识别

【本课目标】：

本课学习使用 openmv 实现识别二维码信息的功能。

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

本节学习 OpenMV Cam 使用镜头校正来检测 QR 码的功能。

一、代码

```
# 二维码识别示例
#
# 这个例子展示了 OpenMV Cam 使用镜头校正来检测 QR 码的功能（请参阅 qrcodes_with_lens_corr.py 脚本以获得更高的性能）。
import sensor, image, time

# 初始化函数
def init_setup():
    global sensor, clock                        # 设置为全局变量
    sensor.reset()                              # 重置感光元件
    sensor.set_pixformat(sensor.RGB565)         # 此处设置的是彩色
    sensor.set_framesize(sensor.QVGA)           # 设置图像分辨率为 QVGA，即 320x240
    sensor.skip_frames(10)                      # 等待一些帧，使以上设置生效
    sensor.set_auto_gain(False)                 # 必须关闭白平衡功能，以防止图像冲洗…
    clock = time.clock()                        # 创建时钟对象

# 主函数
def main():
    clock.tick()                            # 更新 FPS 帧率时钟
    img = sensor.snapshot()                 # 拍一张图片并返回图像
    img.lens_corr(1.8)                      # 畸变矫正，1.8 的强度参数对于 2.8mm 镜头来说是不错的。
    codes = img.find_qrcodes()              # 查找图像中的二维码
    for code in codes:
        img.draw_rectangle(code.rect(), color = (255, 0, 0))    # 用红线将二维码框出来
        #print(code)                          # 打印二维码的所有信息
        msg = code.payload()                # 获取二维码的内容
        print('二维码的内容为：%s' % msg)  # 打印二维码的内容
    if not codes:                           # 如果图像中未发现二维码时打印帧率，注:实际 FPS 更高，流 FB 使它更慢。
        print("FPS %f" % clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()                            # 执行初始化函数
    try:
        while 1:
            main()                              # 执行主函数
    except:
```

```
37.         pass
```

二、运行效果

代码运行后，openmv 会将识别到的二维码信息打印到串行端口窗口中。

三、知识点

关于 img.find_qrcodes 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_qrcodes，可以查找到 image.find_qrcodes 的使用方法及 QRCode 类 – 二维码对象的使用。

【本课小结】：

本课学习了使用 openmv 实现识别二维码信息的功能

第 12 课 条形码识别

【本课目标】：

本课学习使用 openmv 实现识别条形码信息的功能

【课前准备】：

OpenMV、USB 数据线

【本课内容】：

本节学习使用 OpenMV 来检测条形码。

注意条形码检测不适用于 M4 相机。

条形码检测可以在 OpenMV Cam 的 OV7725 相机模块的 640x480 分辨率下运行。也可在 RGB565 模式下工作，但分辨率较低。也就是说，条形码检测需要更高的分辨率才能正常工作，因此应始终以 640x480 的灰度运行。

一、代码

```
# 条形码识别例程
#
# 使用 OpenMV 来检测条形码。条形码检测不适用于 M4 相机。

import sensor, image, time, math

# 初始化函数
def init_setup():
    global sensor, clock                    # 设置为全局变量
    sensor.reset()                          # 重置感光元件
    sensor.set_pixformat(sensor.GRAYSCALE)  # 此处设置的是灰度
    sensor.set_framesize(sensor.VGA)        # 设置图像分辨率为 VGA，即 640x480
    sensor.set_windowing((640, 80))         # V Res of 80 == less work (40 for 2X the speed).
    sensor.skip_frames(time=2000)           # 等待 2 秒，使以上设置生效
    sensor.set_auto_gain(False)             # 必须关闭此功能，以防止图像冲洗…
    sensor.set_auto_whitebal(False)         # 必须关闭此功能，以防止图像冲洗…
    clock = time.clock()                    # 创建时钟对象

    # 条形码检测可以在 OpenMV Cam 的 OV7725 相机模块的 640x480 分辨率下运行。
    # 条码检测也可在 RGB565 模式下工作，但分辨率较低。
    # 也就是说，条形码检测需要更高的分辨率才能正常工作，因此应始终以 640x480 的灰度运行。

# 判断条形码类型的函数
def barcode_name(code):
    if(code.type() == image.EAN2):
        return "EAN2"
    if(code.type() == image.EAN5):
        return "EAN5"
    if(code.type() == image.EAN8):
        return "EAN8"
```

```
    if(code.type() == image.UPCE):
        return "UPCE"
    if(code.type() == image.ISBN10):
        return "ISBN10"
    if(code.type() == image.UPCA):
        return "UPCA"
    if(code.type() == image.EAN13):
        return "EAN13"
    if(code.type() == image.ISBN13):
        return "ISBN13"
    if(code.type() == image.I25):
        return "I25"
    if(code.type() == image.DATABAR):
        return "DATABAR"
    if(code.type() == image.DATABAR_EXP):
        return "DATABAR_EXP"
    if(code.type() == image.CODABAR):
        return "CODABAR"
    if(code.type() == image.CODE39):
        return "CODE39"
    if(code.type() == image.PDF417):
        return "PDF417"
    if(code.type() == image.CODE93):
        return "CODE93"
    if(code.type() == image.CODE128):
        return "CODE128"

# 主函数
def main():
    while 1:
        clock.tick()                            # 更新 FPS 帧率时钟
        img = sensor.snapshot()                 # 拍一张图片并返回图像
        codes = img.find_barcodes()             # 从图像中查找条形码
        for code in codes:
            img.draw_rectangle(code.rect())
            print_args = (barcode_name(code), code.payload(), (180 * code.rotation())/math.pi, code.quality()
, clock.fps())
            print("Barcode %s, Payload \"%s\", rotation %f (degrees), quality %d, FPS %f" % print_args)
            msg = code.payload()                # 条形码的信息
            print('条形码信息为：%s' % msg)    # 打印条形码的内容
        if not codes:                           # 如果图像中未发现条形码时打印帧率，注:实际 FPS 更高，流 FB 使它更慢。
            print("FPS %f" % clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()                                # 执行初始化函数
    try:
        main()                                  # 执行主函数
    except:
        pass
```

二、运行效果

代码运行后，openmv 会将识别到的条形码的信息打印到串行端口窗口中。

三、知识点

关于 img.find_barcodes 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 find_barcodes，可以查找到 image.find_barcodes 的使用方法及 BarCode 类 – 条形码对象的使用。

【本课小结】：

本课学习了使用 openmv 实现识别条形码信息的功能.

第 13 课 数字识别（只适用于 openmv4 h7 plus）

【本课目标】：

本节学习通过训练神经网络模型实现识别数字的功能

【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现数字识别只适用于 openmv h7 plus。

2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 num_detection.py 来使用即可，可以用提供的配套的测试图片来进行测试。

3. 你也可以自己训练想要识别的一些数字的神经网络模型来使用，具体训练方法会在下面讲到。

一、代码

```
# Tensorflow 数字识别

import sensor, image, time, os, tf, uos, gc

net = None
labels = None

# 初始化函数
def num_init_setup():
    global sensor, net, labels, clock      # 设置为全局变量
    sensor.reset()                         # 初始化感光元件
    sensor.set_pixformat(sensor.GRAYSCALE) # 设置图像格式为灰度
    sensor.set_framesize(sensor.QVGA)      # 设置图像大小为 QVGA (320x240)
    sensor.set_windowing((240, 240))       # 设置窗口大小为 240x240
    sensor.skip_frames(10)                 # 跳过一些帧，使以上设置生效

    try:
        # 加载模型文件
        net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free() - (64*1024)
))
    except Exception as e:
        print(e)
        raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt file onto
the mass-storage device? (' + str(e) + ')')

```

```
    try:
        labels = [line.rstrip('\n') for line in open("labels.txt")]
    except Exception as e:
        raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file onto the mass-storage device? (' + str(e) + ')')

    clock = time.clock()        # 创建时钟对象

# 主函数
def num_main():
    while 1:
        clock.tick()                # 更新 FPS 帧率时钟
        img = sensor.snapshot()     # 拍一张照片并返回图像

        # default settings just do one detection... change them to search the image...
        for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
            #print("**********\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
            img.draw_rectangle(obj.rect())
            # 将分类和对应的相似度以列表套元组形式返回
            predictions_list = list(zip(labels, obj.output()))
            # 打印分类和与图像中检测到的物体的对应的相似度
            #for i in range(len(predictions_list)):
                #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
            if max(obj.output()) > 0.7:   # 当识别到的最大的相似度大于 0.7 时才认为是识别到了数字
                # 将识别到的数字赋值给 num
                num = labels[obj.output().index(max(obj.output()))]
                # 打印识别到的数字
                print('识别到的数字是 %s' % num)

        #print(clock.fps(), "fps")  # 打印帧率

# 程序入口
if __name__ == '__main__':
    num_init_setup()  # 执行初始化函数
    try:
        num_main()    # 执行主函数
    except:
        pass
```

二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中，然后在 openmv ide 中打开 num_detection.py 运行使用即可，可以使用提供的测试图片来进行测试。

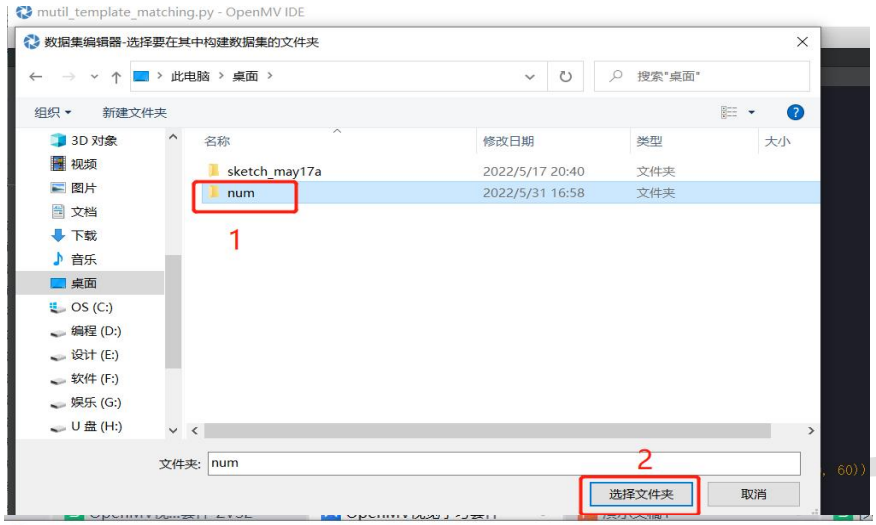
三、运行效果

代码运行后，openmv 会将识别到的数字在串行终端窗口打印出来。

四、知识点

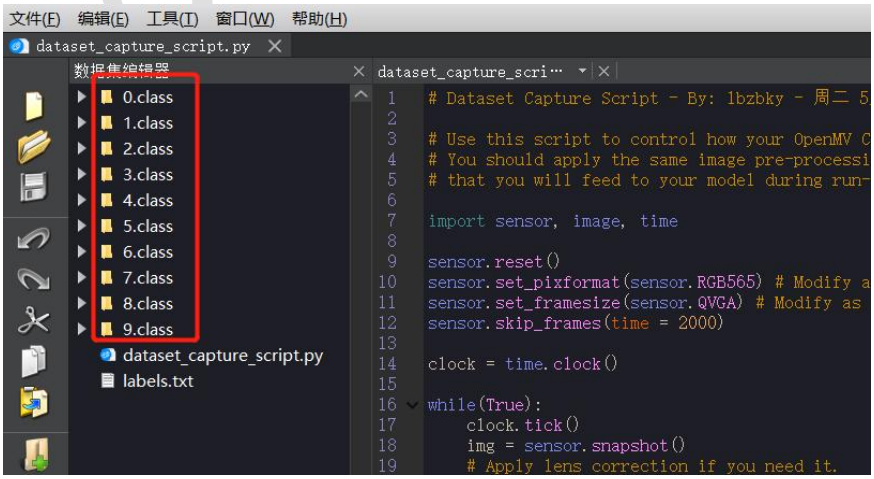
训练神经网络模型的方法：

1.打开 openmv ide，点击'工具'-->'数据集编辑器'，选择'新数据集'，然后点击鼠标右键，新建一个文件夹命名为 num，然后点击'选择文件夹'

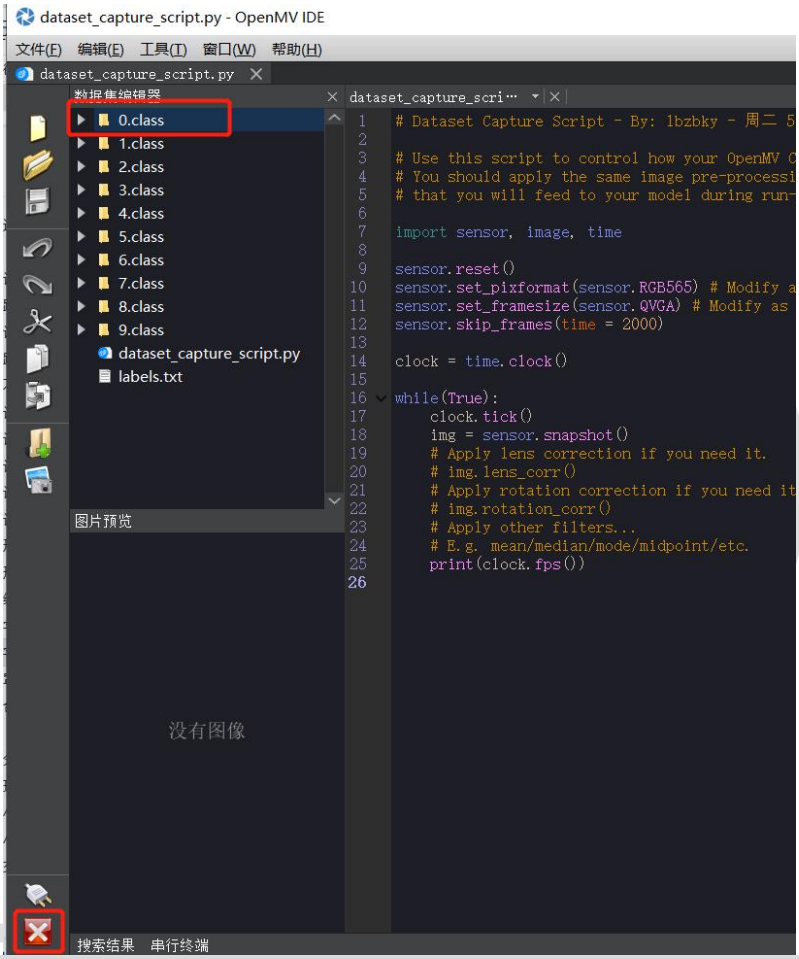


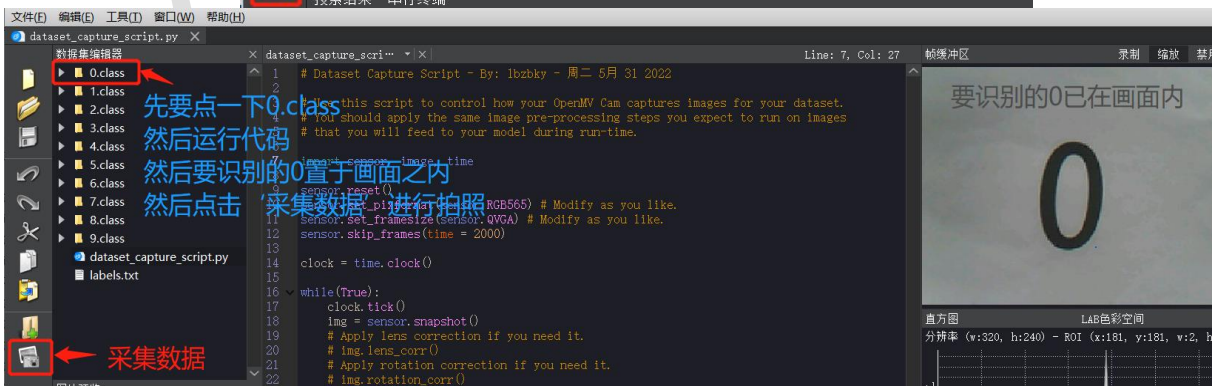


2.在出现的界面中，点击'新类文件夹'图标来创建子文件夹，然后为其命名（以分类命名，比如要识别的是 0，就以 0 命名），共创建 0 到 9 共 10 个子文件夹。都创建完成之后如下所示：



3.点击运行按钮开始运行程序，然后点一下 0.class，然后把要识别的 0 的图片置于 openmv 摄像头画面之内，点击'采集数据'按钮来拍照，可以将数字微调角度等然后拍照，拍 200 张左右即可。

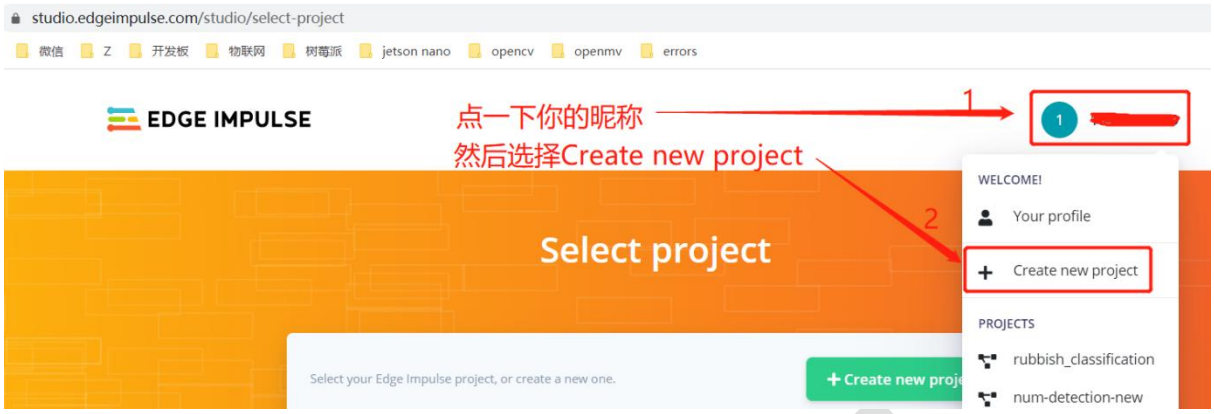




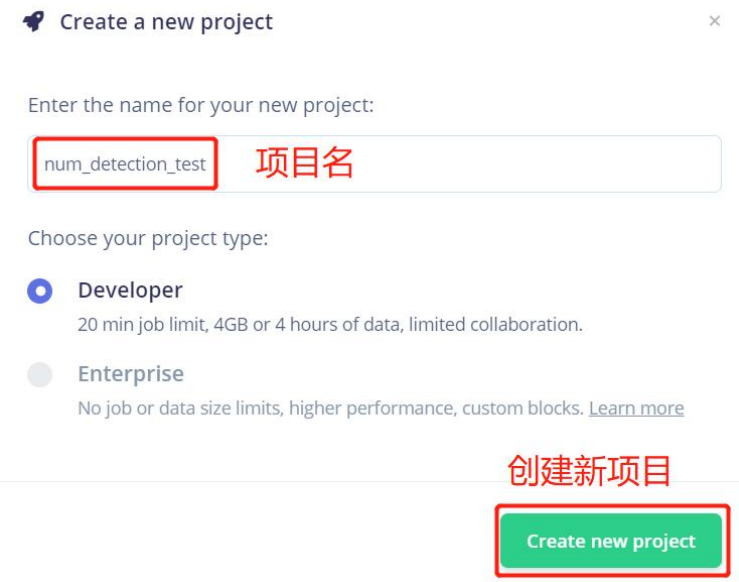
4.将数字 0 拍完之后，再点一下 1.class，然后将数字 1 置于 openmv 摄像头画面内，点击'采集数据'进行拍照，同样拍 200 张左右。其他数字同理，拍哪个数字时，就点一下对应的 class 文件夹，然后再将对应数字置于摄像头画面内，点击'采集数据'按钮进行拍照，拍 200 张左右。

5.都拍完之后，可以点击停止按钮使代码停止运行。

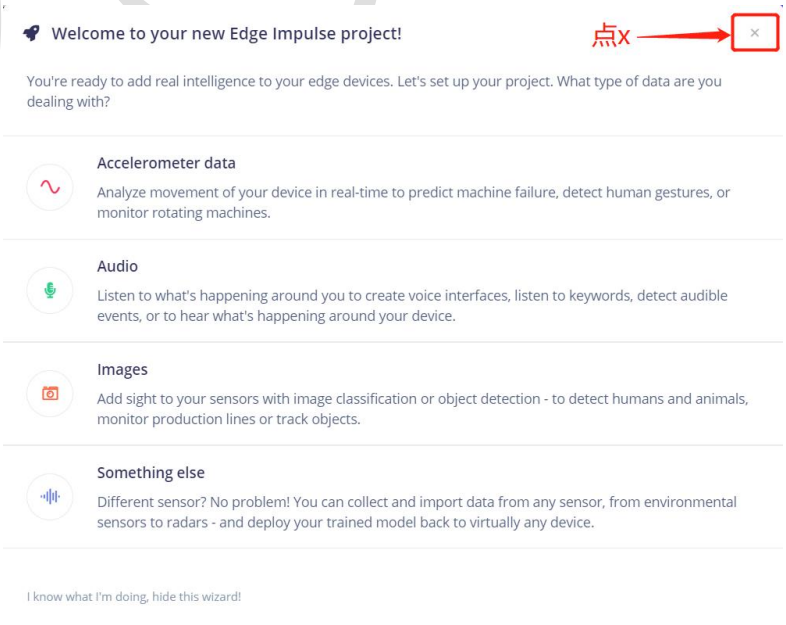
6.打开浏览器，输入网址 https://edgeimpulse.com/，注册并登录进去。



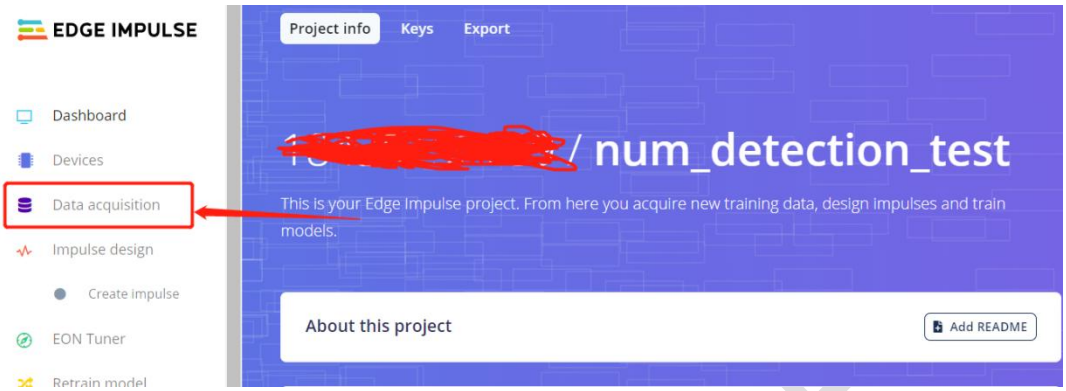
7.在弹出的界面中为项目起一个名字，然后点击'Create new project'



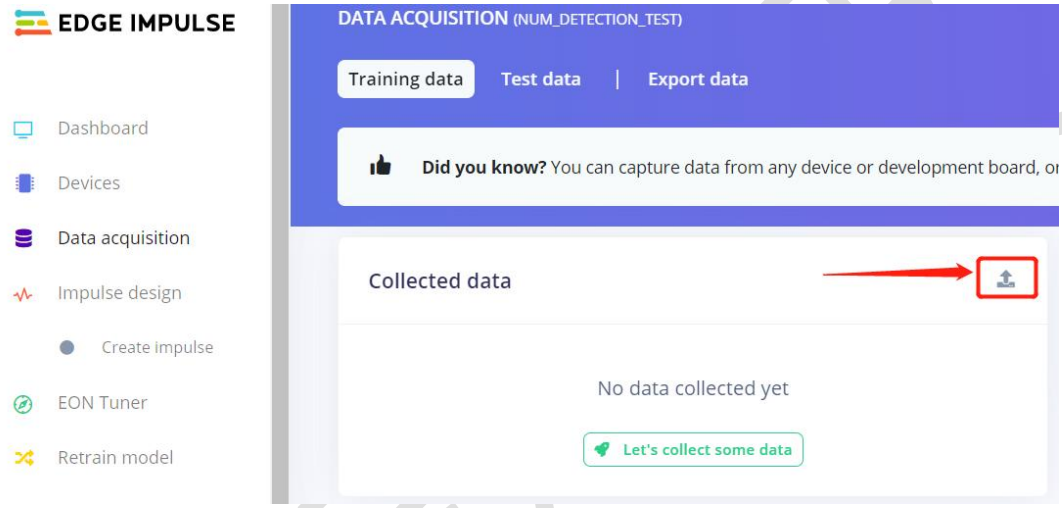
8.将弹出的如下窗口×掉。



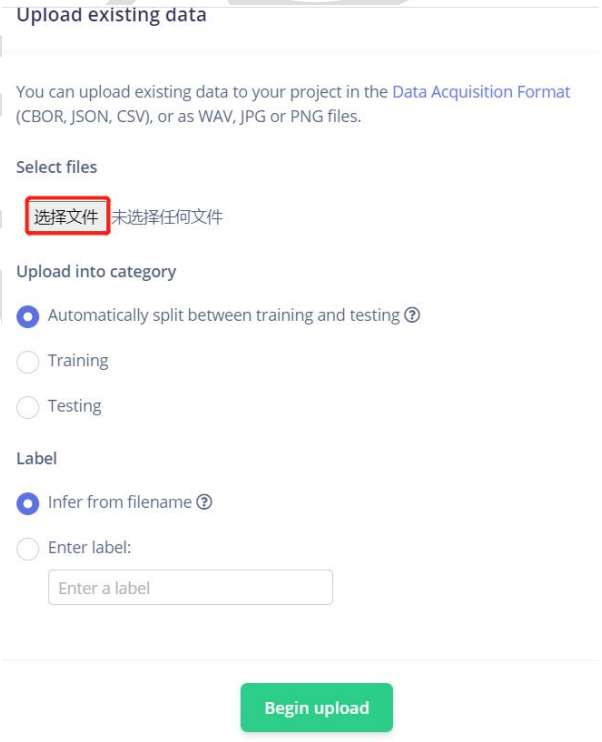
9.点击 Data acquisition



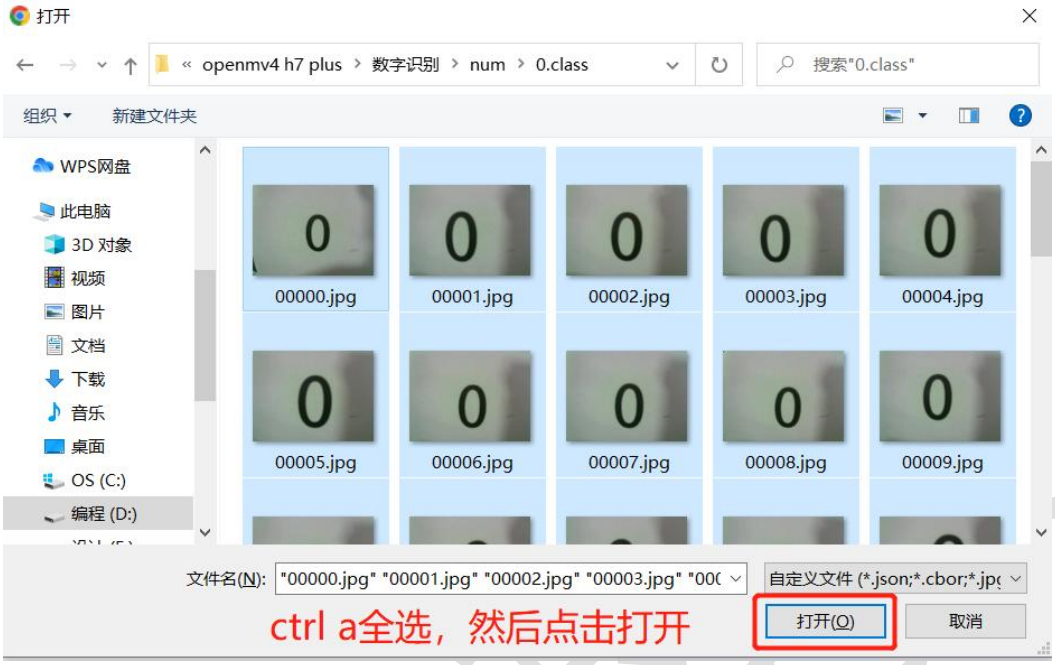
10.在弹出的界面中点击上传按钮



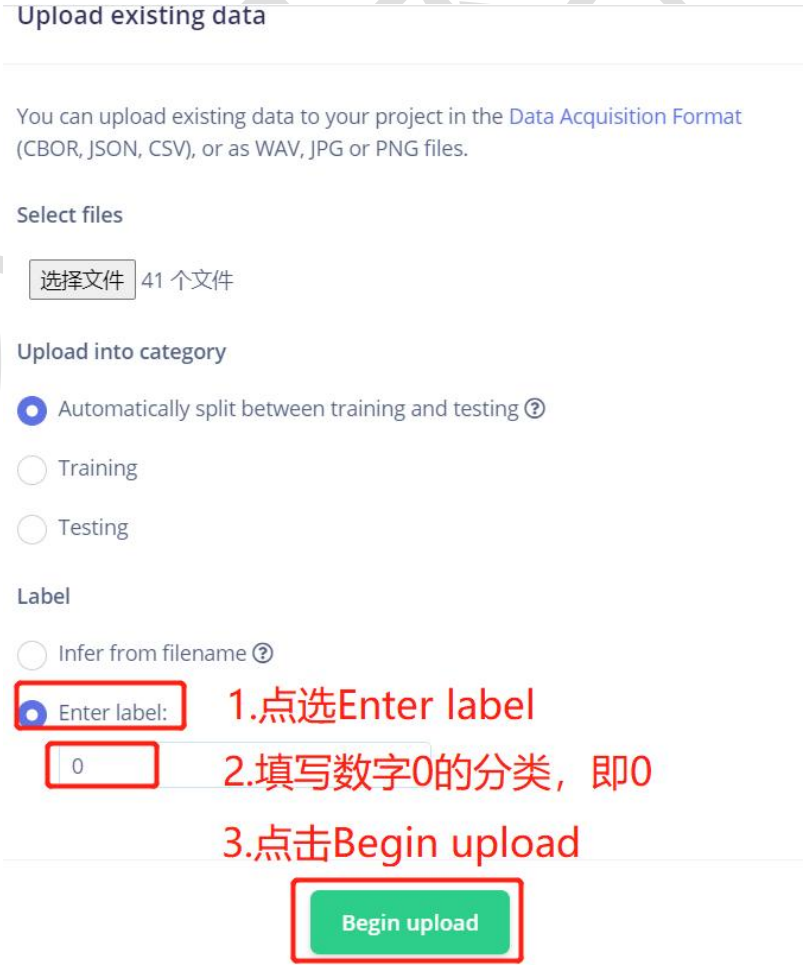
11.点击'选择文件'



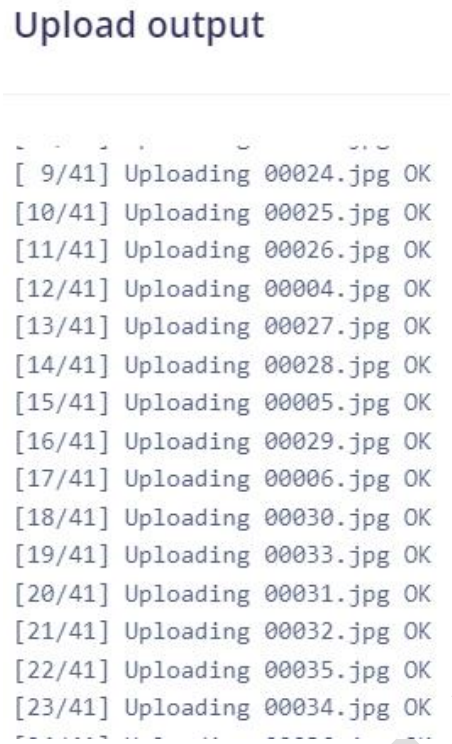
12.选择拍好的那些 0.class 文件夹中的数字 0 的图片，可以按键盘快捷键 ctrl a 全选，然后点击'打开'



13.点选 Enter label，填写数字 0 的分类为 0，然后点击 Begin upload



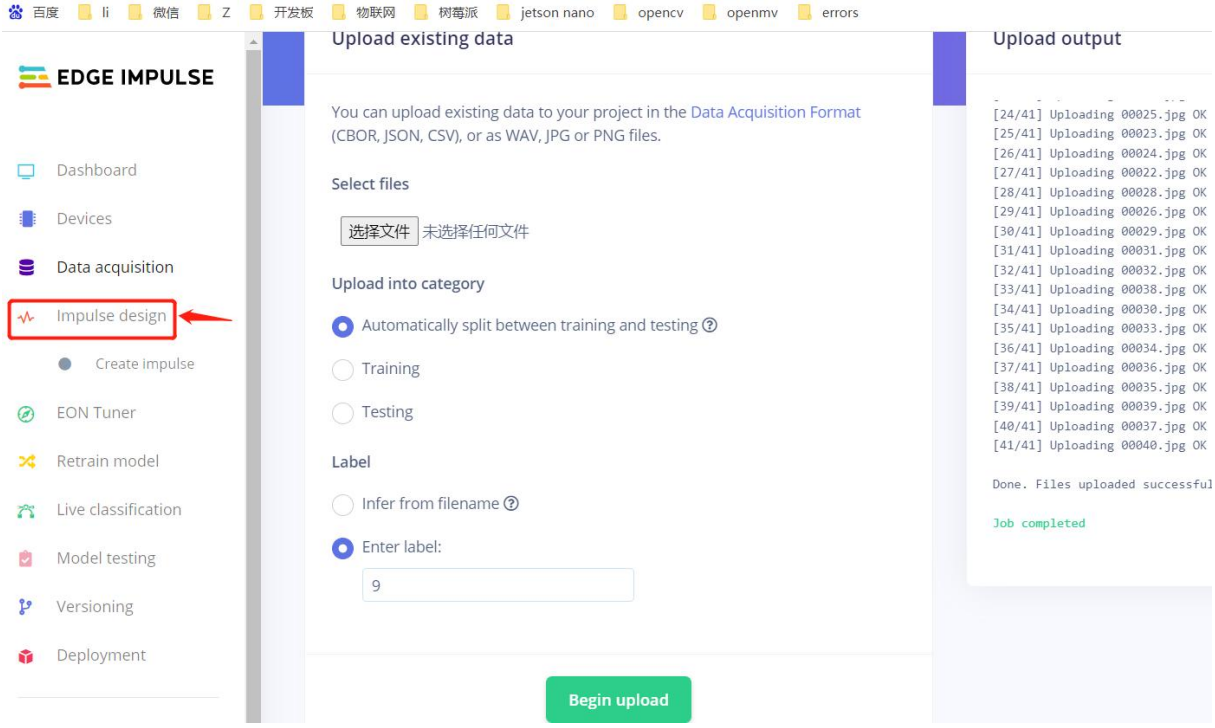
14.然后等待右侧上传，如下左图为上传中，右图为上传完成（会提示 Job completed）。



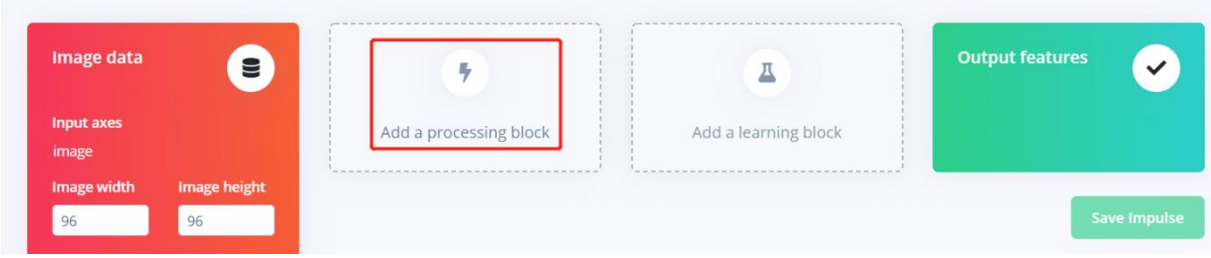


15.重复第 11 到第 14 步，只不过此次上传 1.class 中的数字 1 的图片，分类处填写 1，然后上传。再次重复 11-14 步将分类 2 上传，以此类推，直到将 0-9 都上传完成。

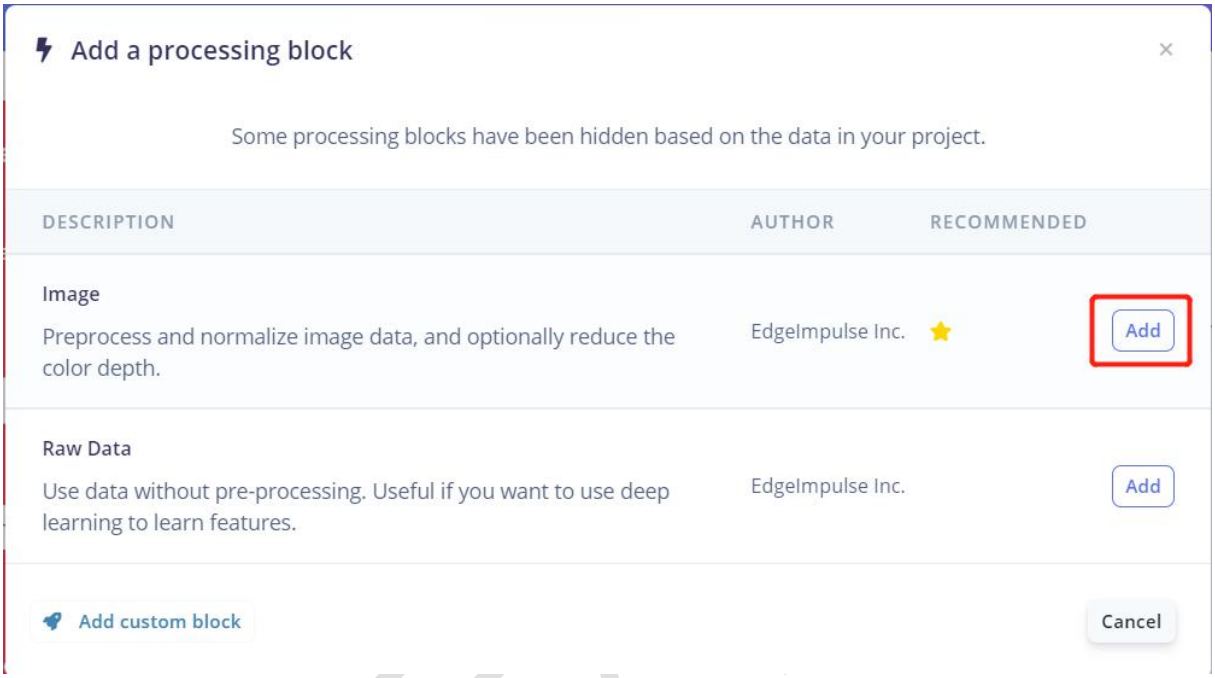
16.点击 Impulse design



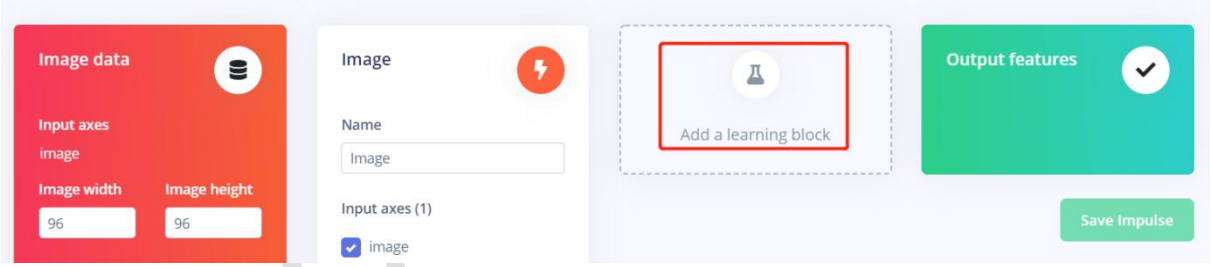
17.在弹出的界面中，点击 Add a processing block



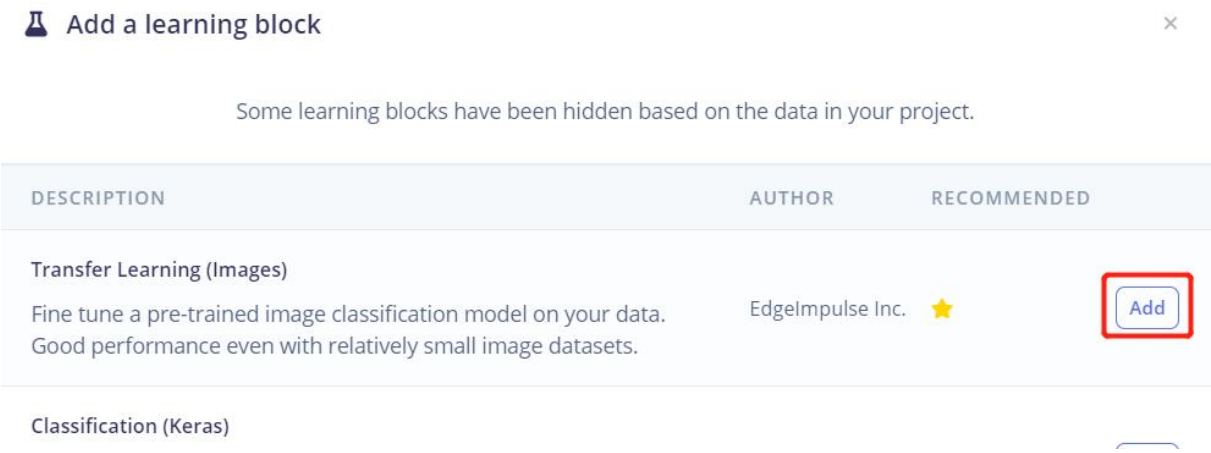
18.点击 Image 后面对应的那个 Add 按钮



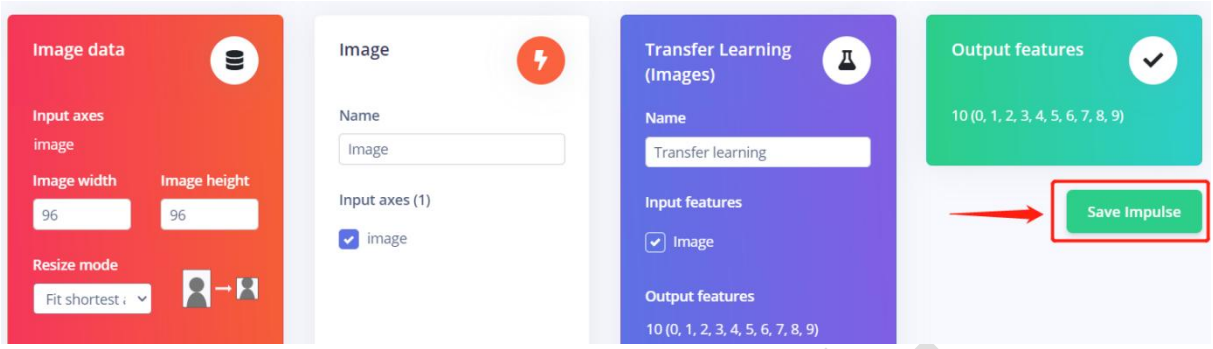
19.点击 Add a learning block

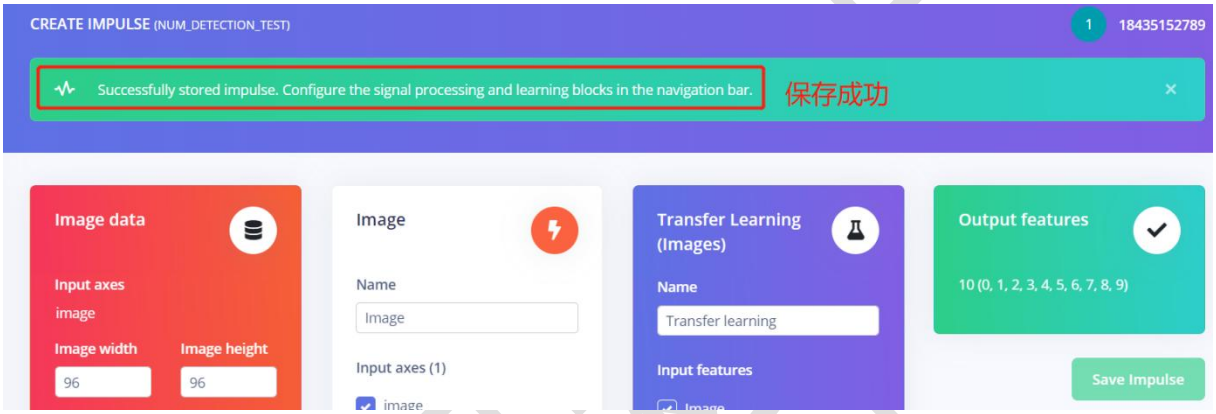


20.点击 Transfer Learning 后面的那个 Add 按钮

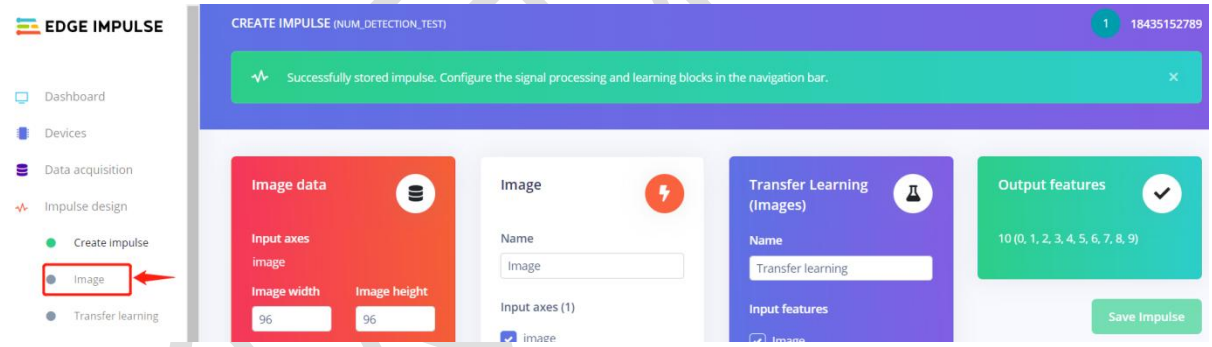


21.点击 Save Impulse，然后等待出现 Successfully...则表示保存成功。

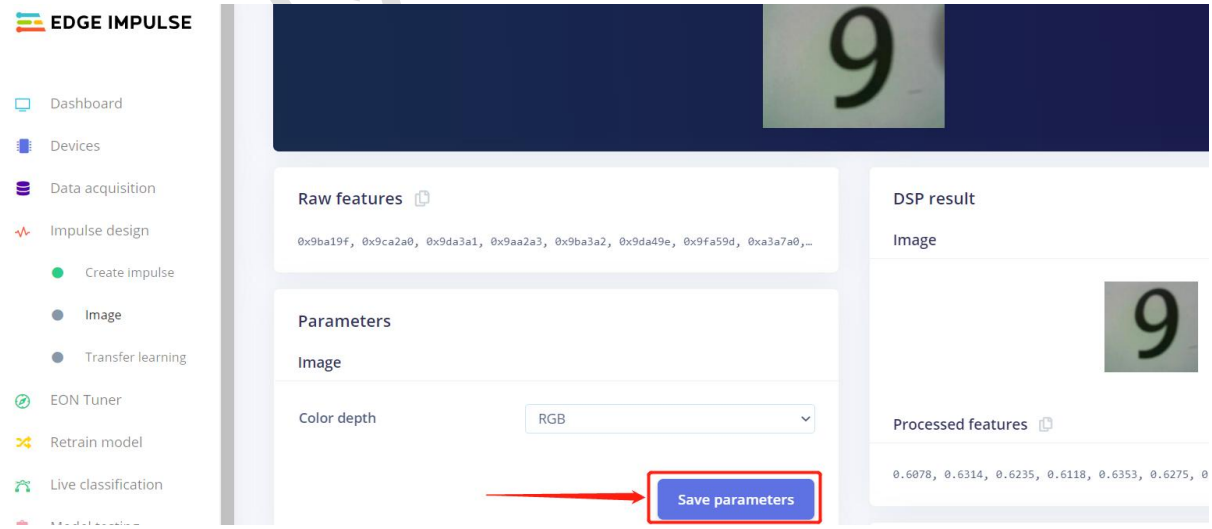




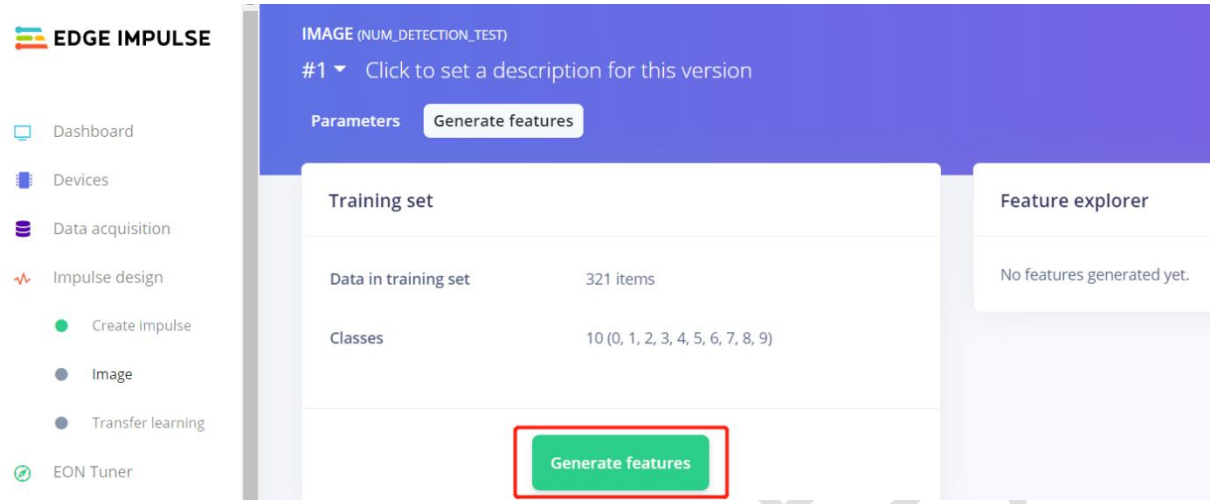
22.点击 Image

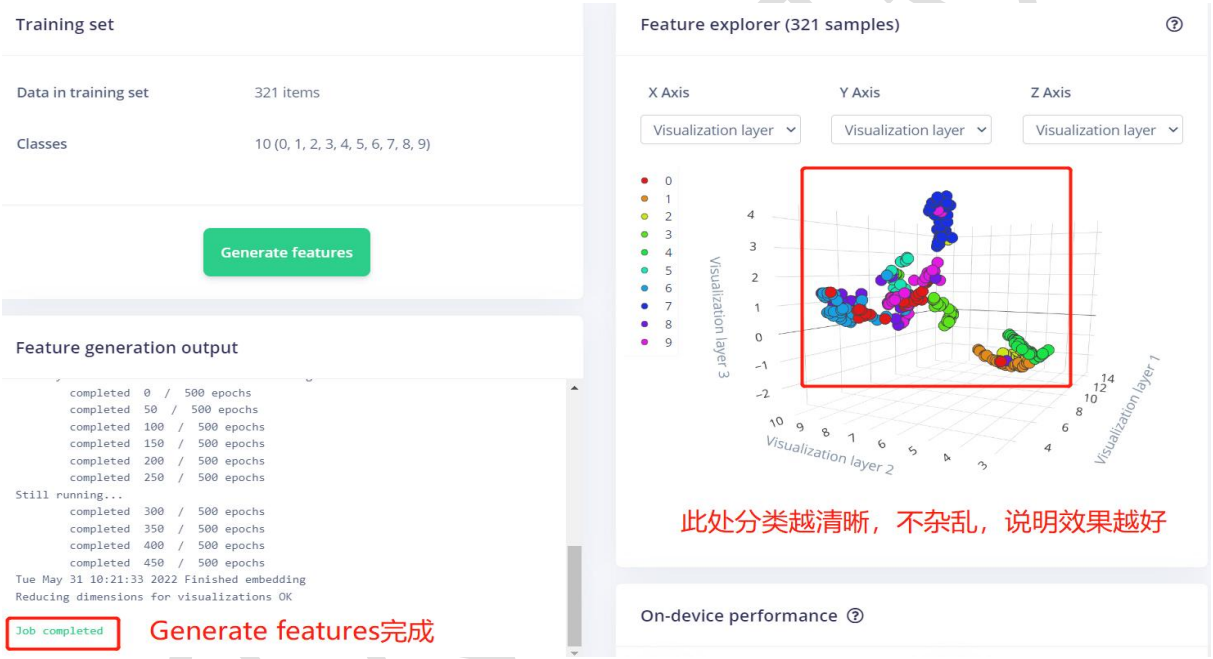


23.点击 Save parameters

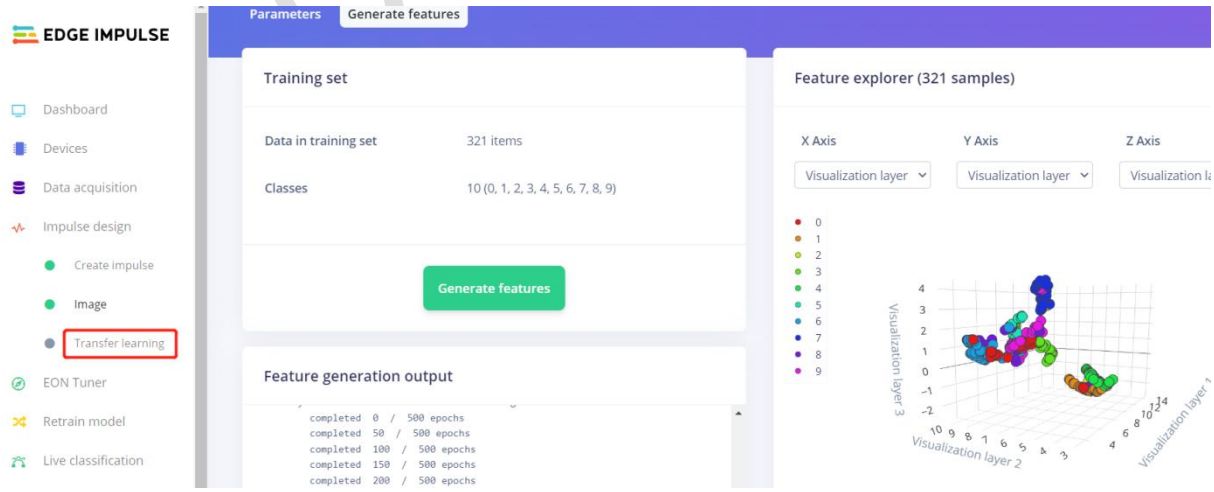


24.点击 Generate features，然后等待 Generate features 完成(会提示 Job completed)。

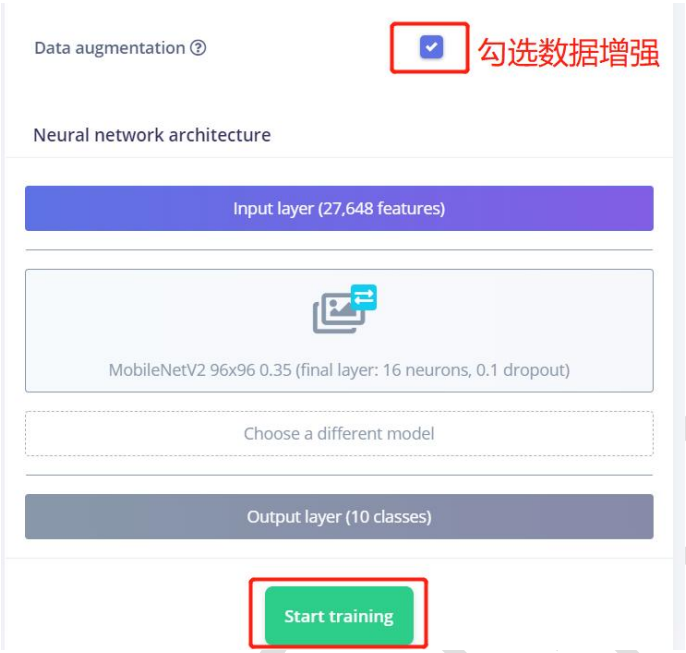




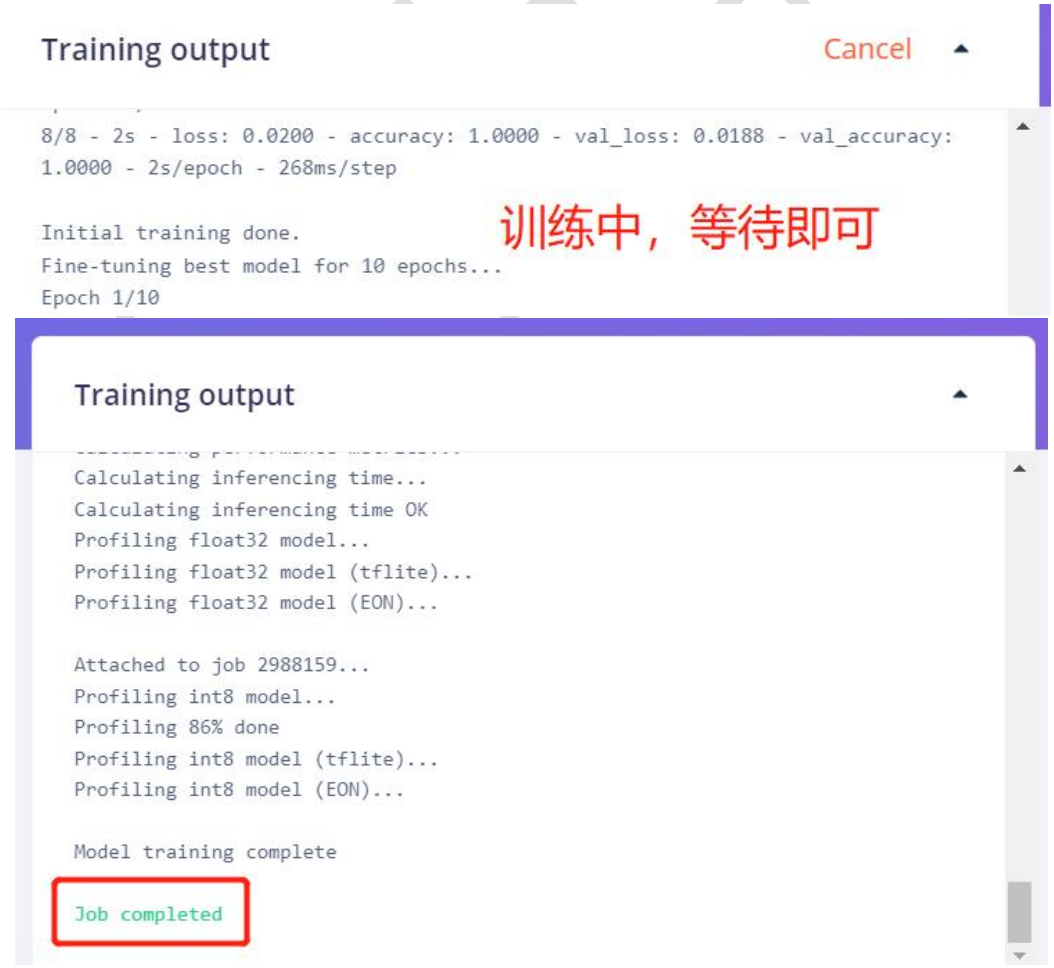
25.点击 Transfer learning



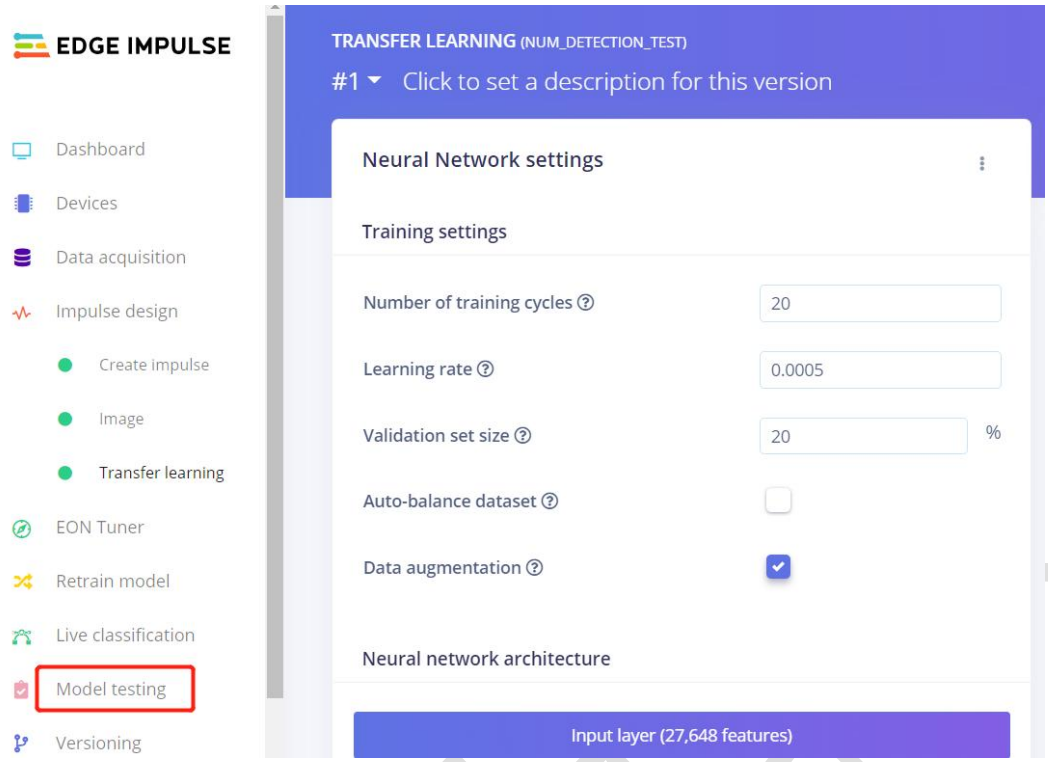
26.如图勾选数据增强，然后点击 Start training 即可开始训练模型。



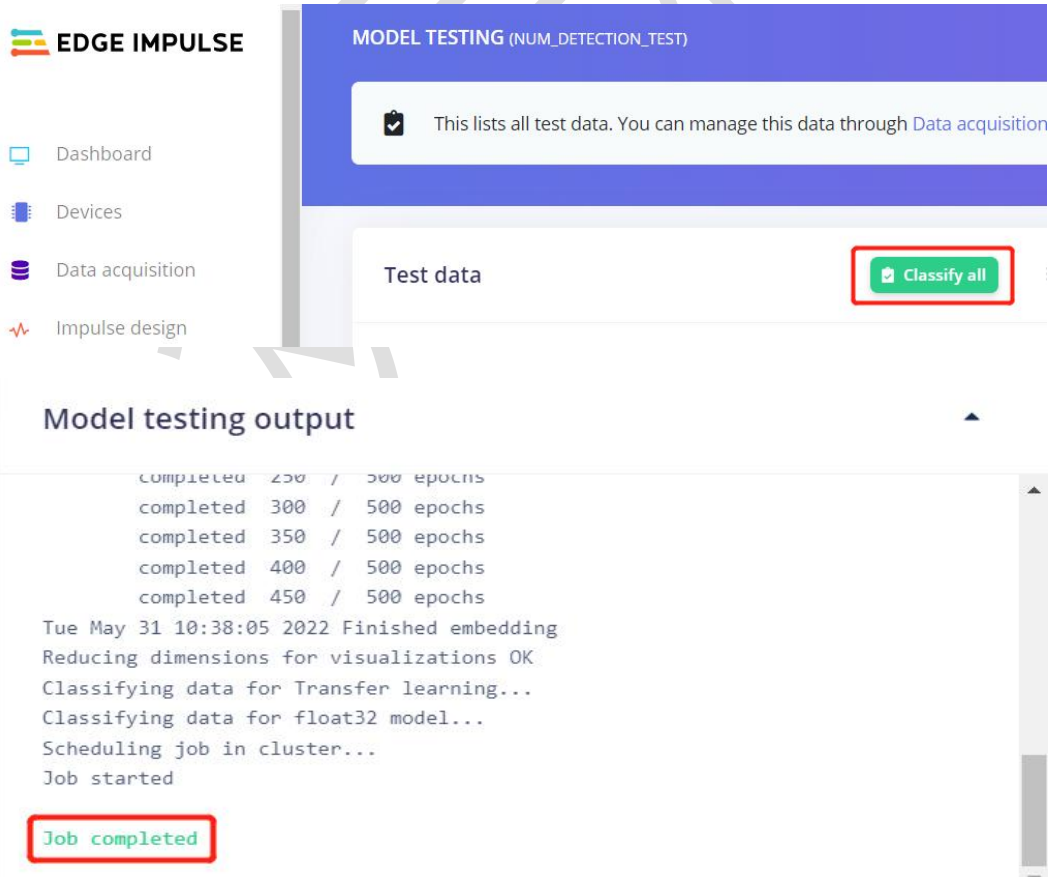
27.然后等待训练完成即可。如下第一张图为训练中，第二张图为训练完成。此网站训练模型是比较智能的，当它训练到效果非常好时，会自动结束，然后完成训练。



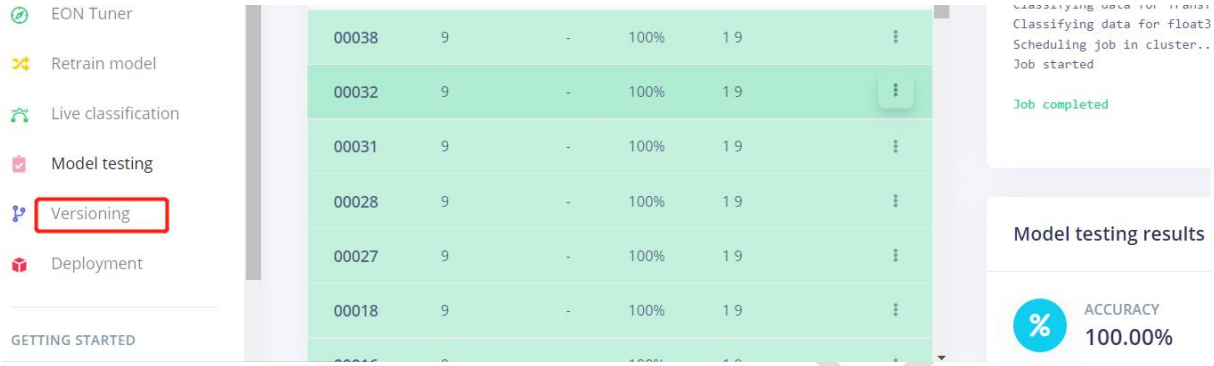
28.训练完成后，点击 Model testing 来测试模型。



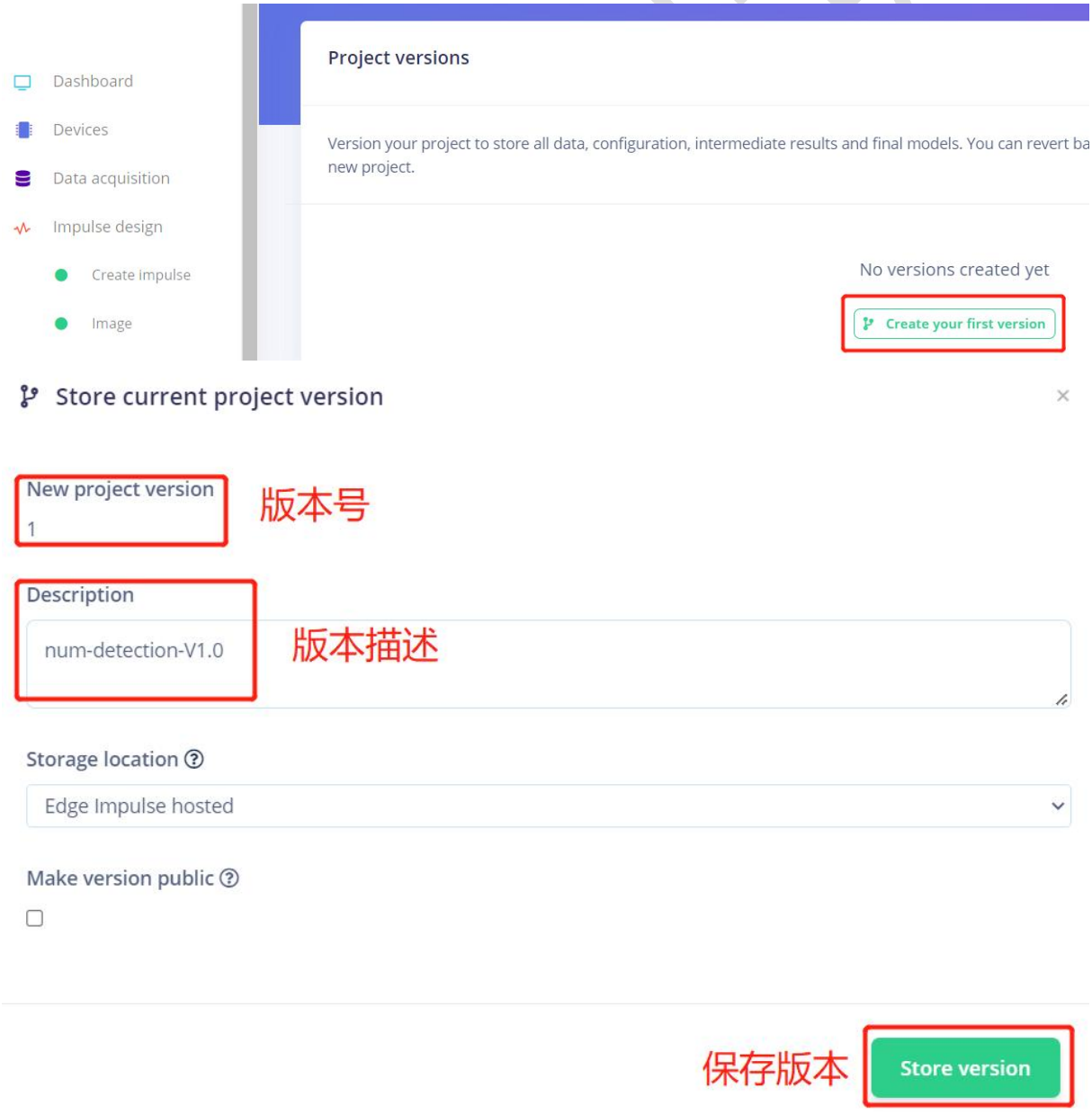
29.点击 Classify all，然后等待验证完成即可。验证完成如下第二张图所示（会提示 Job completed），并且下面会显示测试结果准确率。



30.点击 Versioning 为你的模型创建版本，也可跳过。

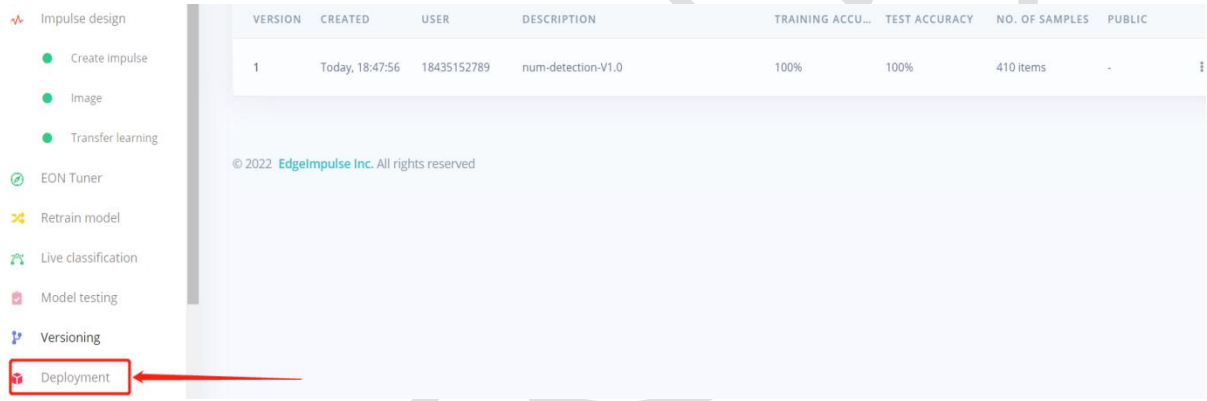


31.点击 Create your first version，然后在弹出的界面中填写版本描述，然后点击 Store version，然后等待一会儿，版本保存成功（会提示 Job completed）

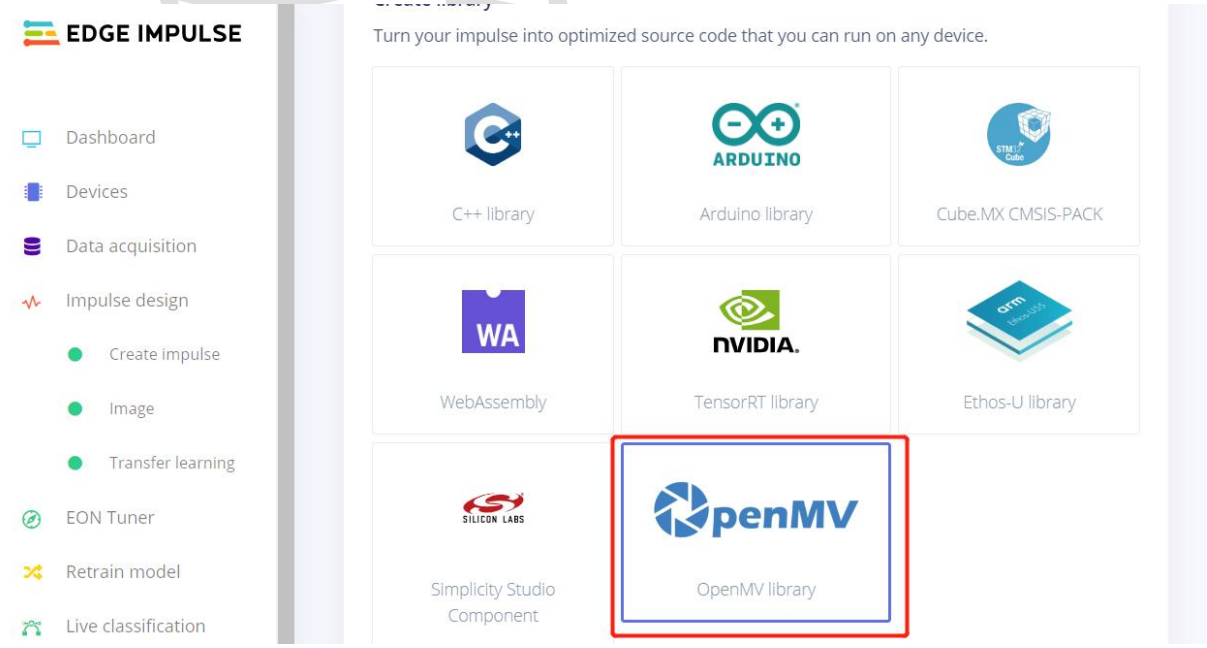




32.在版本保存完成后，点 x 号，然后点击左侧的 Deployment

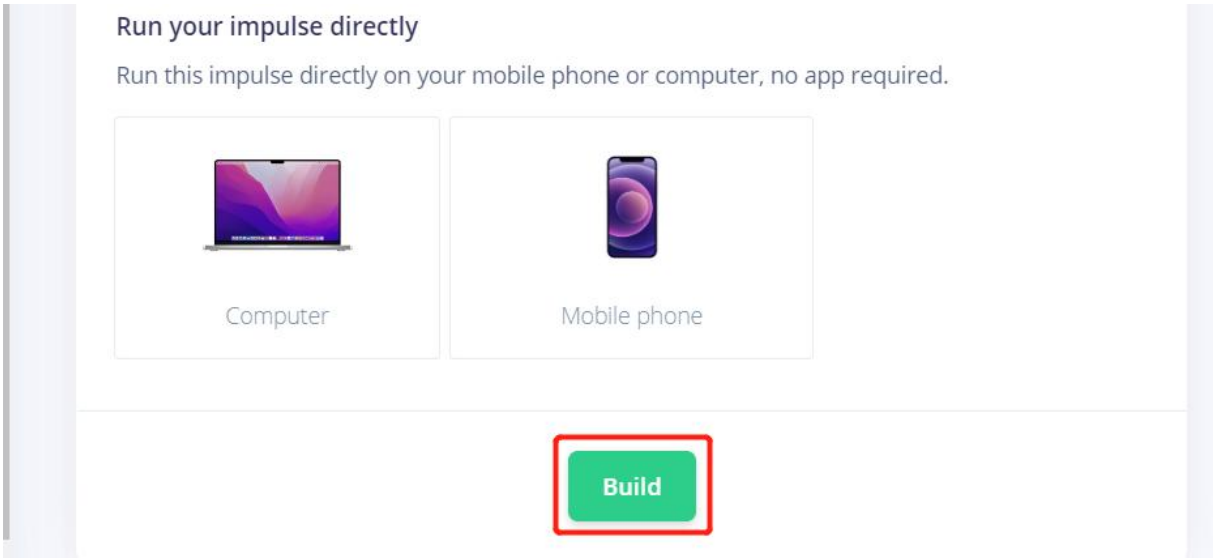


33.点击 OpenMV library

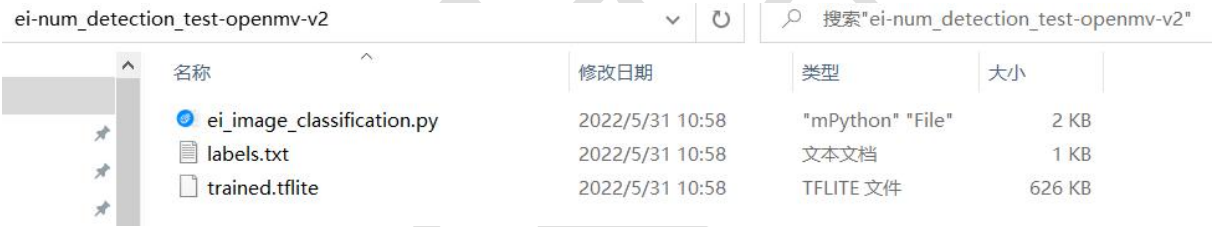


34.然后点击 Build，等待几秒钟，它会让你选择下载位置，我们选一个下载位置进行下

载即可。



35.将下载的压缩包解压，然后可以看到共有 3 个文件，其中：trained.tflite 为训练好的神经网络模型文件，labels.txt 是配套的分类文件，ei_image_classification.py 为配套代码，模型和分类文件最好不要改名（在代码中会用到，不宜改名，如果改名的话，在代码中也要做相应修改），代码可以改名



注意：你训练好的这个模型文件 trained.tflite 和分类文件 labels.txt 是配套我们提供的那个代码文件 num_detection.py 的，可以将你训练的这个模型文件 trained.tflite 和分类文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行 num_detection.py 来使用的。

【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现数字识别只适用于 openmv h7 plus。

2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 num_detection.py 来使用即可，可以用提供的配套的测试图片来进行测试。

3. 你也可以自己训练想要识别的一些数字的神经网络模型来使用。

第 14 课 口罩识别（只适用于 openmv4 h7 plus)

【本课目标】：

本课学习通过训练神经网络模型实现识别口罩的功能

【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现口罩识别只适用于 openmv h7 plus。

2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 mask_detection.py 来使用即可。

3. 你也可以自己训练神经网络模型来使用，具体训练方法在上一节数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称是 face 和 mask，然后在网址中上传文件时的分类名需要对应为 face 和 mask，其他步骤都一样。

一、代码

```
# Tensorflow 口罩识别

import sensor, image, time, os, tf, uos, gc

net = None
labels = None

# 初始化函数
def mask_init_setup():
    global sensor, net, labels, clock      # 设置为全局变量
    sensor.reset()                         # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)    # 设置图像格式为彩色
    sensor.set_framesize(sensor.QVGA)      # 设置图像大小为 QVGA (320x240)
    sensor.set_windowing((240, 240))       # 设置 window 窗口大小为 240x240
    sensor.skip_frames(10)                 # 跳过一些帧，使以上设置生效

    try:
        # 加载模型文件
        net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free() - (64*1024)
))
    except Exception as e:
        print(e)
        raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt file onto
the mass-storage device? (' + str(e) + ')')

    try:
```

```
        labels = [line.rstrip('\n') for line in open("labels.txt")]
    except Exception as e:
        raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file onto the mass-storage device? (' + str(e) + ')')

    clock = time.clock()                    # 创建时钟对象

# 主函数
def mask_main():
    while 1:
        clock.tick()                # 更新 FPS 帧率时钟
        img = sensor.snapshot()     # 拍一张照片并返回图像

        # default settings just do one detection... change them to search the image...
        for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
            #print("**********\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
            img.draw_rectangle(obj.rect())
            # 将分类和对应的相似度以列表套元组形式返回
            predictions_list = list(zip(labels, obj.output()))
            # 打印分类和与图像中检测到的物体的对应的相似度
            #for i in range(len(predictions_list)):
                #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
            if max(obj.output()) > 0.9:   # 当识别到的最大的相似度大于 0.9 时才认为是识别到了
                # 将识别到的类别赋值给 result
                result = labels[obj.output().index(max(obj.output()))]
                if result == 'mask':
                    # 打印识别到的类别
                    print('检测到已戴好口罩')
                elif result == 'face':
                    print('检测到未佩戴口罩，请佩戴好口罩')

        #print(clock.fps(), "fps")  # 打印帧率

# 程序入口
if __name__ == '__main__':
    mask_init_setup()  # 执行初始化函数
    try:
        mask_main()    # 执行主函数
    except:
        pass
```

二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中，然后在 openmv ide 中打开 mask_detection.py 运行使用即可。

三、运行效果

代码运行后，openmv 会将识别到的结果在串行终端窗口打印出来。

四、知识点

训练神经网络模型的方法在上一节数字识别课程中已经讲过，此处不再赘述。

关于 tensorflow 神经网络模型的使用可以参考官方文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.tf.html#

【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现口罩识别只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 mask_detection.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在上一节数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称是 face 和 mask，然后在网址中上传文件时的分类名需要对应为 face 和 mask，其他步骤都一样。

第 15 课 综合例程

【本课目标】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动、语音识别模块

【本课内容】：

本综合例程包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车、前进、后退、左转、右转、停止、复位。

注意颜色相关的需要根据实际情况调节阈值范围并将代码中的阈值范围替换掉，分辨人脸需提前收集好人脸图片放在对应文件夹中并将代码按照前面的教程进行一些修改，具体使用方法在下面会看到。

一、代码

```
# 综合例程
# 包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车
# 注意颜色相关的需要根据实际情况调节阈值范围，分辨人脸需提前收集好人脸图片
import sensor, image, time, pyb
from AI_Functions import (
    multi_color_detection, face_detection, color_tracking, face_tracking,
    face_recognition, qrcode_detection, mutil_template_num_matching,
    car_follow_color, car_follow_line
)
from ZL_SDK import (zl_uart3, zl_led, zl_car_run, zl_pan_tilt, pid)

# 串口接收数据后对应执行的命令值
uart_recv_data_command = None

# 1.对应每个功能的一个标志位，标志位用来初始化一次感光元件 2.串口接收到的对应数据 3.数据对应指令
flag_dict = {
    'multi_color_detection': {'flag': True, 'uart_recv_data': b'$YSSB!', 'command': 'YSSB'},
    'face_detection': {'flag': True, 'uart_recv_data': b'$RLSB!', 'command': 'RLSB'},
    'color_tracking': {'flag': True, 'uart_recv_data': b'$YSGS!', 'command': 'YSGS'},
    'face_tracking': {'flag': True, 'uart_recv_data': b'$RLGS!', 'command': 'RLGS'},
    'face_recognition': {'flag': True, 'uart_recv_data': b'$FBRL!', 'command': 'FBRL'},
    'qrcode_detection': {'flag': True, 'uart_recv_data': b'$EWMSB!', 'command': 'EWMSB'},
    'mutil_template_num_matching': {'flag': True, 'uart_recv_data': b'$SZSB!', 'command': 'SZSB'},
    'car_follow_color': {'flag': True, 'uart_recv_data': b'$ZXQ!', 'command': 'ZXQ'},
    'car_follow_line': {'flag': True, 'uart_recv_data': b'$XJMS!', 'command': 'XJMS'},
}
# 串口接收到对应指令控制小车运动及云台运动
motor_pan_tilt_command_dict = {
    '前进': b'$QJ!',
    '后退': b'$HT!',
```

```
    '左转': b'$ZZ!',
    '右转': b'$YZ!',
    '停止': b'$TZ!',
    '复位': b'$RESET!',
}

# 标志位处理函数，标志位用来初始化一次感光元件，并让其他功能对应的 flag 变为真
def flag_func(flag):
    for i in flag_dict:
        if i == flag:
            flag_dict[i]['flag'] = False
        else:
            flag_dict[i]['flag'] = True
    print(flag_dict)

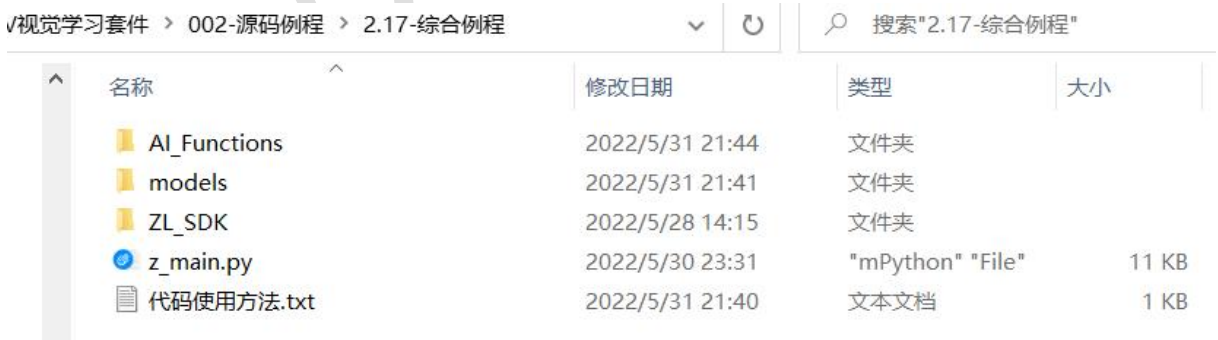
# 串口接收数据的处理
def uart_recv_data_handle():
    global uart_recv_data_command
    uart_recv_data = zl_uart3.main()
    #print(uart_recv_data, type(uart_recv_data))
    if uart_recv_data:
        if uart_recv_data == flag_dict['multi_color_detection']['uart_recv_data']:          # 颜色识别
            uart_recv_data_command = flag_dict['multi_color_detection']['command']
        elif uart_recv_data == flag_dict['face_detection']['uart_recv_data']:               # 人脸识别
            uart_recv_data_command = flag_dict['face_detection']['command']
        elif uart_recv_data == flag_dict['color_tracking']['uart_recv_data']:               # 颜色跟随
            uart_recv_data_command = flag_dict['color_tracking']['command']
        elif uart_recv_data == flag_dict['face_tracking']['uart_recv_data']:                # 人脸跟随
            uart_recv_data_command = flag_dict['face_tracking']['command']
        elif uart_recv_data == flag_dict['face_recognition']['uart_recv_data']:             # 分辨人脸
            uart_recv_data_command = flag_dict['face_recognition']['command']
        elif uart_recv_data == flag_dict['qrcode_detection']['uart_recv_data']:             # 二维码识别
            uart_recv_data_command = flag_dict['qrcode_detection']['command']
        elif uart_recv_data == flag_dict['mutil_template_num_matching']['uart_recv_data']: # 数字识别
            uart_recv_data_command = flag_dict['mutil_template_num_matching']['command']
        elif uart_recv_data == flag_dict['car_follow_color']['uart_recv_data']:             # 追小球
            uart_recv_data_command = flag_dict['car_follow_color']['command']
        elif uart_recv_data == flag_dict['car_follow_line']['uart_recv_data']:              # 循迹模式
            uart_recv_data_command = flag_dict['car_follow_line']['command']
        elif uart_recv_data == motor_pan_tilt_command_dict['前进']:                         # 前进
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'QJ'
        elif uart_recv_data == motor_pan_tilt_command_dict['后退']:                         # 后退
            zl_car_run.car_run(zl_car_run.backward_speed, zl_car_run.backward_speed)
            uart_recv_data_command = 'HT'
        elif uart_recv_data == motor_pan_tilt_command_dict['左转']:                         # 左转
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'ZZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['右转']:                         # 右转
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'YZ'
```

```
82.         elif uart_recv_data == motor_pan_tilt_command_dict['停止']:                    # 停止
83.             zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)
84.             uart_recv_data_command = 'TZ'
85.         elif uart_recv_data == motor_pan_tilt_command_dict['复位']:                    # 复位
86.             zl_pan_tilt.middle()
87.             uart_recv_data_command = 'FW'
88.         #print(uart_recv_data_command)
89.
90.
91. # 功能执行函数
92. def function_run():
93.     if uart_recv_data_command == flag_dict['multi_color_detection']['command']:  # 颜色识别功能
94.         if flag_dict['multi_color_detection']['flag']:                           # 只初始化一次
95.             multi_color_detection.init_setup()                                   # 颜色识别初始化
96.             flag_func('multi_color_detection')
97.         multi_color_detection.main()                                             # 执行颜色识别主函数
98.
99.     elif uart_recv_data_command == flag_dict['face_detection']['command']:       # 人脸识别功能
100.         if flag_dict['face_detection']['flag']:                                  # 只初始化一次
101.             face_detection.init_setup()                                          # 人脸识别初始化
102.             flag_func('face_detection')
103.         face_detection.main()                                                    # 执行人脸识别主函数
104.
105.     elif uart_recv_data_command == flag_dict['color_tracking']['command']:       # 颜色跟随功能
106.         if flag_dict['color_tracking']['flag']:                                  # 只初始化一次
107.             color_tracking.init_setup()                                          # 颜色跟随初始化
108.             flag_func('color_tracking')
109.         color_tracking.main()                                                    # 执行颜色跟随主函数
110.
111.     elif uart_recv_data_command == flag_dict['face_tracking']['command']:       # 人脸跟随功能
112.         if flag_dict['face_tracking']['flag']:                                   # 只初始化一次
113.             face_tracking.init_setup()                                           # 人脸跟随初始化
114.             flag_func('face_tracking')
115.         face_tracking.main()                                                     # 执行人脸跟随主函数
116.
117.     elif uart_recv_data_command == flag_dict['face_recognition']['command']:     # 分辨人脸功能
118.         if flag_dict['face_recognition']['flag']:                                # 只初始化一次
119.             face_recognition.init_setup()                                        # 分辨人脸初始化
120.             flag_func('face_recognition')
121.         face_recognition.main()                                                  # 执行分辨人脸主函数
122.
123.     elif uart_recv_data_command == flag_dict['qrcode_detection']['command']:     # 二维码识别功能
124.         if flag_dict['qrcode_detection']['flag']:                                # 只初始化一次
125.             qrcode_detection.init_setup()                                        # 二维码识别初始化
126.             flag_func('qrcode_detection')
127.         qrcode_detection.main()                                                  # 执行二维码主函数
128.
129.     elif uart_recv_data_command == flag_dict['mutil_template_num_matching']['command']:  # 数字识别功能
130.         if flag_dict['mutil_template_num_matching']['flag']:                     # 只初始化一次
131.             mutil_template_num_matching.init_setup()                             # 数字识别初始化
132.             flag_func('mutil_template_num_matching')
```

```
133.         mutil_template_num_matching.main()                                  # 执行数字识别主函数
134.
135.     elif uart_recv_data_command == flag_dict['car_follow_color']['command']:    # 追小球功能
136.         if flag_dict['car_follow_color']['flag']:                              # 只初始化一次
137.             car_follow_color.init_setup()                                      # 追小球功能初始化
138.             flag_func('car_follow_color')
139.         car_follow_color.main()                                                # 执行追小球主函数
140.
141.     elif uart_recv_data_command == flag_dict['car_follow_line']['command']:     # 循迹功能
142.         if flag_dict['car_follow_line']['flag']:                               # 只初始化一次
143.             car_follow_line.init_setup()                                       # 循迹模式初始化
144.             flag_func('car_follow_line')
145.         car_follow_line.main()                                                 # 执行循迹主函数
146.
147. # 主函数
148. def main():
149.     uart_recv_data_handle()  # 串口接收数据并处理
150.     function_run()           # 根据接收到的数据执行不同的功能
151.
152.
153. # 程序入口
154. if __name__ == '__main__':
155.     zl_led.init_setup()      # led 灯初始化
156.     zl_uart3.init_setup()    # 串口初始化
157.     for i in range(6):       # led 灯亮三下，代表初始化完成，可以开始执行各项功能了
158.         zl_led.main(0.3)     # 每次亮 0.3 秒
159.     zl_pan_tilt.middle()     # 云台在程序开始后复位
160.     try:                     # 异常处理
161.         while 1:             # 无限循环
162.             main()           # 执行主函数
163.     except:
164.         zl_pan_tilt.middle() # 云台复位
165.         zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

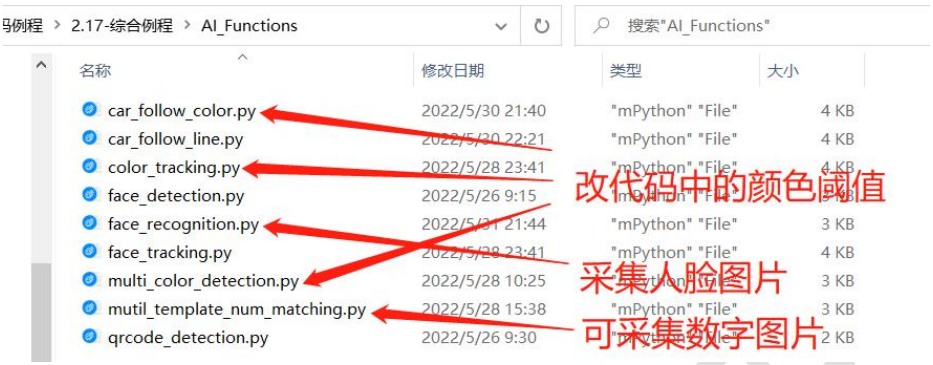
二、代码使用方法

1.在综合例程中共有如下几个文件和文件夹



2.在使用综合例程之前，需要将涉及到的颜色相关的代码，如颜色识别和颜色跟随代码中的颜色阈值按照前面的教程进行修改，分辨人脸的代码需要先采集人脸图片并存放到

models 文件夹中的 ZLTech 中的对应位置，数字识别可以使用我们提供的模板，也可自己根据前面的教程采集数字图片模板并存放到 models 文件夹中的 template_num 中。



3.将 AI_Functions、models、ZL_SDK 文件夹都保存到 openmv 的盘符中，然后在 openmv ide 中将 z_main.py 文件存到 openmv 中，脱机运行即可，也可在线运行，不过要注意不要让小车跑了。

三、运行效果

代码运行后，我们主要通过语音来控制小车。以下为对应的语音及功能：

颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别（注意：此处的数字识别使用的是多模板匹配，需将固件升回至 4.5.2 左右才可正常使用）、追小球、循迹模式、前进、后退、左转、右转、停止、复位

语音模块的基本用法：你说'小灵小灵'可以将它唤醒，它会随机回复你，如果你将它唤醒后，在 20 秒内未用语音控制它，它会再次进入睡眠状态，等待你将它唤醒；

注意我们的语音功能都是需要在将它唤醒之后才控制它的。有些功能是可以使用不同的话语来控制的。你将它唤醒后，说'前进'或'往前走'，则小车会往前走；说'循迹模式'，则小车会开始循黑线。

语音	对应功能
颜色识别	颜色识别
颜色跟随	颜色跟随
人脸识别	人脸识别
人脸跟随	人脸跟随
分辨人脸	分辨人脸
二维码识别	二维码识别
数字识别	数字识别
追小球	小车追小球
循迹模式	巡线小车
前进、往前走	小车前进
后退、向后走、倒着走	小车后退
左转、向左转	小车左转
右转、向右转	小车右转
停止、停下	小车停止
复位	云台复位

【本课小结】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序注意在运行程序之前根据教程做好准备工作，不然程序可能出错。

众灵科技

拓展篇

第 1 课 垃圾分类（只适用于 openmv4 h7 plus）

【课程目标】：

本课学习通过训练神经网络模型实现垃圾分类的功能

【课前准备】：

OpenMV4 H7 PLUS、USB 数据线

【本课内容】：

注意：

1. 使用训练神经网络模型的方法实现垃圾分类只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 rubbish_classification.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称需要是你要识别的垃圾的名称，然后在网址中上传文件时的分类名需要是你要识别的垃圾的名称，其他步骤都一样。

一、代码

```
# Tensorflow 垃圾分类

import sensor, image, time, os, tf, uos, gc

net = None
labels = None

# 初始化函数
def rubbish_init_setup():
    global sensor, net, labels, clock      # 设置为全局变量
    sensor.reset()                         # 初始化感光元件
    sensor.set_pixformat(sensor.GRAYSCALE) # 设置图像格式为灰度
    sensor.set_framesize(sensor.QVGA)      # 设置图像大小为 QVGA (320x240)
    sensor.set_windowing((240, 240))       # 设置窗口大小为 240x240
    sensor.skip_frames(10)                 # 跳过一些帧，使以上设置生效

    try:
        # 加载模型文件
        net = tf.load("trained.tflite", load_to_fb=uos.stat('trained.tflite')[6] > (gc.mem_free() - (64*1024)
))
    except Exception as e:
        print(e)
        raise Exception('Failed to load "trained.tflite", did you copy the .tflite and labels.txt file onto
the mass-storage device? (' + str(e) + ')')

    try:
        labels = [line.rstrip('\n') for line in open("labels.txt")]
    except Exception as e:
```

```
            raise Exception('Failed to load "labels.txt", did you copy the .tflite and labels.txt file onto the mass-storage device? (' + str(e) + ')')

    clock = time.clock()        # 创建时钟对象

# 主函数
def rubbish_main():
    while 1:
        clock.tick()                # 更新 FPS 帧率时钟
        img = sensor.snapshot()     # 拍一张照片并返回图像

        # default settings just do one detection... change them to search the image...
        for obj in net.classify(img, min_scale=1.0, scale_mul=0.8, x_overlap=0.5, y_overlap=0.5):
            #print("**********\nPredictions at [x=%d,y=%d,w=%d,h=%d]" % obj.rect())
            img.draw_rectangle(obj.rect())
            # 将分类和对应的相似度以列表套元组形式返回
            predictions_list = list(zip(labels, obj.output()))
            # 打印分类和与图像中检测到的物体的对应的相似度
            #for i in range(len(predictions_list)):
                #print("%s = %f" % (predictions_list[i][0], predictions_list[i][1]))
            if max(obj.output()) > 0.7:   # 当识别到的最大的相似度大于 0.7 时才认为是识别到了垃圾
                # 将识别到的垃圾名称赋值给 rubbish_name
                rubbish_name = labels[obj.output().index(max(obj.output()))]
                # 打印识别到的垃圾名称
                print('识别到的垃圾是 %s' % rubbish_name)

        #print(clock.fps(), "fps")  # 打印帧率

# 程序入口
if __name__ == '__main__':
    rubbish_init_setup()  # 执行初始化函数
    try:
        rubbish_main()    # 执行主函数
    except:
        pass
```

二、代码使用方法

将 labels.txt 和 trained.tflite 复制粘贴到 openmv 的盘符中，然后在 openmv ide 中打开 rubbish_classification.py 运行使用即可。

三、运行效果

代码运行后，openmv 会将识别到的结果在串行终端窗口打印出来。

我们示例源码程序使用的模型文件只包含了以下四种垃圾，可以使用我们的测试图片来进行测试。



四、知识点

训练神经网络模型的方法在上一节数字识别课程中已经讲过，此处不再赘述。

关于 tensorflow 神经网络模型的使用可以参考官方文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.tf.html#

【本课小结】：

注意：

1. 使用训练神经网络模型的方法实现垃圾分类只适用于 openmv h7 plus。
2. 在我们的资料中的对应源码例程中有提供的训练好的模型文件 trained.tflite 和配套文件 labels.txt，你可以直接将模型文件 trained.tflite 和配套文件 labels.txt 复制粘贴到 openmv 的盘符中，然后在 ide 中运行代码 rubbish_classification.py 来使用即可。
3. 你也可以自己训练神经网络模型来使用，具体训练方法在数字识别中已经讲过，可以参考，不同的地方主要就是新类文件夹的名称需要是你要识别的垃圾的名称，然后在网址中上传文件时的分类名需要是你要识别的垃圾的名称，其他步骤都一样。

第 2 课 追小球的小车

【课程目标】：

本课来实现小车追小球的功能。

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动

【本课内容】：

本节来学习让小车追小球，具体控制小车运动使用的是 pid 算法

pid 算法是控制中运用非常多的一个算法，原理网上有很多，可以参考下面这两个链接：

https://zh.wikipedia.org/wiki/PID 控制器

http://baike.baidu.com/link?url=-obQq78Ur4bTeqA10bInjO6y0euQFcWL9WW18vq2hA3fyHN3rt32o79F2WPE7cK0Di9M6904rlHD9ttvVTySIK

一、代码

```
# 追小球的小车
import sensor, image, time
from pyb import LED
from ZL_SDK import zl_car_run, zl_pan_tilt, zl_uart3
from ZL_SDK.pid import PID

target_threshold = (15, 58, 39, 74, 62, 18)     # 颜色阈值
size_threshold = 475                            # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车后退
x_pid = PID(p=0.7, i=1, imax=100)               # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)             # 控制小车速度

# 初始化函数
def init_setup():
    global sensor, clock                        # 设置为全局变量
    sensor.reset()                              # 初始化感光元件
    sensor.set_vflip(True)
    sensor.set_hmirror(True)
    sensor.set_pixformat(sensor.RGB565)         # 将图像格式设置为彩色
    sensor.set_framesize(sensor.QQVGA)          # 将图像大小设置为 QQVGA，即 160*120
    sensor.skip_frames(10)                      # 跳过 10 帧使以上设置生效
    sensor.set_auto_whitebal(False)             # 关闭白平衡
    clock = time.clock()                        # 创建时钟对象
    zl_pan_tilt.main(0, -50)                    # 云台倾斜向下，方便查找目标


# 查找图像中最大的色块
def find_max(blobs):
    max_size = 0
    for blob in blobs:
        if blob[2] * blob[3] > max_size:
```

```
            max_blob = blob
            max_size = blob[2] * blob[3]
    return max_blob

# 主函数
def main():
    clock.tick()                                                    # 更新 FPS 帧率时钟
    img = sensor.snapshot()                                         # 拍一张照片并返回图像

    blobs = img.find_blobs([target_threshold])                      # 在图像中查找目标颜色区域
    if blobs:                                                       # 当查找到目标颜色区域后
        max_blob = find_max(blobs)                                  # 查找图像中最大的目标色块
        x_error = max_blob[5] - img.width() / 2                     # x 轴差值
        print(max_blob[2]*max_blob[3])
        h_error = max_blob[2] * max_blob[3] - size_threshold        # 面积差值
        print("x error: %s , h error: %s" % (x_error, h_error))
        img.draw_rectangle(max_blob[0:4])                           # 将找到的目标颜色区域用矩形框出来
        img.draw_cross(max_blob[5], max_blob[6])                    # 在目标颜色区域的中心点处画十字
        x_output = x_pid.get_pid(x_error, 1)
        h_output = h_pid.get_pid(h_error, 1)
        print("x_output %s ,h_output %s" % (x_output, h_output))
        zl_car_run.car_run(int(-h_output-x_output)*10, int(-h_output+x_output)*10)  # 小车开始追小球
    else:
        zl_car_run.car_run(400, -400)                               # 当未找到目标颜色区域时，让小车原地转圈找寻目标颜色

# 程序入口
if __name__ == '__main__':
    init_setup()                # 执行初始化函数
    zl_uart3.init_setup()       # 串口初始化
    try:
        while 1:
            main()              # 执行主函数
    except:
        zl_pan_tilt.middle()    # 云台复位
        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

1.根据实际情况修改 car_follow_color.py 代码中的如下四项：

```
target_threshold = (15, 58, 39, 74, 62, 18)     # 颜色阈值
size_threshold = 475                            # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车
x_pid = PID(p=0.7, i=1, imax=100)               # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)             # 控制小车速度
```

target_threshold 是小球的颜色阈值，可以根据实际情况来修改阈值，我这里使用的是一个红色木块。

size_threshold 是小球的像素点数，当小于这个像素点数时小车前进，大于这个像素点数时小车后退。可以根据实际情况修改该值。我这里使用的是 3cm*3cm 的红色小木块，对应 size_threshold 值大约为 475.

x_pid 控制的是小车的方向，也就是如果你发现小车方向拐的太大时，可以将该值

调小一些来使用。

h_pid 控制的是小车的速度，也就是当你发现小车追小球时速度稍微快了一些，可以将该值调小一些来使用。

2.将 ZL_SDK 文件夹保存到 openmv 的盘符中，然后在 openmv ide 中把 car_follow_color.py 保存到 openmv 中脱机运行使用即可，注意运行之前调整颜色阈值（调整阈值可参考颜色识别章节）

三、运行效果

小车会跟随小球（注意不一定是小球，其他也可以，比如我这里使用一个红色小木块）移动，当小球靠近小车时，小车后退，当小球远离小车时，小车跟着前进，并且当小球拐弯时，小车也会跟着拐弯。当小球未出现在小车的视野内时，小车会在原地转圈寻找小球，找到小球后继续跟随。

四、知识点

1.本节控制小车运动使用的是 pid 算法

pid 算法是控制中运用非常多的一个算法，原理网上有很多，可以参考下面这两个链接：

https://zh.wikipedia.org/wiki/PID 控制器

http://baike.baidu.com/link?url=-obQq78Ur4bTeqA10bIniO6y0euQFcWL9WW18vq2hA3fyHN3rt32o79F2WPE7cK0Di9M6904rlHD9ttvVTySIK

2.在主代码中，具体需要修改的就是如下四项：

```
target_threshold = (15, 58, 39, 74, 62, 18)     # 颜色阈值
size_threshold = 475                            # 定义一个目标阈值区域的像素点数，小于这个阈值时使小车前进，大于这个阈值时使小车
x_pid = PID(p=0.7, i=1, imax=100)               # 控制小车方向
h_pid = PID(p=0.13, i=0.1, imax=50)             # 控制小车速度
```

target_threshold 是小球的颜色阈值，可以根据实际情况来修改阈值，我这里使用的是一个红色木块。

size_threshold 是小球的像素点数，当小于这个像素点数时小车前进，大于这个像素点数时小车后退。可以根据实际情况修改该值。我这里使用的是 3cm*3cm 的红色小木块，对应 size_threshold 值大约为 475.

x_pid 控制的是小车的方向，也就是如果你发现小车方向拐的太大时，可以将该值调小一些来使用。

h_pid 控制的是小车的速度，也就是当你发现小车追小球时速度稍微快了一些，可以将该值调小一些来使用。

3.关于小车追小球也可参考官方文档及教程：

文档：

https://book.openmv.cc/project/zhui-xiao-qiu-de-xiao-8f665d28-project-pan-tilt-md.html

视频教程：https://singtown.com/learn/49239/

【本课小结】：

本节学习了让小车追小球，具体控制小车运动使用的是 pid 算法

众灵科技

第 3 课 巡线小车

【课程目标】：

本课学习通过视觉识别、快速线性回归的方法来实现巡线功能

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动

【本课内容】：

使用快速线性回归来实现巡线

快速线性回归，可以使用 get_regression 快速返回视野中的一条回归直线，可以得到直线的角度 theta、直线的偏移距离 rho，然后根据角度和偏移距离动态的调整小车的速度。

一、代码

```
# 巡线小车
# 使用快速线性回归来实现
# 快速线性回归，可以使用 get_regression 快速返回视野中的一条回归直线，
# 可以得到直线的角度 theta、直线的偏移距离 rho，然后根据角度和偏移距离动态的调整小车的速度
import sensor, image, time
from pyb import LED
from ZL_SDK import zl_car_run, zl_pan_tilt, zl_uart3
from ZL_SDK.pid import PID

THRESHOLD = (32, 6, -23, 9, -18, 45)       # 线的颜色阈值，此处为白底黑线

rho_pid = PID(p=0.5, i=0)                  # 控制线在视野中的位置，即处于图像的左侧右侧或中间位置
theta_pid = PID(p=0.005, i=0)              # 控制直线角度的偏移

# 初始化函数
def init_setup():
    global sensor, clock                   # 设置为全局变量

    LED(1).on()                            # 打开板载 LED 灯补光，使巡线环境稳定一些
    LED(2).on()
    LED(3).on()

    sensor.reset()                         # 初始化感光元件
    sensor.set_pixformat(sensor.RGB565)    # 设置图像格式为彩色
    sensor.set_framesize(sensor.QQQVGA)            # 设置图像大小为 QQQVGA，80x60 (4,800 pixels) - O(N^2) max = 2,3040,000.
    sensor.skip_frames(10)                         # 跳过 10 帧使以上设置生效，WARNING: If you use QQVGA it may take seconds
    clock = time.clock()                   # 创建时钟对象
    zl_pan_tilt.main(0, -65)               # 云台倾斜向下，方便查找黑线

# 主函数
def main():
    clock.tick()                                        # 更新 FPS 帧率时钟
```

```
        img = sensor.snapshot().binary([THRESHOLD])                 # 拍一张照片并返回图像，图像为二值化格式，即黑线部分显示为白色，非黑色区域显示为黑色
        line = img.get_regression([(100,100)], robust=True)         # 快速线性回归，可以快速返回视野中的一条回归直线，可得到直线的角度、偏移的距离
        if (line):
            rho_err = abs(line.rho()) - img.width() / 2             # 计算直线相对于图像中央偏移的距离
            if line.theta() > 90:                                   # 通过对直线角度的判断来进行坐标的变换，即以直线最远点画竖线来作为 x 轴的零点位置
                theta_err = line.theta() - 180
            else:
                theta_err = line.theta()
            img.draw_line(line.line(), color=127)                   # 将直线画出来
            print(rho_err,line.magnitude(),rho_err)
            if line.magnitude() > 8:                                # magnitude 为霍夫变换后的直线的模，值越大说明效果越好
                #if -40<b_err<40 and -30<t_err<30:
                rho_output = rho_pid.get_pid(rho_err, 1)
                theta_output = theta_pid.get_pid(theta_err, 1)
                output = rho_output + theta_output
                zl_car_run.car_run(int(50+output)*10, int(50-output)*10)  # 开始巡线
            else:
                zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
        else:
            zl_car_run.car_run(650,-650)  # 小车原地旋转，找寻黑线
            pass
        #print(clock.fps())

# 程序入口
if __name__ == '__main__':
    init_setup()    # 执行初始化函数
    zl_uart3.init_setup()
    try:
        while 1:
            main()  # 执行主函数
    except:
        zl_pan_tilt.middle()    # 云台复位
        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

将 ZL_SDK 文件夹保存到 openmv 的盘符中，然后在 openmv ide 中将 car_follow_line.py 存到 openmv 中脱机运行即可，注意运行之前可能需要根据环境调整颜色阈值

三、运行效果

将代码存到 openmv 之后，上电即可脱机运行，将小车放在黑线上，小车就可以巡黑线走了。

四、知识点

注意：rho_pid 控制线在视野中的位置，即处于图像的左侧右侧或中间位置

theta_pid 控制直线角度的偏移

```
rho_pid = PID(p=0.5, i=0)                    # 控制线在视野中的位置，即处于图像的左侧右侧或中间位置
theta_pid = PID(p=0.005, i=0)                # 控制直线角度的偏移
```

关于 img.get_regression 方法可以在官方文档找到

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/omv.image.html#

打开上述网址之后，按键盘快捷键 ctrl f，然后搜索 get_regression，可以查找到 image.get_regression 的使用方法及 Line 类 – 直线对象的使用。

关于巡线小车也可参考官方教程：https://singtown.com/learn/50037/

【本课小结】：

本课学习了通过视觉识别、快速线性回归的方法来实现巡线功能。

第 4 课 微信小程序控制

【课程目标】：

本课学习使用蓝牙模块通过微信小程序来控制 openmv 小车。

【课前准备】：

OpenMV、扩展总线板、云台、底座、zlink 模块、电源、总线双路驱动、蓝牙模块

【本课内容】：

本课学习使用蓝牙模块通过微信小程序来控制 openmv 小车，注意要将蓝牙模块插到 zlink 模块的 TTL 串口上使用。

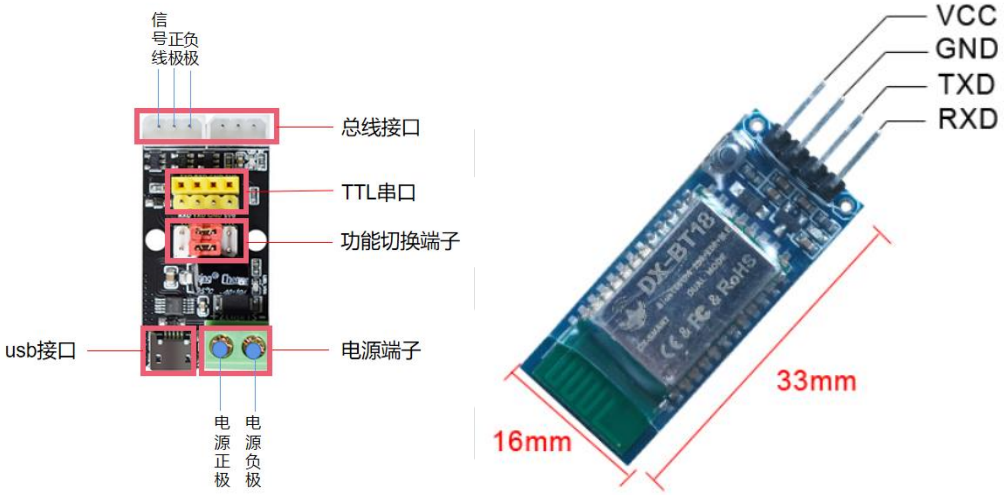
注意蓝牙模块的 VCC 对应 zlink 模块 TTL 串口的 5V0

蓝牙模块的 GND 对应 zlink 模块 TTL 串口的 GND

蓝牙模块的 TXD 对应 zlink 模块 TTL 串口的 RXD

蓝牙模块的 RXD 对应 zlink 模块 TTL 串口的 TXD

功能切换端子的跳线帽要短接 TTL 和 ZX，即左侧



注意：本课代码同综合例程代码

本综合例程包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车、前进、后退、左转、右转、停止、复位。

注意颜色相关的需要根据实际情况调节阈值范围并将代码中的阈值范围替换掉，分辨人脸需提前收集好人脸图片放在对应文件夹中并将代码按照前面的教程进行一些修改，具体使用方法在下面会看到。

一、代码

```
1.  # 综合例程
2.  # 包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车
3.  # 注意颜色相关的需要根据实际情况调节阈值范围，分辨人脸需提前收集好人脸图片
```

```
import sensor, image, time, pyb
from AI_Functions import (
    multi_color_detection, face_detection, color_tracking, face_tracking,
    face_recognition, qrcode_detection, mutil_template_num_matching,
    car_follow_color, car_follow_line
)
from ZL_SDK import (zl_uart3, zl_led, zl_car_run, zl_pan_tilt, pid)

# 串口接收数据后对应执行的命令值
uart_recv_data_command = None

# 1.对应每个功能的一个标志位，标志位用来初始化一次感光元件 2.串口接收到的对应数据 3.数据对应指令
flag_dict = {
    'multi_color_detection': {'flag': True, 'uart_recv_data': b'$YSSB!', 'command': 'YSSB'},
    'face_detection': {'flag': True, 'uart_recv_data': b'$RLSB!', 'command': 'RLSB'},
    'color_tracking': {'flag': True, 'uart_recv_data': b'$YSGS!', 'command': 'YSGS'},
    'face_tracking': {'flag': True, 'uart_recv_data': b'$RLGS!', 'command': 'RLGS'},
    'face_recognition': {'flag': True, 'uart_recv_data': b'$FBRL!', 'command': 'FBRL'},
    'qrcode_detection': {'flag': True, 'uart_recv_data': b'$EWMSB!', 'command': 'EWMSB'},
    'mutil_template_num_matching': {'flag': True, 'uart_recv_data': b'$SZSB!', 'command': 'SZSB'},
    'car_follow_color': {'flag': True, 'uart_recv_data': b'$ZXQ!', 'command': 'ZXQ'},
    'car_follow_line': {'flag': True, 'uart_recv_data': b'$XJMS!', 'command': 'XJMS'},
}
# 串口接收到对应指令控制小车运动及云台运动
motor_pan_tilt_command_dict = {
    '前进': b'$QJ!',
    '后退': b'$HT!',
    '左转': b'$ZZ!',
    '右转': b'$YZ!',
    '停止': b'$TZ!',
    '复位': b'$RESET!',
}

# 标志位处理函数，标志位用来初始化一次感光元件，并让其他功能对应的 flag 变为真
def flag_func(flag):
    for i in flag_dict:
        if i == flag:
            flag_dict[i]['flag'] = False
        else:
            flag_dict[i]['flag'] = True
    print(flag_dict)

# 串口接收数据的处理
def uart_recv_data_handle():
    global uart_recv_data_command
    uart_recv_data = zl_uart3.main()
    #print(uart_recv_data, type(uart_recv_data))
    if uart_recv_data:
        if uart_recv_data == flag_dict['multi_color_detection']['uart_recv_data']:        # 颜色识别
            uart_recv_data_command = flag_dict['multi_color_detection']['command']
        elif uart_recv_data == flag_dict['face_detection']['uart_recv_data']:             # 人脸识别
            uart_recv_data_command = flag_dict['face_detection']['command']
```

```
        elif uart_recv_data == flag_dict['color_tracking']['uart_recv_data']:                # 颜色跟随
            uart_recv_data_command = flag_dict['color_tracking']['command']
        elif uart_recv_data == flag_dict['face_tracking']['uart_recv_data']:                 # 人脸跟随
            uart_recv_data_command = flag_dict['face_tracking']['command']
        elif uart_recv_data == flag_dict['face_recognition']['uart_recv_data']:              # 分辨人脸
            uart_recv_data_command = flag_dict['face_recognition']['command']
        elif uart_recv_data == flag_dict['qrcode_detection']['uart_recv_data']:              # 二维码识别
            uart_recv_data_command = flag_dict['qrcode_detection']['command']
        elif uart_recv_data == flag_dict['mutil_template_num_matching']['uart_recv_data']: # 数字识别
            uart_recv_data_command = flag_dict['mutil_template_num_matching']['command']
        elif uart_recv_data == flag_dict['car_follow_color']['uart_recv_data']:              # 追小球
            uart_recv_data_command = flag_dict['car_follow_color']['command']
        elif uart_recv_data == flag_dict['car_follow_line']['uart_recv_data']:               # 循迹模式
            uart_recv_data_command = flag_dict['car_follow_line']['command']
        elif uart_recv_data == motor_pan_tilt_command_dict['前进']:                          # 前进
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'QJ'
        elif uart_recv_data == motor_pan_tilt_command_dict['后退']:                          # 后退
            zl_car_run.car_run(zl_car_run.backward_speed, zl_car_run.backward_speed)
            uart_recv_data_command = 'HT'
        elif uart_recv_data == motor_pan_tilt_command_dict['左转']:                          # 左转
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'ZZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['右转']:                          # 右转
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'YZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['停止']:                          # 停止
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'TZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['复位']:                          # 复位
            zl_pan_tilt.middle()
            uart_recv_data_command = 'FW'
        #print(uart_recv_data_command)


# 功能执行函数
def function_run():
    if uart_recv_data_command == flag_dict['multi_color_detection']['command']:  # 颜色识别功能
        if flag_dict['multi_color_detection']['flag']:                           # 只初始化一次
            multi_color_detection.init_setup()                                   # 颜色识别初始化
            flag_func('multi_color_detection')
        multi_color_detection.main()                                             # 执行颜色识别主函数

    elif uart_recv_data_command == flag_dict['face_detection']['command']:       # 人脸识别功能
        if flag_dict['face_detection']['flag']:                                  # 只初始化一次
            face_detection.init_setup()                                          # 人脸识别初始化
            flag_func('face_detection')
        face_detection.main()                                                    # 执行人脸识别主函数

    elif uart_recv_data_command == flag_dict['color_tracking']['command']:       # 颜色跟随功能
        if flag_dict['color_tracking']['flag']:                                  # 只初始化一次
```

```
            color_tracking.init_setup()                                    # 颜色跟随初始化
            flag_func('color_tracking')
        color_tracking.main()                                              # 执行颜色跟随主函数

    elif uart_recv_data_command == flag_dict['face_tracking']['command']:  # 人脸跟随功能
        if flag_dict['face_tracking']['flag']:                             # 只初始化一次
            face_tracking.init_setup()                                     # 人脸跟随初始化
            flag_func('face_tracking')
        face_tracking.main()                                               # 执行人脸跟随主函数

    elif uart_recv_data_command == flag_dict['face_recognition']['command']:  # 分辨人脸功能
        if flag_dict['face_recognition']['flag']:                          # 只初始化一次
            face_recognition.init_setup()                                  # 分辨人脸初始化
            flag_func('face_recognition')
        face_recognition.main()                                            # 执行分辨人脸主函数

    elif uart_recv_data_command == flag_dict['qrcode_detection']['command']:  # 二维码识别功能
        if flag_dict['qrcode_detection']['flag']:                          # 只初始化一次
            qrcode_detection.init_setup()                                  # 二维码识别初始化
            flag_func('qrcode_detection')
        qrcode_detection.main()                                            # 执行二维码主函数

    elif uart_recv_data_command == flag_dict['mutil_template_num_matching']['command']:  # 数字识别功能
        if flag_dict['mutil_template_num_matching']['flag']:               # 只初始化一次
            mutil_template_num_matching.init_setup()                       # 数字识别初始化
            flag_func('mutil_template_num_matching')
        mutil_template_num_matching.main()                                 # 执行数字识别主函数

    elif uart_recv_data_command == flag_dict['car_follow_color']['command']:  # 追小球功能
        if flag_dict['car_follow_color']['flag']:                          # 只初始化一次
            car_follow_color.init_setup()                                  # 追小球功能初始化
            flag_func('car_follow_color')
        car_follow_color.main()                                            # 执行追小球主函数

    elif uart_recv_data_command == flag_dict['car_follow_line']['command']:  # 循迹功能
        if flag_dict['car_follow_line']['flag']:                           # 只初始化一次
            car_follow_line.init_setup()                                   # 循迹模式初始化
            flag_func('car_follow_line')
        car_follow_line.main()                                             # 执行循迹主函数

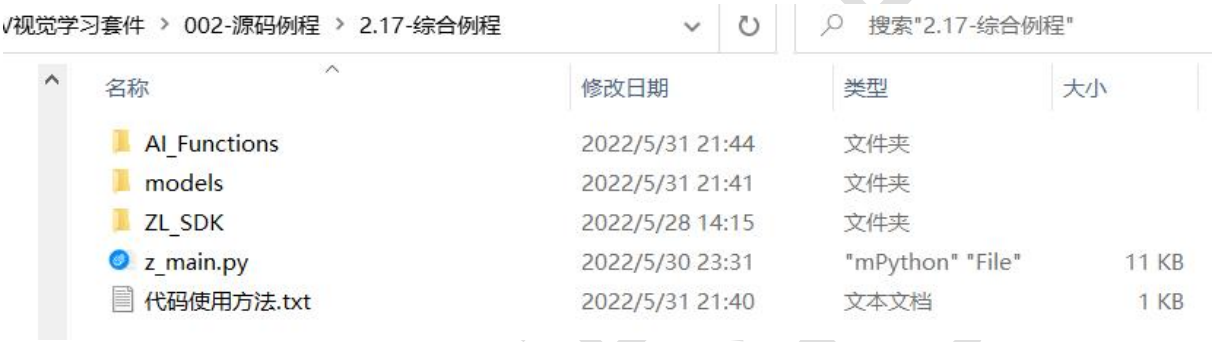
# 主函数
def main():
    uart_recv_data_handle()  # 串口接收数据并处理
    function_run()           # 根据接收到的数据执行不同的功能


# 程序入口
if __name__ == '__main__':
    zl_led.init_setup()      # led 灯初始化
    zl_uart3.init_setup()    # 串口初始化
    for i in range(6):       # led 灯亮三下，代表初始化完成，可以开始执行各项功能了
```

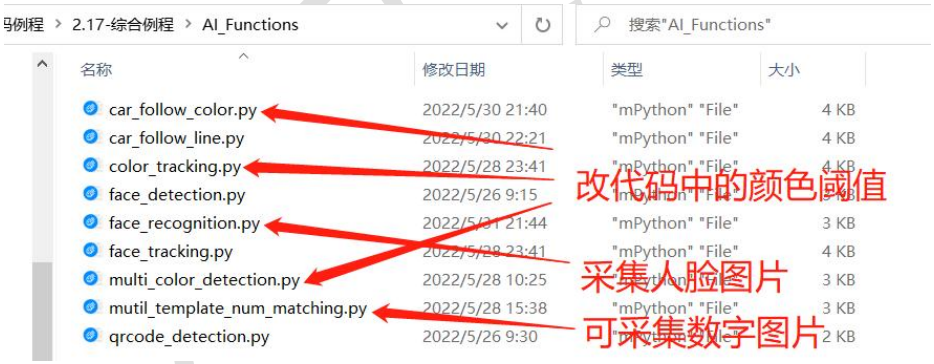
```
158.        zl_led.main(0.3)      # 每次亮 0.3 秒
159.    zl_pan_tilt.middle()      # 云台在程序开始后复位
160.    try:                      # 异常处理
161.        while 1:              # 无限循环
162.            main()            # 执行主函数
163.    except:
164.        zl_pan_tilt.middle() # 云台复位
165.        zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

二、代码使用方法

1.在综合例程中共有如下几个文件和文件夹



2.在使用综合例程之前，需要将涉及到的颜色相关的代码，如颜色识别和颜色跟随代码中的颜色阈值按照前面的教程进行修改，分辨人脸的代码需要先采集人脸图片并存放到 models 文件夹中的 ZLTech 中的对应位置，数字识别可以使用我们提供的模板，也可自己根据前面的教程采集数字图片模板并存放到 models 文件夹中的 template_num 中。



3.将 AI_Functions、models、ZL_SDK 文件夹都保存到 openmv 的盘符中，然后在 openmv ide 中将 z_main.py 文件存到 openmv 中，脱机运行即可，也可在线运行，不过要注意不要让小车跑了。

三、运行效果

代码运行后，我们主要通过蓝牙来控制小车。以下是各功能及对应指令：

对应功能	指令
颜色识别	$YSSB!
颜色跟随	$YSGS!
人脸识别	$RLSB!
人脸跟随	$RLGS!
分辨人脸	$FBRL!
二维码识别	$EWMSB!
数字识别	$SZSB!
小车追小球	$ZXQ!
巡线小车	$XJMS!
小车前进	$QJ!
小车后退	$HT!
小车左转	$ZZ!
小车右转	$YZ!
小车停止	$TZ!
云台复位	$RESET!

蓝牙模块的基本用法：

1. 首先根据上面的教程将蓝牙模块插到 zlink 模块上，然后通过扩展板供电
2. 在微信小程序中搜索小程序'众灵机器人'
3. 进入众灵机器人小程序，然后找到自定义设置，点击进入自定义设置，然后点击右上角的红色蓝牙图标





4.在弹出的界面中点击'点击连接'来连接蓝牙，注意如果你没有打开蓝牙的话，它会提醒你先打开蓝牙，连接成功后，会自动返回自定义设置界面，并且右上角的蓝牙图标会变为蓝色（下次如果连接的话会自动连接）。





5.点击左上角的设置图标，可以设置每个按钮的功能，注意设置完每个按钮之后都要点一下对应的'应用'按钮



比如这里我设置如下 2 个示例：





注意：此处我只设置了按下的命令，如果你需要按下和抬起都有对应的命令，也可以在对应功能的抬起命令那里填写对应指令。比如我这里可以将按钮 1 设置为按下时前进，抬起时后退。



当我将所有功能都设置完成后，会如下图所示，当然你也可以变换位置：



然后就可以按这些配置好的按钮来操控小车了。

如下是各功能对应的指令：

对应功能	指令
颜色识别	$YSSB!
颜色跟随	$YSGS!
人脸识别	$RLSB!
人脸跟随	$RLGS!
分辨人脸	$FBRL!
二维码识别	$EWMSB!
数字识别	$SZSB!
小车追小球	$ZXQ!
巡线小车	$XJMS!
小车前进	$QJ!
小车后退	$HT!
小车左转	$ZZ!
小车右转	$YZ!
小车停止	$TZ!
云台复位	$RESET!

【本课小结】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序，然后通过小程序来调用。

注意在运行程序之前根据教程做好准备工作，不然程序可能出错。

众灵科技

第 5 课 语音交互

【课程目标】：

本课学习使用语音识别模块来控制 openmv 小车。

【课前准备】：

OpenMV、扩展板、云台、底座、zlink 模块、电源、总线双路驱动、语音识别模块

【本课内容】：

本综合例程包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车、前进、后退、左转、右转、停止、复位。

注意颜色相关的需要根据实际情况调节阈值范围并将代码中的阈值范围替换掉，分辨人脸需提前收集好人脸图片放在对应文件夹中并将代码按照前面的教程进行一些修改，具体使用方法在下面会看到。

一、代码

```
# 综合例程
# 包含颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、巡线小车、追小球的小车
# 注意颜色相关的需要根据实际情况调节阈值范围，分辨人脸需提前收集好人脸图片
import sensor, image, time, pyb
from AI_Functions import (
    multi_color_detection, face_detection, color_tracking, face_tracking,
    face_recognition, qrcode_detection, mutil_template_num_matching,
    car_follow_color, car_follow_line
)
from ZL_SDK import (zl_uart3, zl_led, zl_car_run, zl_pan_tilt, pid)

# 串口接收数据后对应执行的命令值
uart_recv_data_command = None

# 1.对应每个功能的一个标志位，标志位用来初始化一次感光元件 2.串口接收到的对应数据 3.数据对应指令
flag_dict = {
    'multi_color_detection': {'flag': True, 'uart_recv_data': b'$YSSB!', 'command': 'YSSB'},
    'face_detection': {'flag': True, 'uart_recv_data': b'$RLSB!', 'command': 'RLSB'},
    'color_tracking': {'flag': True, 'uart_recv_data': b'$YSGS!', 'command': 'YSGS'},
    'face_tracking': {'flag': True, 'uart_recv_data': b'$RLGS!', 'command': 'RLGS'},
    'face_recognition': {'flag': True, 'uart_recv_data': b'$FBRL!', 'command': 'FBRL'},
    'qrcode_detection': {'flag': True, 'uart_recv_data': b'$EWMSB!', 'command': 'EWMSB'},
    'mutil_template_num_matching': {'flag': True, 'uart_recv_data': b'$SZSB!', 'command': 'SZSB'},
    'car_follow_color': {'flag': True, 'uart_recv_data': b'$ZXQ!', 'command': 'ZXQ'},
    'car_follow_line': {'flag': True, 'uart_recv_data': b'$XJMS!', 'command': 'XJMS'},
}
# 串口接收到对应指令控制小车运动及云台运动
motor_pan_tilt_command_dict = {
    '前进': b'$QJ!',
    '后退': b'$HT!',
```

```
    '左转': b'$ZZ!',
    '右转': b'$YZ!',
    '停止': b'$TZ!',
    '复位': b'$RESET!',
}

# 标志位处理函数，标志位用来初始化一次感光元件，并让其他功能对应的 flag 变为真
def flag_func(flag):
    for i in flag_dict:
        if i == flag:
            flag_dict[i]['flag'] = False
        else:
            flag_dict[i]['flag'] = True
    print(flag_dict)

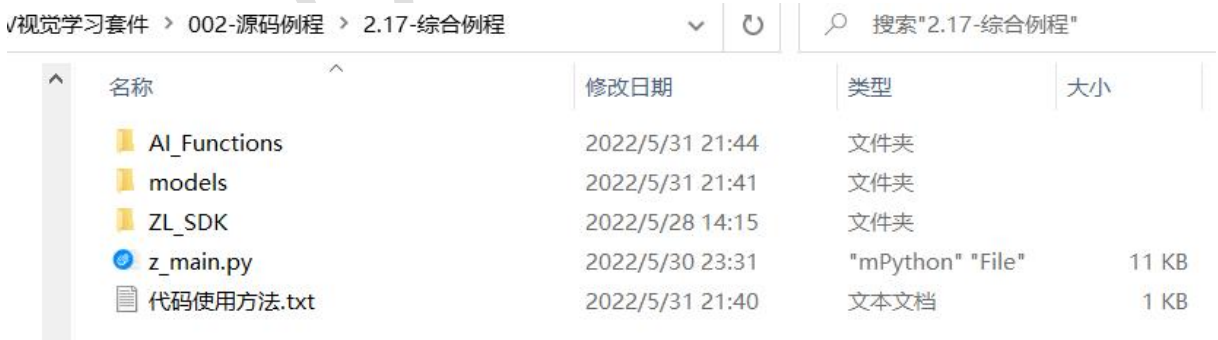
# 串口接收数据的处理
def uart_recv_data_handle():
    global uart_recv_data_command
    uart_recv_data = zl_uart3.main()
    #print(uart_recv_data, type(uart_recv_data))
    if uart_recv_data:
        if uart_recv_data == flag_dict['multi_color_detection']['uart_recv_data']:          # 颜色识别
            uart_recv_data_command = flag_dict['multi_color_detection']['command']
        elif uart_recv_data == flag_dict['face_detection']['uart_recv_data']:               # 人脸识别
            uart_recv_data_command = flag_dict['face_detection']['command']
        elif uart_recv_data == flag_dict['color_tracking']['uart_recv_data']:               # 颜色跟随
            uart_recv_data_command = flag_dict['color_tracking']['command']
        elif uart_recv_data == flag_dict['face_tracking']['uart_recv_data']:                # 人脸跟随
            uart_recv_data_command = flag_dict['face_tracking']['command']
        elif uart_recv_data == flag_dict['face_recognition']['uart_recv_data']:             # 分辨人脸
            uart_recv_data_command = flag_dict['face_recognition']['command']
        elif uart_recv_data == flag_dict['qrcode_detection']['uart_recv_data']:             # 二维码识别
            uart_recv_data_command = flag_dict['qrcode_detection']['command']
        elif uart_recv_data == flag_dict['mutil_template_num_matching']['uart_recv_data']: # 数字识别
            uart_recv_data_command = flag_dict['mutil_template_num_matching']['command']
        elif uart_recv_data == flag_dict['car_follow_color']['uart_recv_data']:             # 追小球
            uart_recv_data_command = flag_dict['car_follow_color']['command']
        elif uart_recv_data == flag_dict['car_follow_line']['uart_recv_data']:              # 循迹模式
            uart_recv_data_command = flag_dict['car_follow_line']['command']
        elif uart_recv_data == motor_pan_tilt_command_dict['前进']:                         # 前进
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'QJ'
        elif uart_recv_data == motor_pan_tilt_command_dict['后退']:                         # 后退
            zl_car_run.car_run(zl_car_run.backward_speed, zl_car_run.backward_speed)
            uart_recv_data_command = 'HT'
        elif uart_recv_data == motor_pan_tilt_command_dict['左转']:                         # 左转
            zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.forward_speed)
            uart_recv_data_command = 'ZZ'
        elif uart_recv_data == motor_pan_tilt_command_dict['右转']:                         # 右转
            zl_car_run.car_run(zl_car_run.forward_speed, zl_car_run.stop_speed)
            uart_recv_data_command = 'YZ'
```

```
82.         elif uart_recv_data == motor_pan_tilt_command_dict['停止']:                    # 停止
83.             zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)
84.             uart_recv_data_command = 'TZ'
85.         elif uart_recv_data == motor_pan_tilt_command_dict['复位']:                    # 复位
86.             zl_pan_tilt.middle()
87.             uart_recv_data_command = 'FW'
88.         #print(uart_recv_data_command)
89.
90.
91. # 功能执行函数
92. def function_run():
93.     if uart_recv_data_command == flag_dict['multi_color_detection']['command']:  # 颜色识别功能
94.         if flag_dict['multi_color_detection']['flag']:                           # 只初始化一次
95.             multi_color_detection.init_setup()                                   # 颜色识别初始化
96.             flag_func('multi_color_detection')
97.         multi_color_detection.main()                                             # 执行颜色识别主函数
98.
99.     elif uart_recv_data_command == flag_dict['face_detection']['command']:       # 人脸识别功能
100.         if flag_dict['face_detection']['flag']:                                  # 只初始化一次
101.             face_detection.init_setup()                                          # 人脸识别初始化
102.             flag_func('face_detection')
103.         face_detection.main()                                                    # 执行人脸识别主函数
104.
105.     elif uart_recv_data_command == flag_dict['color_tracking']['command']:       # 颜色跟随功能
106.         if flag_dict['color_tracking']['flag']:                                  # 只初始化一次
107.             color_tracking.init_setup()                                          # 颜色跟随初始化
108.             flag_func('color_tracking')
109.         color_tracking.main()                                                    # 执行颜色跟随主函数
110.
111.     elif uart_recv_data_command == flag_dict['face_tracking']['command']:       # 人脸跟随功能
112.         if flag_dict['face_tracking']['flag']:                                   # 只初始化一次
113.             face_tracking.init_setup()                                           # 人脸跟随初始化
114.             flag_func('face_tracking')
115.         face_tracking.main()                                                     # 执行人脸跟随主函数
116.
117.     elif uart_recv_data_command == flag_dict['face_recognition']['command']:     # 分辨人脸功能
118.         if flag_dict['face_recognition']['flag']:                                # 只初始化一次
119.             face_recognition.init_setup()                                        # 分辨人脸初始化
120.             flag_func('face_recognition')
121.         face_recognition.main()                                                  # 执行分辨人脸主函数
122.
123.     elif uart_recv_data_command == flag_dict['qrcode_detection']['command']:     # 二维码识别功能
124.         if flag_dict['qrcode_detection']['flag']:                                # 只初始化一次
125.             qrcode_detection.init_setup()                                        # 二维码识别初始化
126.             flag_func('qrcode_detection')
127.         qrcode_detection.main()                                                  # 执行二维码主函数
128.
129.     elif uart_recv_data_command == flag_dict['mutil_template_num_matching']['command']:  # 数字识别功能
130.         if flag_dict['mutil_template_num_matching']['flag']:                     # 只初始化一次
131.             mutil_template_num_matching.init_setup()                             # 数字识别初始化
132.             flag_func('mutil_template_num_matching')
```

```
133.         mutil_template_num_matching.main()                                  # 执行数字识别主函数
134.
135.     elif uart_recv_data_command == flag_dict['car_follow_color']['command']:    # 追小球功能
136.         if flag_dict['car_follow_color']['flag']:                              # 只初始化一次
137.             car_follow_color.init_setup()                                      # 追小球功能初始化
138.             flag_func('car_follow_color')
139.         car_follow_color.main()                                                # 执行追小球主函数
140.
141.     elif uart_recv_data_command == flag_dict['car_follow_line']['command']:    # 循迹功能
142.         if flag_dict['car_follow_line']['flag']:                               # 只初始化一次
143.             car_follow_line.init_setup()                                       # 循迹模式初始化
144.             flag_func('car_follow_line')
145.         car_follow_line.main()                                                 # 执行循迹主函数
146.
147. # 主函数
148. def main():
149.     uart_recv_data_handle()  # 串口接收数据并处理
150.     function_run()           # 根据接收到的数据执行不同的功能
151.
152.
153. # 程序入口
154. if __name__ == '__main__':
155.     zl_led.init_setup()      # led 灯初始化
156.     zl_uart3.init_setup()    # 串口初始化
157.     for i in range(6):       # led 灯亮三下，代表初始化完成，可以开始执行各项功能了
158.         zl_led.main(0.3)     # 每次亮 0.3 秒
159.     zl_pan_tilt.middle()     # 云台在程序开始后复位
160.     try:                     # 异常处理
161.         while 1:             # 无限循环
162.             main()           # 执行主函数
163.     except:
164.         zl_pan_tilt.middle() # 云台复位
165.         zl_car_run.car_run(zl_car_run.stop_speed, zl_car_run.stop_speed)  # 小车停止
```

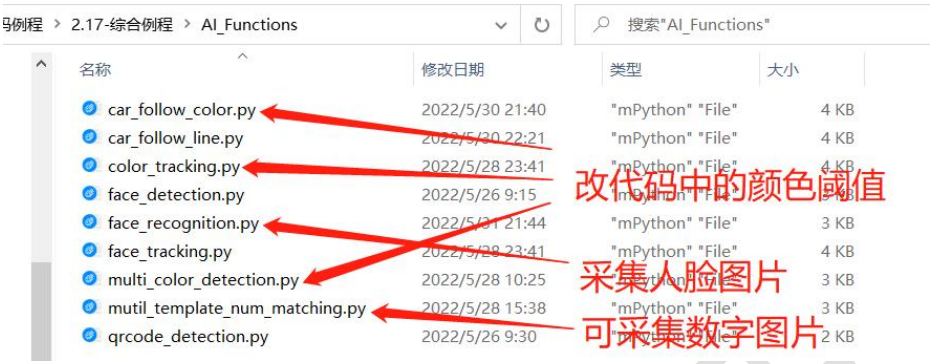
二、代码使用方法

1.在此例程中共有如下几个文件和文件夹



2.在使用综合例程之前，需要将涉及到的颜色相关的代码，如颜色识别和颜色跟随代码中的颜色阈值按照前面的教程进行修改，分辨人脸的代码需要先采集人脸图片并存放到

models 文件夹中的 ZLTech 中的对应位置，数字识别可以使用我们提供的模板，也可自己根据前面的教程采集数字图片模板并存放到 models 文件夹中的 template_num 中。



3.将 AI_Functions、models、ZL_SDK 文件夹都保存到 openmv 的盘符中，然后在 openmv ide 中将 z_main.py 文件存到 openmv 中，脱机运行即可，也可在线运行，不过要注意不要让小车跑了。

三、运行效果

代码运行后，我们主要通过语音来控制小车。

语音识别模块：



以下为对应的语音及功能：

颜色识别、颜色跟随、人脸识别、人脸跟随、分辨人脸、二维码识别、数字识别、追小球、循迹模式、前进、后退、左转、右转、停止、复位

语音模块的基本用法：你说'小灵小灵'或'你好小灵'可以将它唤醒，它会随机回复你，如果你将它唤醒后，在 20 秒内未用语音控制它，它会再次进入睡眠状态，等待你将它唤醒；

注意我们的语音功能都是需要在将它唤醒之后才控制它的。有些功能是可以使用不同的话语来控制的。你将它唤醒后，说'前进'或'往前走'，则小车会往前走；说'循迹模式'，则小车会开始循黑线。

唤醒词	小灵小灵
命令词	对应功能
颜色识别	颜色识别
颜色跟随	颜色跟随
人脸识别	人脸识别
人脸跟随	人脸跟随
分辨人脸	分辨人脸
二维码识别	二维码识别
数字识别	数字识别
追小球	小车追小球
循迹模式	巡线小车
前进、往前走	小车前进
后退、向后走、倒着走	小车后退
左转、向左转	小车左转
右转、向右转	小车右转
停止、停下	小车停止
复位	云台复位

【本课小结】：

本课将本套件基本所有的功能做了一个整合，通过运行本程序来调用其他示例程序，然后通过语音识别模块来控制。

注意在运行程序之前根据教程做好准备工作，不然程序可能出错。

项目篇

第 1 课 识别颜色并与 arduino 串口通信夹取木块

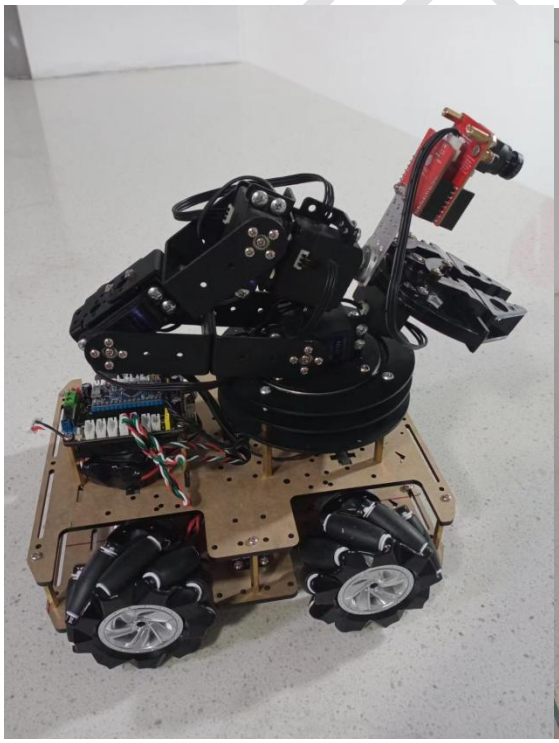
【课程目标】：

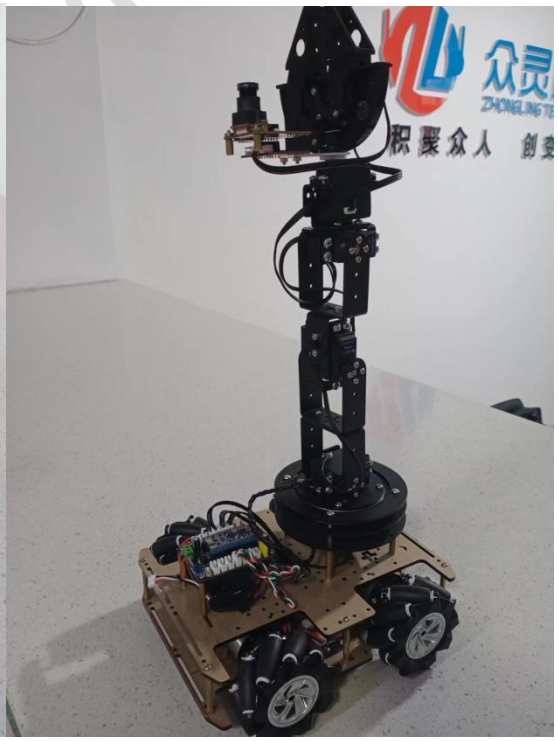
本课学习使用 openmv 识别颜色并通过串口发送数据到 arduino 中来让机械臂夹取对应颜色的木块到指定位置。

注意：本节主要是演示 openmv 与 arduino 间的串口通信功能，硬件方面使用的是我们店里的产品，如果大家购买了我们对应的产品，也可以实际操作一下的，如果没有购买的话，也可以根据代码和下面的讲解来深入了解 openmv 与 arduino 之间的串口通信，其他控制板的话也是类似的用法。

【课前准备】：

Openmv、C5 轮式底盘车+总线机械臂、三合一底板+arduino 核心板、usb 数据线





【本课内容】：

一、代码

1.openmv 端代码

```
'''''
颜色识别夹取木块
红色木块放在正前方，绿色木块放在小车左侧，蓝色木块放在小车右侧
红绿蓝三种颜色可以同时识别，还可自行添加其他颜色阈值来识别其他颜色
# 注意：环境光会对识别有影响，找比较合适的环境来运行此程序。
颜色阈值采集方法：
方法一：
    1.可以在‘工具’-->‘机器视觉’-->‘阈值编辑器’-->‘帧缓冲区’中采集，
    2.拖动 6 个滑块来调整图像，直到要采集的颜色都呈白色，其他都呈黑色为止，
    3.采集好之后将 LAB 阈值复制到代码中的阈值处使用即可。
方法二：
    1.可以在右侧‘帧缓冲区’和‘直方图’的’LAB 色彩空间‘配合采集
    2.先在帧缓冲区中用鼠标左键框出要采集的颜色区域，
    3.然后在直方图中的 LAB 色彩空间中，分别复制 L、A、B 的最小值和最大值到代码中的颜色阈值元组中使用即可
'''
import sensor, time
from pyb import UART
import utime
import json


target_color_appear_count = 0

# 设置颜色阈值，此处设置红绿蓝三种颜色的阈值
thresholds = {'red': (62, 41, -9, 113, 24, 98),       # 红色阈值
              'green': (68, 26, -57, -18, 100, -81),  # 绿色阈值
              'blue': (70, 6, -76, 36, -20, -96)      # 蓝色阈值
             }
# 各颜色对应要执行的动作组
actions = {'start': '$DGT:3-3,1!',
           'red': '$DGT:4-10,1!',
           'green': '$DGT:11-19,1!',
           'blue': '$DGT:20-28,1!'
            }

# 初始化函数
def init_setup():
    global sensor, clock, uart3            # 设置为全局变量
    sensor.reset()                         # 初始化感光元件
     sensor.set_pixformat(sensor.RGB565)     # 设置感光元件图像色彩格式为 RGB565 (RGB565 为彩图，
GRAYSCALE 为灰度图)
```

```
41.     sensor.set_framesize(sensor.QVGA)       # 设置感光元件分辨率大小为 QVGA (QVGA 是 320x240)
42.     sensor.skip_frames(10)                  # 跳过一些帧，使以上的设置生效
43.     sensor.set_auto_gain(False)             # 关闭自动增益。在颜色识别中，需要关闭自动增益
44.     sensor.set_auto_whitebal(False)         # 关闭白平衡。白平衡是默认开启的，在颜色识别中，需要关闭白平
    衡
45.     clock = time.clock()                    # 创建时钟对象
46.     uart3 = UART(3, 115200)                 # 初始化串口 3，设置波特率为 115200
47.     print('初始化完成')
48.     utime.sleep(2)
49.     uart3_send(actions['start'])
50.     utime.sleep(2)
51.
52.
53. # 串口接收数据
54. def uart3_recv():
55.     recv_data = uart3.read()
56.     print('recv data: %s' % recv_data)
57.     return recv_data
58.
59. # 串口发送数据
60. def uart3_send(send_data):
61.     global uart3
62.     uart3.write(send_data)
63.
64.
65. # 查找图像中最大的色块
66. def find_max(blobs):
67.     max_size = 0
68.     for blob in blobs:
69.         if blob[2] * blob[3] > max_size:
70.             max_blob = blob
71.             max_size = blob[2] * blob[3]
72.     return max_blob
73.
74. # 主函数
75. def main():
76.     global target_color_appear_count
77.     while 1:                                # 无限循环
78.         clock.tick()                        # 更新 FPS 帧率时钟
79.         img = sensor.snapshot()             # 拍一张照片并返回图像
80.         # 循环 3 种颜色阈值，在图像中查看是否有对应颜色并做一些操作，thresholds 需为字典格式
81.         for threshold in thresholds:
82.             blobs = img.find_blobs([thresholds[threshold]])                # 在图像中查找目标颜色区
    域
```

```
83.             if blobs:                                                  # 当查找到目标颜色区域后
84.                 max_blob = find_max(blobs)                             # 查找图像中最大的目标色块
85.                 img.draw_rectangle(max_blob[0:4])                      # 将找到的目标颜色区域用矩形框出来
86.                 img.draw_cross(max_blob[5], max_blob[6])               # 在目标颜色区域的中心点处画十字
87.                 target_area = max_blob[2] * max_blob[3]                # 面积
88.                 print('area is  ', target_area)                        # 打印面积
89.
90.                 if 2000 < target_area < 6000:                          # 面积合适时
91.                     target_color_appear_count += 1
92.
93.                 if target_color_appear_count > 5:                      # 识别到的面积合适时的次数大于 5 次时串口发数据
94.                     if threshold == 'red':
95.                         uart3_send(actions['red'])                     # 串口将识别到的颜色对应的动作组发出去
96.                         utime.sleep(15)                                # 延时以便让动作组执行完成
97.                     elif threshold == 'green':
98.                         uart3_send(actions['green'])                   # 串口将识别到的颜色对应的动作组发出去
99.                         utime.sleep(18)                                # 延时以便让动作组执行完成
100.                    elif threshold == 'blue':
101.                        uart3_send(actions['blue'])                    # 串口将识别到的颜色对应的动作组发出去
102.                        utime.sleep(18)                                # 延时以便让动作组执行完成
103.                    print('uart3 send color is ', threshold)
104.                    target_color_appear_count = 0
105.
106.
107. # 程序入口
108. if __name__ == '__main__':
109.     init_setup()                            # 执行初始化函数
110.     try:                                    # 异常处理
111.         main()                              # 执行主函数
112.     except:
113.         pass
```

2.arduino 端代码可参考资料中的 ‘Carbot(无 C6)-智能车出厂程序 20210925’

二、代码使用方法

第一步：将 openmv 的代码保存到 openmv 中

第二步：将 arduino 端代码上传到 arduino 中

第三步：下载动作组到 arduino 中

第四步：将 openmv 用一根总线接到舵机上，打开控制板上的电源开关，等待 2 秒后机械臂会先弯下去，然后将红色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车正前方；将绿色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车左侧；将蓝色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车右侧。

三、运行效果

将 openmv 用一根总线接到舵机上，打开控制板上的电源开关，等待 2 秒后机械臂会先弯下去，然后将红色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车正前方；将绿色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车左侧；将蓝色木块放到小车前方 10cm 处，机械臂会夹取木块放到小车右侧。

注意：如果颜色识别的有问题，可以根据前面的教程修改 openmv 端代码中的对应颜色阈值即可。使用时，摄像头画面中尽量不要有干扰的颜色，手有可能被识别为红色，大家要注意一下。

四、知识点

关于代码中 UART 的使用可以参考官方文档：

https://docs.singtown.com/micropython/zh/latest/openmvcam/library/pyb.UART.html#pyb-uart

【本课小结】：

本课学习了使用 openmv 识别颜色并通过串口发送数据到 arduino 中来让机械臂夹取对应颜色的木块到指定位置。

注意使用时摄像头画面中不要有干扰的颜色。

注意：这个 openmv 示例代码同样适用于我们的多合一 32 控制板和三合一 51 控制板，只需上传轮式车的代码到 32 或 51 中就可以了。

众灵科技

附录：

Zlink 模块百度云链接：

http://pan.baidu.com/s/1gfajYt9 密码：6njz

总线双路驱动百度云链接：

https://pan.baidu.com/s/1Uo5guj0sEc77DT85rO6yqA 提取码：xfeq

蓝牙模块链接：

https://pan.baidu.com/s/19Qp7JKrmafWQZKwN5nrqQg 提取码：iylc

售后服务：

【产品核对】：收到产品时请及时对照发货清单检查配件是否齐全，以及快递运输过程中产品是否损坏等现象，如有问题请及时联系淘宝客服人员（注：自收到货起三日内没有反应问题，视为产品收货正常）。
【资料索取】：学习资料以百度云网盘链接形式发送，收到物品时请联系客服索取。
【组装接线】：散件组装接线时请按照教程进行组装接线，因操作不正确导致产品损坏的，一切后果由自己承担！如有问题请及时联系客服或售后技术人员。
【品质服务】：提供全程有关产品技术支持（可通过电话/QQ/微信等）。
【售后时间】：9：00---20：00（周一到周六）
【技术支持】：联系淘宝客服

关注微信公众号
获取更多产品信息